Università di Brescia

DIPARTIMENTO DI INGEGNERIA MECCANICA E INDUSTRIALE

CORSO DI LAUREA IN INGEGNERIA DELL'AUTOMAZIONE INDUSTRIALE

Design and Implementation of a Motion Control Architecture for an Omnidirectional Robotic Platform with System Assembly and Validation

Relatore:
Chiar.mo Prof. Antonio Visioli

Correlatore:
Chiar.mo Prof. Antonio Pedro Aguiar

Laureando:
Alessandro Vigorelli Matr. 735847

ANNO ACCADEMICO 2025-2026

Contents

List of Figures **ix**

List of Tables **xi**

Acronym Table **xiii**

1 Introduction **1**
1.1 Problem Statement and Motivation 1
1.2 Maneuverability and Constraints 2
1.3 Objectives and Contributions 2

2 Background **5**
2.1 Robot@Factory 4.0 Context 5
2.2 Previous Robotic Framework 7
2.3 Limitations of the Previous Architecture 8
2.4 Positioning of the Present Thesis 9

3 State of the Art and Robot Modelling **11**
3.1 Omnidirectional Mobile Robots 11
3.2 Kinematic Model of the Omnidirectional Robot 12
3.2.1 Kinodynamic and Dynamic Modelling 13
3.3 Model Predictive Control for Omnidirectional Robots 15
3.4 From Previous Work to the Present Control Architecture 18

4 Hardware Design and System Architecture **19**
4.1 Mechanical Redesign and Optimization 20
4.2 Actuators, Sensors and Electronic System 23
4.3 Electrical Design and Connection Architecture 26
4.3.1 Physical Layout and Layer Organization 26
4.3.2 ESP32 Pinout and Wiring Specification 27
4.3.3 Pin Selection Justification 27

5 Physical Implementation and Assembly **31**
5.1 Additive Manufacturing: 3D Printing Phase . . . 31
5.1.1 Printing Statistics and Material Consumption . . . 31
5.2 Mechanical Assembly . . . 33
5.2.1 Layer 1: Traction and Power Distribution . . . 33
5.2.2 Power Management: DC-DC Conversion . . . 33
5.2.3 Layer 2: Logic and Navigation . . . 33
5.3 Assembly Workflow and System Validation . . . 34
5.3.1 Wiring and Electrical Pre-testing . . . 34
5.3.2 Firmware Configuration and Signal Analysis . . . 34
5.3.3 Encoder Integration and Signal Conditioning . . . 35

6 Firmware Development and Experimental Validation **37**
6.1 Development Environment: ESP32 and PlatformIO . . . 37
6.1.1 Configuration: the platformio.ini file . . . 38
6.2 Software Architecture and Project Structure . . . 38
6.2.1 Role of Main Components . . . 39
6.3 Hardware Abstraction: robot_config.h . . . 40
6.4 Experimental Procedures and Main Loop . . . 41
6.4.1 System Initialization and Setup . . . 41
6.4.2 Encoder Validation and Tick Counting . . . 42
6.4.3 Open-Loop Response and First-Order Approximation . . . 43
Linearization and Order Reduction . . . 43
6.4.4 Double Step Test for PID Tuning . . . 43
6.4.5 Closed-Loop . . . 50
Controller Design and Pole-Zero Cancellation . . . 50
Feedforward Compensation and Real-world Constraints . . . 51
Experimental Validation and Tuning Strategy . . . 51

7 Omnicar Platform, ROS Integration and High-Level Software Architecture **55**
7.1 High-Level Software Architecture: From ROS1 Noetic to ROS2 Jazzy . . . 55
7.1.1 Migration and Local Implementation . . . 55
7.1.2 Workspace Structure . . . 56
7.1.3 Experimental Validation: ROS2 Integration and Control Chain . . . 59
Test 1: Encoder Monitoring and Open-Loop Velocity Control . . . 59
Test 2: Manual Control using a Joystick Controller . . . 61
Test 3: Localization and Lidar Integration . . . 62
Test 4: Waypoint Tracking Validation with the `go_to_point` Controller . . . 64
7.1.4 Final Considerations on the Integrated ROS2 Chain . . . 67

8 Motion Control Design and Experimental Evaluation **69**
8.1 Mathematical Model of the OMNICAR Platform 70
8.2 Evaluation Protocol and Metrics 70
8.3 Moving Reference Proportional Controller 70
8.3.1 Control Principle 70
8.3.2 Test Parameters and Results 72
8.4 Linear Segment Controller 83
8.4.1 Control Principle 83
8.4.2 Test Parameters 84
8.5 Nonlinear Model Predictive Contouring Controller 99
8.5.1 Geometric Path Representation 99
8.5.2 Predictive Model 100
8.5.3 Contouring and Lag Errors 101
8.5.4 Cost Function 101
8.5.5 Implementation Structure 102
8.5.6 Limitations and Possible Improvements 103
8.5.7 Test Parameters and Results 104
8.5.8 Experimental Figures 107
8.5.9 Comparison Between Linear Segment and MPCC 121

9 Conclusions **133**

Acknowledgements **137**

List of Figures

1.1 Examples of application scenarios for autonomous mobile robots. 2

2.1 Graph-based representation of the Robot@Factory 4.0 shop floor used for navigation and task planning [3]. 6
2.2 Omnidirectional robotic platform proposed for the Robot@Factory 4.0 framework [3]. 7
2.3 System architecture proposed for the Robot@Factory 4.0 competition . . . 8

3.1 Mecanum wheel motion principle 11
3.2 General receding-horizon MPC structure. At each sampling instant, the current state is used to initialize a finite-horizon prediction. The optimizer computes a sequence of future commands, but only the first one is applied to the robot [12]. 15

4.1 Full assembly model for 3D printing 19
4.2 Battery Case 20
4.3 Lower layer assembled 20
4.4 Upper layer components 21
4.5 Bottom layer assembled 22
4.6 Battery 5000mAh 11.1V 3S1P 24
4.7 ESP32 25
4.8 Driver MDD10A 25
4.9 Upper layer main components 26
4.10 Detailed Connection Architecture designed in EasyEDA 29

5.1 Slicing preview of OMNICAR components in PrusaSlicer 32
5.2 Physical assembly of the OMNICAR lower layer 34
5.3 Oscilloscope capture of the PWM control signal (50% Duty Cycle) 35
5.4 Pull-down resistor implementation for signal conditioning 35
5.5 Real application of Pull-down resistor 35
5.6 Solution for Encoder signal 35
5.7 Conditioned encoder signals (approx. 3.4V peak-to-peak) 36
5.8 Fully assembled OMNICAR platform ready for field testing 36

6.1 First Test PWM in 50Hz 44

6.2 Open-loop no-load test (MA_NL) at a sampling frequency of 50 Hz . . . 45
6.3 Open-loop velocity profile for Motor 2 at 50 Hz sampling rate 46
6.4 Detailed view of the open-loop response at 50 Hz for system identification 47
6.5 Open-loop test results at 100 Hz showing the initial linear noise increase . 48
6.6 Closed-Loop system - Block diagram 50
6.7 Closed-loop control architecture with feedforward - Block diagram 51
6.8 Full closed-loop PI test at 15 rad/s (200 Hz) with Feedforward compensation 53
6.9 PI closed-loop response at 20 rad/s (100 Hz) showing increased measurement jitter 53
6.10 PI closed-loop tracking at 20 rad/s (50 Hz) without Feedforward 54

7.1 rqt_graph showing the active nodes and topics during the encoder test, including reference velocity and measured encoder feedback. 59
7.2 rqt_graph showing the active nodes and topics during the joystick test . . 61
7.3 Comparison between the real robot experiment and the corresponding odometry visualization in RViz. 63
7.4 diagram of the test executed 64
7.5 Position control results. 65
7.6 Position error results. 66
7.7 Controller Architecture 67

8.1 XY map for the MR_R01 scenario. 75
8.2 Target tracking and error results for the MR_R01 scenario. 76
8.3 XY map for the P_R01 scenario. 77
8.4 Target tracking and error results for the P_R01 scenario. 78
8.5 XY map for the P_R02 scenario. 79
8.6 Target tracking and error results for the P_R02 scenario. 80
8.7 XY map for the LIN_01 scenario. 87
8.8 Target tracking and error results for the LIN_01 scenario. 88
8.9 XY map for the LIN_02 scenario. 89
8.10 Target tracking and error results for the LIN_02 scenario. 90
8.11 XY map for the LIN_03 scenario. 91
8.12 Target tracking and error results for the LIN_03 scenario. 92
8.13 XY map for the LIN_04 scenario. 93
8.14 Target tracking and error results for the LIN_04 scenario. 94
8.15 XY map for the LIN_05 scenario. 95
8.16 Target tracking and error results for the LIN_05 scenario. 96
8.17 XY map for the MPC_01 scenario. 107
8.18 Target tracking and error results for the MPC_01 scenario. 108
8.19 XY map for the MPC_02 scenario. 109
8.20 Target tracking and error results for the MPC_02 scenario. 110
8.21 XY map for the MPC_03 scenario. 111
8.22 Target tracking and error results for the MPC_03 scenario. 112
8.23 XY map for the MPC_04 scenario. 113

8.24 Target tracking and error results for the MPC_04 scenario. 114
8.25 XY map for the MPC_05 scenario. 115
8.26 Target tracking and error results for the MPC_05 scenario. 116
8.27 XY map for the MPC_06 scenario. 117
8.28 Target tracking and error results for the MPC_06 scenario. 118
8.29 Overlapped xy trajectories for the LIN-05 and LIN-06 comparison. 125
8.30 Contouring and lag error metrics for the LIN-05 and LIN-06 comparison. . 126
8.31 Overlapped xy trajectories for the MPC-05 and MPC-06 comparison. . . . 127
8.32 Contouring and lag error metrics for the MPC-05 and MPC-06 comparison. 128
8.33 Overlapped xy trajectories for the LIN-05 and MPC-06 comparison. . . . 129
8.34 Contouring and lag error metrics for the LIN-05 and MPC-06 comparison. 130

List of Tables

4.1 ESP32 Complete Pinout for OMNICAR Motion Control 28

5.1 OMNICAR 3D Printing Parameters and Statistics 32

8.1 Moving Reference controller: main test parameters. 72
8.2 Moving Reference controller: experimental results. 73
8.3 Linear Segment controller: configuration parameters of the selected tests. 85
8.4 Linear Segment controller: experimental results of the selected tests. . . . 85
8.5 Main characteristic parameters of the selected MPCC tests. 105
8.6 Compact notation used for vector parameters. 105
8.7 Nonlinear MPCC controller: experimental results of the selected tests. . . 106
8.8 Common-segment comparison between selected Linear Segment and MPCC tests. 122

Acronym Table

Acronym	Meaning
AMR	Autonomous Mobile Robot
DC	Direct Current
ESP32	Espressif Systems 32-bit microcontroller
FF	Feedforward
GPIO	General Purpose Input/Output
IAE	Integral Absolute Error
IMC	Internal Model Control
LIDAR	Light Detection and Ranging
MPC	Model Predictive Control
MPCC	Model Predictive Contouring Control
OCP	Optimal Control Problem
PI	Proportional-Integral
PID	Proportional-Integral-Derivative
PWM	Pulse Width Modulation
RMSE	Root Mean Square Error
ROS	Robot Operating System
ROS2	Robot Operating System 2
TF	Transform Frames
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
FL	Front Left
FR	Front Right
RL	Rear Left
RL	Rear Right

Chapter 1

Introduction

1.1 Problem Statement and Motivation

Modern industrial automation is increasingly moving towards flexible and autonomous systems. Within this framework, autonomous mobile robots represent a key enabling technology for reducing repetitive manual transportation tasks, improving material flow, and increasing the adaptability of production and logistics environments [1, 2]. Among mobile robotic platforms, omnidirectional robots are particularly attractive because they can move in any planar direction without first changing their orientation. This capability is especially relevant in constrained and structured environments such as factory floors, warehouses, hospitals, and service-delivery scenarios, as shown in Figure 1.1. In these contexts, autonomous mobile robots can transport materials, tools, medicines, samples, or goods while navigating through corridors, shelves, workstations, machines, and human operators. Omnidirectional motion is advantageous because it enables lateral corrections, precise docking maneuvers, and flexible target approach without requiring complex multi-step rotations. Similar requirements also appear in educational and research scenarios such as Robot@Factory 4.0, where robots must localize themselves, interpret task-level information, and navigate between warehouses and processing machines in a simplified but realistic industrial environment [3, 4].

(a) Laser-guided vehicle for pallet handling in an industrial environment [5].

(b) Autonomous delivery robot operating in a pedestrian environment [6].

Figure 1.1: Examples of application scenarios for autonomous mobile robots.

This thesis documents the mechanical assembly, hardware architecture design, and implementation of motion-control strategies, including geometric and predictive approaches, for the **OMNICAR** platform. The robotic vehicle is designed as a research and validation platform for autonomous navigation in structured workspaces, with particular attention to the integration between mechanical design, embedded motor control, ROS2-based software architecture, localization, and high-level path-following control.

1.2 Maneuverability and Constraints

Traditional differential drive robots face significant maneuverability challenges in confined industrial environments. This limitation stems from their mechanical design, which prevents the vehicle from moving sideways without first performing a rotation. In robotics, this physical restriction is classified as a **non-holonomic constraint** [1, 2]. Mathematically, this occurs when the number of controllable degrees of freedom is lower than the total degrees of freedom of the workspace. Consequently, these robots cannot follow any arbitrary trajectory instantaneously, often requiring complex maneuvers to navigate. This project addresses these issues by utilizing the **OMNICAR** platform, which allows for fluid and omnidirectional motion.

1.3 Objectives and Contributions

The primary objective of this research is the redesign, development, and experimental validation of an omnidirectional robotic platform, referred to as OMNICAR. The work

extends a previously developed framework into a robust, modular, and research-oriented system, suitable for advanced studies in motion control and autonomous navigation.

The main contributions of this thesis are outlined as follows:

- **Mechanical Redesign and Structural Optimization:** The robotic platform was completely redesigned using a computer-aided design approach. The resulting structure is a two-layer modular chassis, characterized by increased dimensions, improved mechanical robustness, and enhanced accessibility. Particular attention was devoted to structural integrity, component arrangement, and usability, resulting in a design better suited for both experimental validation and iterative development.

- **Hardware Integration and System Layout:** A systematic integration of all core hardware components was carried out, including the Raspberry Pi 5, ESP32 microcontroller, LIDAR sensor, Pi Camera, and motor drivers. Dedicated mechanical supports and optimized placement strategies were developed in order to maximize sensor visibility, reduce electromagnetic interference, and facilitate maintenance operations.

- **Electrical System Design and Implementation:** The electrical architecture was designed from the ground up, including the definition of wiring schematics, signal routing, and microcontroller pin allocation. A clear separation between power and logic domains was implemented to ensure reliability and signal integrity. Additionally, a structured cable management strategy, based on modular connectors, custom routing, and soldered interfaces, was introduced to improve system organization and ease of debugging.

- **Prototyping and Physical Realization:** The platform was physically realized through additive manufacturing techniques and a structured assembly process. The final system integrates mechanical, electrical, and electronic subsystems into a cohesive and fully functional robotic platform, validated through preliminary hardware testing.

Following the completion of the hardware platform, the research was extended to the software and control domains:

- **Embedded Firmware Development:** A modular firmware architecture was developed for the ESP32, enabling real-time motor actuation, encoder acquisition, and communication with high-level systems. The implementation supports deterministic control loops and provides a reliable interface between hardware and software layers.

- **Migration to ROS2 and Software Architecture Redesign:** The high-level software framework was migrated from ROS1 to ROS2 Jazzy, leading to a more scalable and maintainable architecture. The resulting system is organized into modular components for communication, odometry, localization, and control, improving extensibility and system integration.

- **Low-Level Control Design and Experimental Identification:** The dynamic behavior of the actuation system was experimentally characterized, and a closed-loop velocity control strategy based on Proportional-Integral (PI) control was implemented. This contribution ensures stable and accurate wheel-speed regulation, forming the basis for higher-level control algorithms.
- **Development and Experimental Evaluation of Motion-Control Strategies:** Several high-level motion-control algorithms were implemented and validated on the physical platform, including Moving Reference control, Linear Segment path following, and Nonlinear Model Predictive Contouring Control (MPCC). Their performance was assessed through experimental campaigns, providing a comparative analysis in terms of tracking accuracy, robustness, and path-following efficiency.

In summary, this thesis delivers a complete and experimentally validated robotic system, encompassing mechanical design, electrical and embedded development, ROS2-based software architecture, and advanced motion-control strategies. The resulting platform constitutes a flexible experimental testbed for future research in omnidirectional robotics and autonomous navigation. The complete source code, ROS2 workspace, configuration files, and experimental support scripts developed during this work are available in the project repository [7].

Chapter 2

Background

2.1 Robot@Factory 4.0 Context

The development of the OMNICAR platform is framed within the context of autonomous mobile robots for industrial environments. In particular, the Robot@Factory 4.0 competition represents a simplified but realistic factory-floor scenario, where autonomous robots must transport materials between warehouses and processing machines while navigating safely inside a structured workspace [3].

Unlike earlier editions of the Robot@Factory competition, Robot@Factory 4.0 does not rely on continuous physical guiding lines or perimeter walls delimiting the full shop floor. Instead, the robot must operate in a more flexible environment, where localization, navigation, task execution, and software integration become central aspects of the system. The competition environment includes input and output warehouses, processing machines, navigation corridors, and artificial landmarks such as ArUco markers or physical poles, which support robot localization and task-level decision making.

A graph-based representation of the competition environment is shown in Fig. 2.1. In this representation, the shop floor is discretized into nodes and connections that describe the admissible navigation paths between warehouses, machines, and relevant intermediate positions. This type of map is useful because it transforms the physical layout of the factory into a structure that can be exploited by planning and navigation algorithms. The robot can therefore reason about feasible routes, target locations, and transitions between different areas of the workspace.

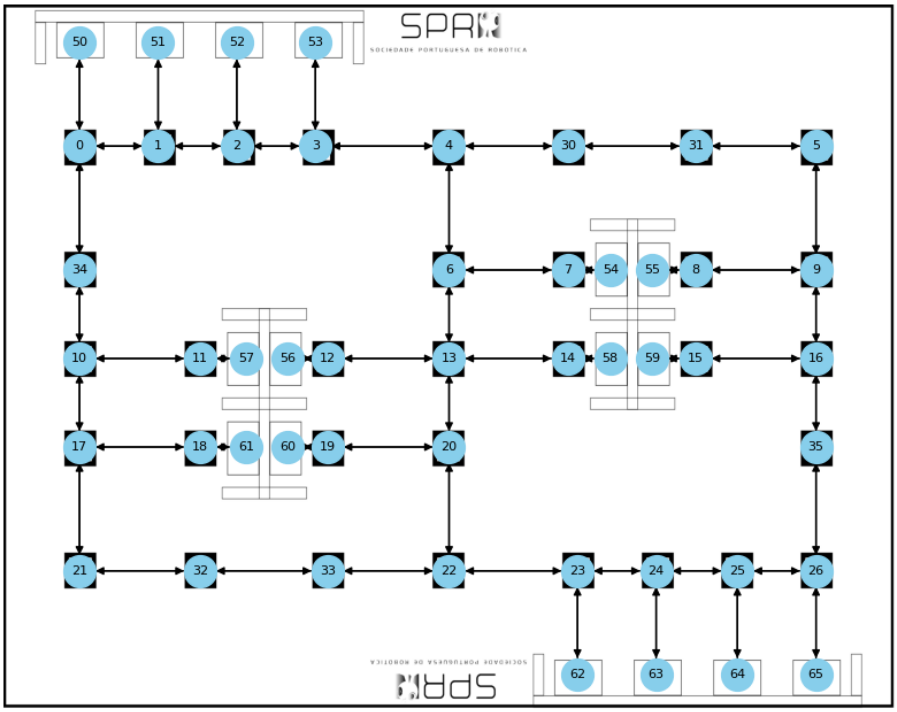


Figure 2.1: Graph-based representation of the Robot@Factory 4.0 shop floor used for navigation and task planning [3].

The reference robotic platform proposed in the previous framework is shown in Fig. 2.2. The platform adopts a four-wheel omnidirectional configuration, allowing planar motion without requiring a preliminary rotation of the robot body. This capability is particularly suitable for the Robot@Factory 4.0 scenario, where the robot must perform precise movements in narrow areas, align with warehouses and machines, and correct its position during navigation. The same general motivation is preserved in the present thesis, which starts from the previous framework and focuses on redesigning, implementing, and experimentally validating an improved OMNICAR platform for motion-control research.

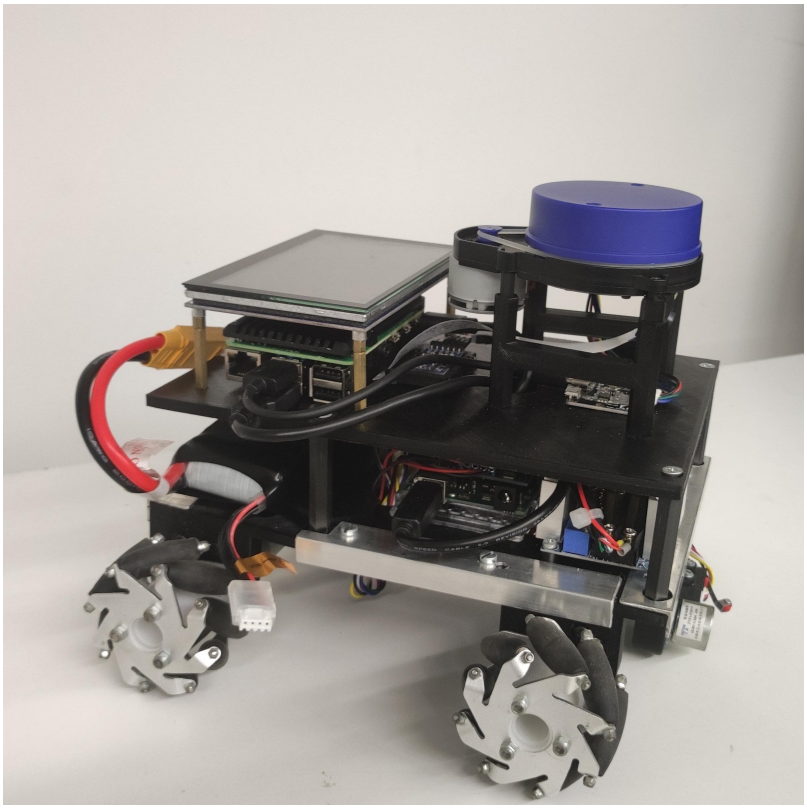


Figure 2.2: Omnidirectional robotic platform proposed for the Robot@Factory 4.0 framework [3].

This setup is representative of flexible industrial automation, where autonomy is not limited to low-level motion but requires the integration of mechanical design, embedded control, localization, path planning, and high-level software architecture. For this reason, Robot@Factory 4.0 provides a meaningful benchmark for developing and validating autonomous mobile robots intended to operate in structured industrial-like environments.

2.2 Previous Robotic Framework

The work developed before this thesis provided a complete robotic framework targeting Robot@Factory 4.0. The framework was conceived as a modular and open architecture, including the main hardware, firmware, and software components required to build and operate an autonomous mobile robot in the competition environment [3].

From the hardware viewpoint, the previous platform adopted a four-wheel omnidirectional configuration. This mechanical choice is particularly suitable in constrained industrial-like spaces because the robot can move in any planar direction without requiring a preliminary body rotation. The system also included onboard processing units, motor drivers, and sensing modules for perception and localization.

At firmware level, the architecture provided the low-level interface between onboard computer and actuators. This layer handled wheel command execution, encoder feedback acquisition, and communication with high-level ROS nodes, effectively acting as the bridge between physical actuation and supervisory control, this is shown in Figure 2.3.

At software level, the framework was organized into modular blocks for odometry, localization, planning, and control. The ROS-based organization enabled a clean separation between functionalities, improving maintainability and easing future extension of individual modules [8].

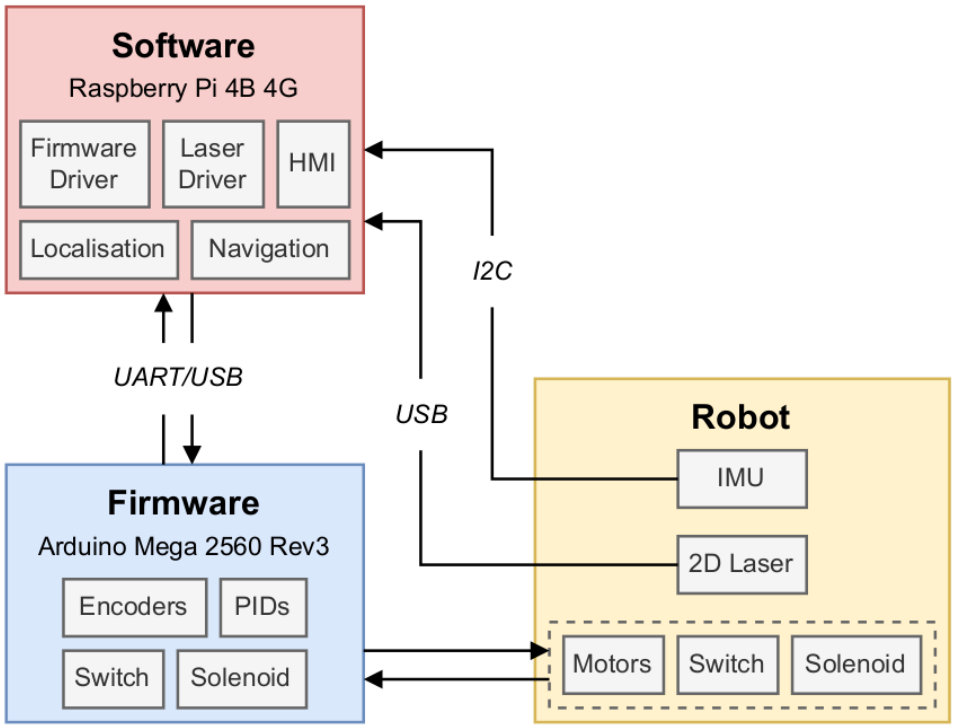


Figure 2.3: System architecture proposed for the Robot@Factory 4.0 competition

2.3 Limitations of the Previous Architecture

Although the previous framework represented a complete and functional baseline for Robot@Factory 4.0, several limitations motivated the development of the present work. The previous implementation was heavily oriented toward competition requirements.

Consequently, a redesign was possible to transform the platform into an experimental version reusable for motion control research. The software stack was based on ROS1.

While ROS1 supports modular development, ROS2 provides a more suitable communication infrastructure for distributed robotic systems, improved maintainability, and better long-term support [8]. The mechanical and electronic layout still required improvements

to simplify maintenance, increase robustness, and provide cleaner integration of the Raspberry Pi, ESP32, motor drivers, battery, lidar, and auxiliary components.

2.4 Positioning of the Present Thesis

Starting from the previous framework, this thesis focuses on redesigning, implementing, and experimentally validating an improved OMNICAR platform. The work can be interpreted as a continuation and extension of the existing system, with the objective of transforming it into a more robust and flexible research platform for omnidirectional motion control.

The main extensions introduced in this thesis are:

- mechanical redesign and physical reconstruction of the robotic platform;
- reorganization of electronic layout and wiring architecture;
- development and validation of ESP32 firmware for low-level motor control;
- migration of the high-level software architecture from ROS1 to ROS2 Jazzy;
- experimental validation of odometry, localization, teleoperation, and waypoint regulation;
- implementation and comparison of different motion-control strategies, including proportional moving-reference control, linear-segment path following, and nonlinear Model Predictive Contouring Control (MPCC) [9, 10].

Chapter 3

State of the Art and Robot Modelling

3.1 Omnidirectional Mobile Robots

Mobile robots can be classified according to the kinematic constraints imposed by their locomotion system. Differential-drive robots, for example, are non-holonomic systems: they cannot instantaneously move sideways and must usually combine rotations and forward motions to reach a desired pose. This limitation can reduce maneuverability in confined environments.

Omnidirectional robots overcome this limitation by using special wheel configurations, such as omni wheels or mecanum wheels, which allow independent control of the planar velocity components, see Figure 3.1 [1, 2].

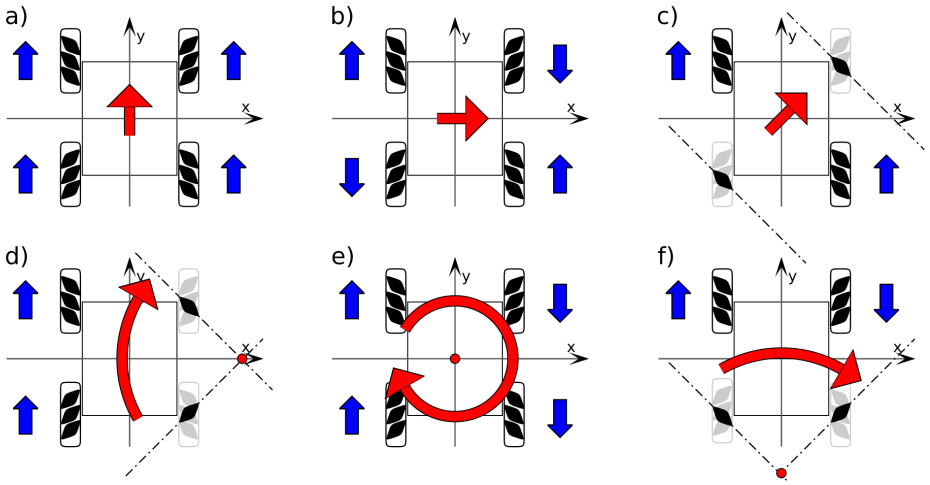


Figure 3.1: Mecanum wheel motion principle. [11].

In ideal conditions, an omnidirectional robot can control its longitudinal velocity,

lateral velocity, and angular velocity around the vertical axis. This makes the platform particularly suitable for factory-like environments, where the robot must navigate through narrow corridors, align with machines or warehouses, and execute precise lateral corrections.

Let the planar pose of the robot be defined as

$$\mathbf{q} = \begin{bmatrix} x & y & \theta \end{bmatrix}^T, \tag{3.1}$$

where x and y describe the position of the robot in the global reference frame, while θ is the yaw angle.

The corresponding velocity vector in the global frame is

$$\dot{\mathbf{q}} = \begin{bmatrix} \dot{x} & \dot{y} & \dot{\theta} \end{bmatrix}^T. \tag{3.2}$$

For an omnidirectional robot, the control objective is usually expressed in terms of the desired planar velocity

$$\mathbf{u} = \begin{bmatrix} v_x & v_y & \omega \end{bmatrix}^T, \tag{3.3}$$

where v_x and v_y are the translational velocity components and ω is the angular velocity.

3.2 Kinematic Model of the Omnidirectional Robot

A simplified kinematic model assumes that the robot velocity can be directly commanded. Under this assumption, the planar motion of the robot can be written as

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} \tag{3.4}$$

This model corresponds to three decoupled integrators and is useful for high-level control design [1, 2]. It assumes that the lower-level velocity controller is sufficiently fast and accurate, so that the commanded velocity is rapidly reached by the physical robot.

However, the velocity components used by the high-level controller are usually expressed in the global frame, while the wheel velocities must be computed in the robot frame. The transformation between the global translational velocity and the robot-frame velocity can be obtained through a planar rotation:

$$\begin{bmatrix} v \\ v_n \end{bmatrix} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix} \tag{3.5}$$

The velocity vector in the robot frame can then be mapped into individual wheel-speed references through the inverse kinematic matrix of the omnidirectional platform.

For a four-wheel omnidirectional platform, the body-frame velocity vector is defined as

$$\mathbf{u}_b = \begin{bmatrix} v_x & v_y & \omega \end{bmatrix}^T, \tag{3.6}$$

where v_x and v_y are the longitudinal and lateral velocity components expressed in the robot frame, and ω is the angular velocity around the vertical axis. The corresponding wheel angular velocity references can be written as

$$\boldsymbol{r}_w = \begin{bmatrix} r_{w,1} & r_{w,2} & r_{w,3} & r_{w,4} \end{bmatrix}^T = \mathbf{T}^{-1}\mathbf{u}_b, \tag{3.7}$$

where $\mathbf{T}^{-1}$ is the inverse kinematic matrix of the platform.

For the adopted wheel ordering

$$\boldsymbol{r}_w = \begin{bmatrix} r_{\mathrm{FL}} & r_{\mathrm{FR}} & r_{\mathrm{RL}} & r_{\mathrm{RR}} \end{bmatrix}^T,$$

and assuming a symmetric four-wheel omnidirectional configuration, the inverse kinematic matrix can be expressed as

$$\mathbf{T}^{-1} = \frac{1}{r}\begin{bmatrix} 1 & -1 & -(L_x + L_y) \\ 1 & 1 & (L_x + L_y) \\ 1 & 1 & -(L_x + L_y) \\ 1 & -1 & (L_x + L_y) \end{bmatrix}, \tag{3.8}$$

where r is the wheel radius, while L_x and L_y are the longitudinal and lateral distances from the robot center to each wheel. In this thesis, these parameters are later implemented in the firmware configuration as $r = 0.06$ m, $L_x = 0.185$ m, and $L_y = 0.220$ m, as discussed in Chapter 6.

3.2.1 Kinodynamic and Dynamic Modelling

The purely kinematic prediction used in Section 8.5 is effective when the robot moves at moderate speed, but it assumes that commanded body velocities are applied instantaneously. In practice, the low-level loop (PI + feedforward) implemented on the ESP32 and the motor–wheel dynamics introduce a finite transient response, as discussed in Chapter 6.

Starting from the inverse kinematic relation introduced in Eq. (3.8), the body velocity command is mapped into wheel-speed references as

$$\boldsymbol{r}_w = \mathbf{T}^{-1}\mathbf{u}. \tag{3.9}$$

A more realistic kinodynamic approximation models each wheel channel with a first-order closed-loop response:

$$\frac{\Omega_{w,i}(s)}{R_{w,i}(s)} = \frac{1}{\tau_i s + 1}, \qquad i = 1, \ldots, 4. \tag{3.10}$$

where $R_{w,i}(s)$ is the wheel-speed reference, $\Omega_{w,i}(s)$ is the measured wheel speed, and τ_i is the effective time constant of the embedded closed-loop channel. In time domain:

$$\dot{\omega}_{w,i}(t) = \frac{1}{\tau_i}(r_{w,i}(t) - \omega_{w,i}(t)). \tag{3.11}$$

The actual wheel-speed vector $\boldsymbol{\omega}_w$ is then mapped back to robot velocity through forward kinematics:

$$\begin{bmatrix} v_x^{\text{act}} \\ v_y^{\text{act}} \\ \omega^{\text{act}} \end{bmatrix} = \mathbf{T}\boldsymbol{\omega}_w. \tag{3.12}$$

This model captures the dominant actuator lag and improves consistency between predicted and measured motion, especially near saturation or during aggressive path-progression tuning. The same modelling choice is coherent with first-order wheel dynamics commonly adopted in MPC-based omnidirectional tracking formulations [9].

3.3 Model Predictive Control for Omnidirectional Robots

Model Predictive Control (MPC) is an optimization-based control strategy in which the control action is obtained by repeatedly solving a finite-horizon Optimal Control Problem (OCP). Instead of computing the command only from the instantaneous error, as in many reactive or feedback controllers, MPC uses a model of the system to predict how the state will evolve in the near future. At each sampling instant, the controller evaluates a sequence of possible future inputs and selects the one that minimizes a cost function while satisfying the constraints imposed on the system [10, 12].

The central idea of MPC is the receding-horizon principle. At time step k, the current state of the system is measured or estimated and used as the initial condition of the prediction. The optimizer then computes an optimal sequence of future inputs over a finite horizon. Although a complete sequence is obtained, only the first input is applied to the real plant. At the next sampling instant, the state is updated and the optimization is solved again. This repeated re-planning mechanism gives MPC its feedback nature and allows the controller to react to disturbances, modelling errors, and changes in the reference.

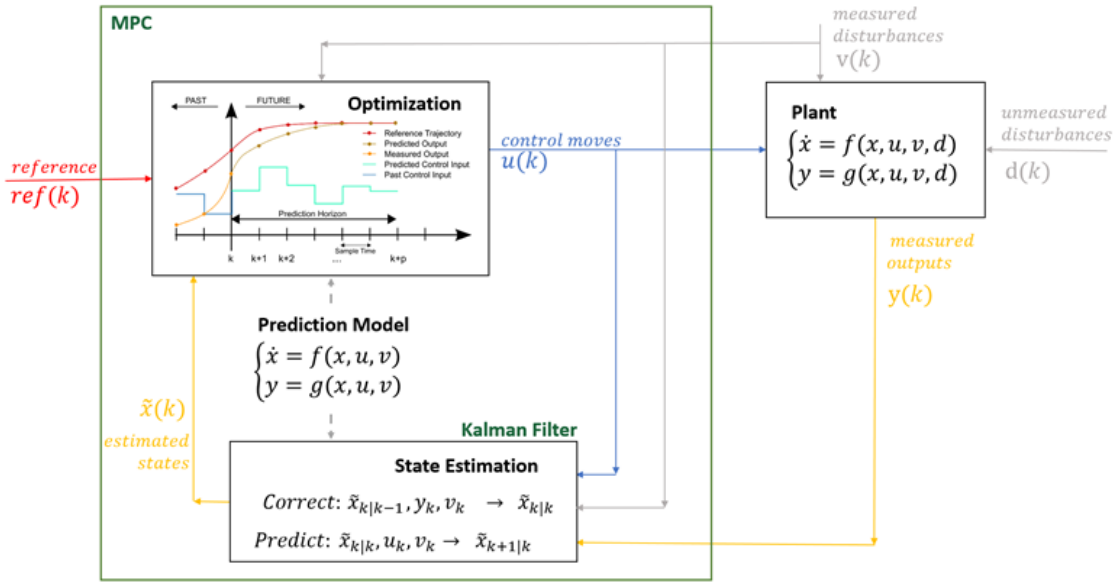


Figure 3.2: General receding-horizon MPC structure. At each sampling instant, the current state is used to initialize a finite-horizon prediction. The optimizer computes a sequence of future commands, but only the first one is applied to the robot [12].

A general discrete-time predictive model can be written as

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k), \tag{3.13}$$

where $\mathbf{x}_k \in R^n$ is the state vector, $\mathbf{u}_k \in R^m$ is the control input, and $f(\cdot)$ is the model used

to predict the system evolution. Depending on the application, this model may be linear, nonlinear, kinematic, dynamic, or identified from experimental data. In mobile robotics, the model is often selected as a compromise between accuracy and computational cost. A more detailed model can improve prediction quality, but it also increases the complexity of the optimization problem and may reduce real-time feasibility.

In a standard trajectory-tracking formulation, the MPC cost function penalizes the deviation between the predicted state and the desired reference, together with the magnitude and variation of the control input. A typical finite-horizon cost can be expressed as

$$J = \sum_{i=0}^{N-1} \left(\|\mathbf{x}_{k+i} - \mathbf{x}_{k+i}^{\text{ref}}\|_{\mathbf{Q}}^2 + \|\mathbf{u}_{k+i}\|_{\mathbf{R}}^2 + \|\Delta\mathbf{u}_{k+i}\|_{\mathbf{S}}^2 \right), \tag{3.14}$$

where N is the prediction horizon. The matrix $\mathbf{Q}$ weights the tracking error, $\mathbf{R}$ penalizes large control inputs, and $\mathbf{S}$ penalizes abrupt command variations. The term $\Delta\mathbf{u}_{k+i}$ is usually defined as the difference between two consecutive commands and is particularly important in robotic applications, where aggressive changes in velocity references may excite actuator dynamics or produce unstable motion.

The optimization problem is solved subject to the prediction model and to possible constraints:

$$\begin{aligned} \mathbf{x}_{k+i+1} &= f(\mathbf{x}_{k+i}, \mathbf{u}_{k+i}), \\ \mathbf{x}_k &= \hat{\mathbf{x}}_k, \\ \mathbf{u}_{\min} &\leq \mathbf{u}_{k+i} \leq \mathbf{u}_{\max}, \\ \Delta\mathbf{u}_{\min} &\leq \Delta\mathbf{u}_{k+i} \leq \Delta\mathbf{u}_{\max}. \end{aligned} \tag{3.15}$$

The first constraint enforces the system dynamics, while the others represent physical or design limitations. Additional constraints can also be introduced to account for obstacle avoidance, workspace limits, safety margins, or terminal conditions. This ability to include constraints explicitly is one of the main advantages of MPC with respect to classical control strategies.

For omnidirectional mobile robots, MPC is especially attractive because the platform can command independent planar velocity components [2, 9]. A simplified high-level input vector can be written as

$$\mathbf{u}_k = \begin{bmatrix} v_{x,k} & v_{y,k} & \omega_k \end{bmatrix}^T, \tag{3.16}$$

where $v_{x,k}$ and $v_{y,k}$ are the translational velocity components and ω_k is the yaw rate. These inputs can be directly constrained according to actuator limits, maximum admissible velocity, acceleration bounds, or smoothness requirements. This is relevant because an omnidirectional robot can generate lateral corrections without first aligning its body with the path, but the corresponding wheel speeds may still saturate if the requested motion is too aggressive.

The predictive model used inside MPC may be purely kinematic or may include additional dynamic effects. A kinematic model is computationally efficient and is often sufficient for moderate-speed motion, especially when a lower-level velocity controller

tracks the commanded velocity accurately. However, real robots are affected by motor dynamics, actuator saturation, wheel slip, delays, and imperfect odometry. If these phenomena are not represented in the prediction model, the optimizer may produce commands that are optimal for the simplified model but less effective on the real system. Therefore, the design of an MPC controller always involves a trade-off between model fidelity, numerical complexity, and robustness to modelling errors.

A further distinction must be made between trajectory tracking and path following. In trajectory tracking, the reference is time-indexed: at each future time instant, the robot is expected to be close to a desired pose on a predefined trajectory. The controller therefore minimizes the error between the predicted pose and a reference pose associated with the same time step. This formulation is suitable when the timing of the motion is prescribed in advance.

In path following, instead, the reference is primarily geometric. The objective is to move along a path while keeping the lateral deviation small, but the exact timing along the curve may be adjusted by the controller. This is useful when the robot must follow a shape accurately while adapting its speed to actuator limits, curvature, localization errors, or transient conditions. In this context, Model Predictive Contouring Control (MPCC) introduces a curvilinear path coordinate, usually denoted as γ, and separates the tracking error into two components: contouring error and lag error.

The contouring error measures the lateral distance between the robot and the reference path, while the lag error measures the displacement along the tangent direction of the path. If $\mathbf{p}_k$ is the predicted robot position, $\mathbf{p}_r(\gamma_k)$ is the reference point sampled at the curvilinear coordinate γ_k, and $\mathbf{t}(\gamma_k)$, $\mathbf{n}(\gamma_k)$ are the local tangent and normal vectors, the two errors can be expressed as

$$e_{l,k} = (\mathbf{p}_k - \mathbf{p}_r(\gamma_k))^T \mathbf{t}(\gamma_k), \qquad e_{c,k} = (\mathbf{p}_k - \mathbf{p}_r(\gamma_k))^T \mathbf{n}(\gamma_k). \tag{3.17}$$

The contouring error $e_{c,k}$ is directly related to geometric path accuracy, while the lag error $e_{l,k}$ indicates whether the robot is ahead of or behind the virtual reference point. By weighting these two quantities differently, the controller can prioritize lateral accuracy, path progression, or a compromise between the two.

In an MPCC formulation, the path coordinate can be included in the predicted state:

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \psi_k & \gamma_k \end{bmatrix}^T, \tag{3.18}$$

and the control vector can be augmented with the virtual path advancement:

$$\mathbf{u}_k = \begin{bmatrix} v_{x,k} & v_{y,k} & \omega_k & \dot{\gamma}_k \end{bmatrix}^T. \tag{3.19}$$

In this way, the controller does not only decide how the robot should move, but also how fast the reference point should advance along the path. A positive reward on $\dot{\gamma}_k$ can be introduced to encourage forward progress, while penalties on contouring error, lag error, heading error, control effort, and command increments maintain the motion accurate and smooth.

The controller developed in this thesis follows this MPCC formulation and is specialized for an omnidirectional robot following a planar path. The predictive controller balances geometric accuracy, path advancement, command effort, and smoothness within the same finite-horizon optimization problem. The detailed implementation, including the adopted prediction model, cost function, sampled control search, and experimental validation, is presented in Section 8.5.

3.4 From Previous Work to the Present Control Architecture

Previous work on the OMNICAR platform showed that MPC can significantly improve the motion performance of an omnidirectional robot. In particular, the use of a predictive controller allows the robot to operate closer to its actuation limits while maintaining stable trajectory tracking.

The present thesis builds on this direction but focuses on a different development stage. Instead of starting from an already finalized platform, this work includes the complete redesign, assembly, firmware development, ROS2 migration, and experimental validation of the robotic system. The final control chapter then compares progressively more advanced motion-control strategies, starting from simpler proportional and linear-segment controllers and moving toward a nonlinear Model Predictive Contouring Controller.

This progression provides an incremental validation path. First, the robot is made mechanically and electronically reliable. Then, the low-level velocity-control layer is validated. After that, ROS2 communication, odometry, localization, and teleoperation are tested. Finally, high-level controllers are implemented and compared experimentally using trajectory-tracking and path-following metrics.

Chapter 4

Hardware Design and System Architecture

The OMNICAR platform, shown in Figure 4.1, is built upon a modular two-layer chassis, specifically designed to optimize the arrangement of sensors and processing units. Unlike traditional designs that strictly separate power and control electronics, this layout focuses on operational convenience and sensor efficiency.

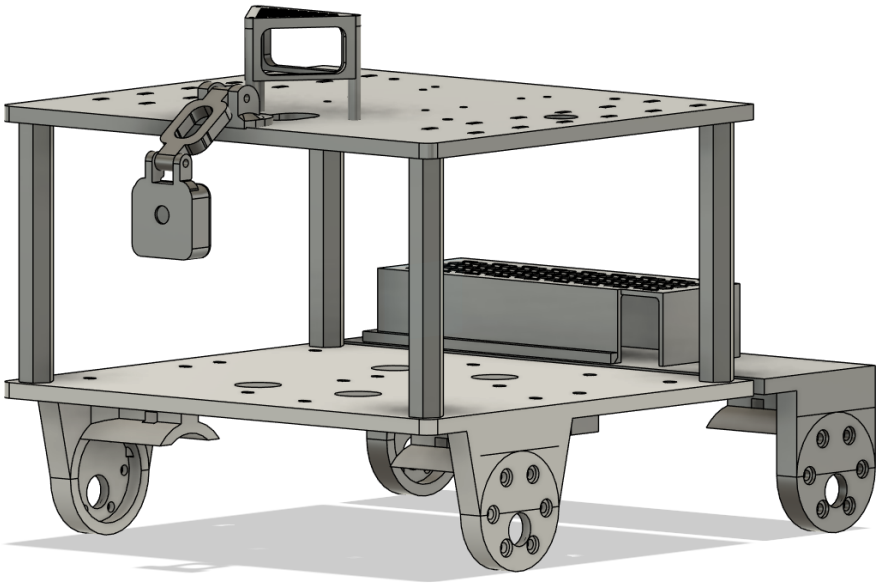


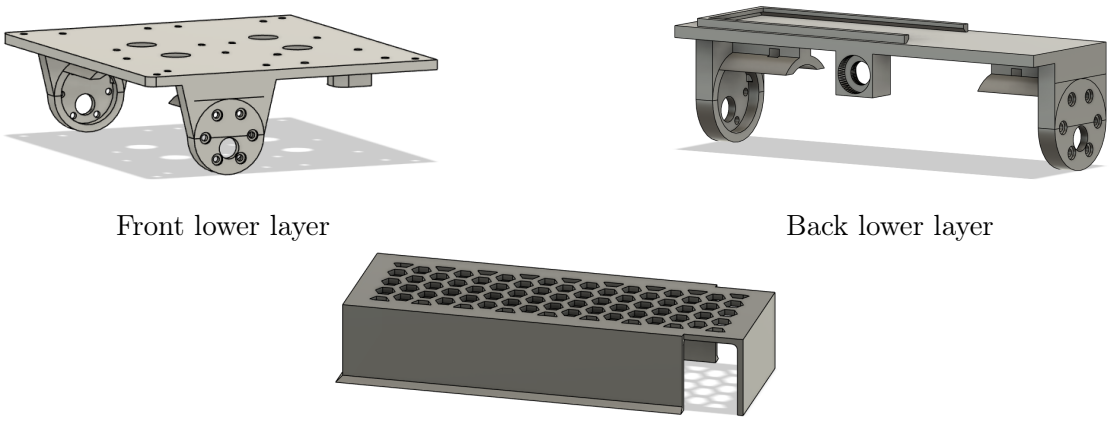
Figure 4.1: Full assembly model for 3D printing

4.1 Mechanical Redesign and Optimization

The **lower layer** houses the DC-DC converter, ESP32 microcontroller, motor drivers, and the battery, ensuring a low center of gravity for better dynamic stability. In this revision, the **motor mounts** have been significantly raised and reinforced to improve structural rigidity and better accommodate the mechanical stresses during high-torque maneuvers. Additionally, the mounts now feature dedicated slots for **nylon zip ties**, providing a secondary fastening method to ensure the motors remain perfectly aligned and secure.

Furthermore, a custom **battery Case** has been designed and implemented. It utilizes a **sliding rail mechanism** that allows the Case to snap into place securely without the need for screws. This design choice significantly improves the robot's usability, allowing for rapid battery replacement during testing phases.

The **lower layer** houses the ESP32 microcontroller, motor drivers, and battery, keeping the center of mass low and reducing the wiring distance to the motors.



Front lower layer

Back lower layer

Battery Case

Figure 4.2: Battery Case

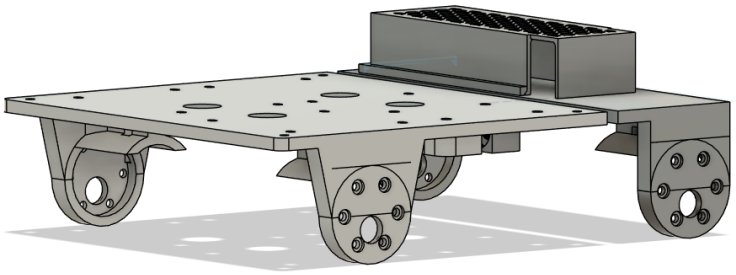
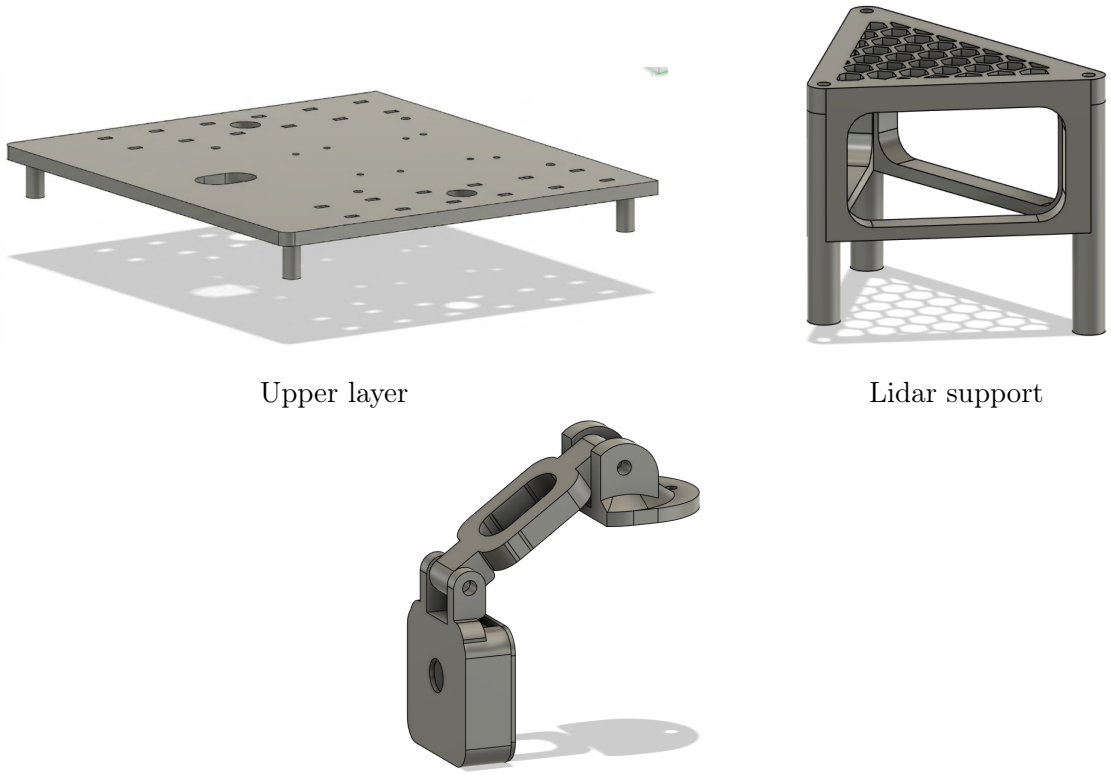


Figure 4.3: Lower layer assembled

The **upper layer** is dedicated to the high-level navigation suite. It provides an elevated platform for the LIDAR and the Pi Camera, ensuring a clear line of sight for environment mapping and obstacle detection. Additionally, this placement allows the Raspberry Pi 5 to be located directly adjacent to the LIDAR, simplifying the USB/Serial connection and reducing electromagnetic interference in high-speed data transmission [1, 13].



Upper layer

Lidar support

Stand pi camera

Figure 4.4: Upper layer components

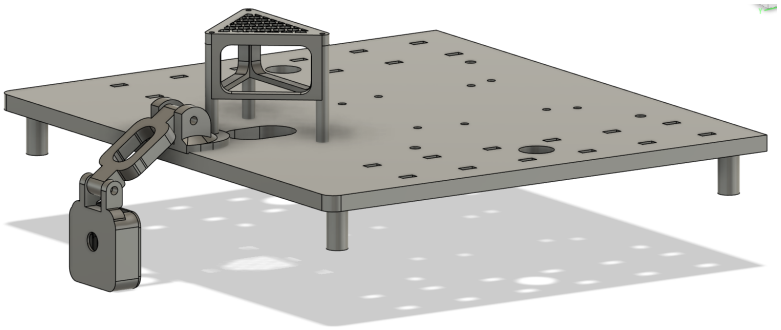


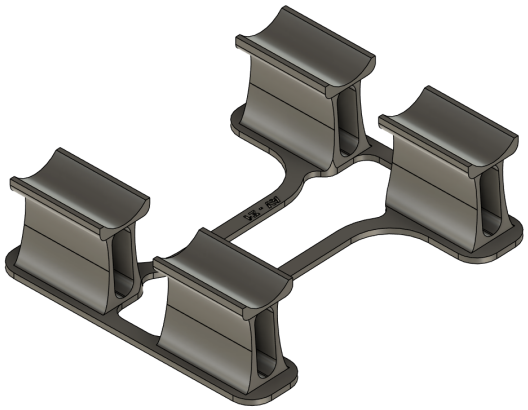
Figure 4.5: Bottom layer assembled

To facilitate the development and validation of the control algorithms, a custom-designed support stand was developed for the OMNICAR platform. This component proved to be **essential during the debugging and initial testing phases**, particularly for verifying the response of the four independent motors.

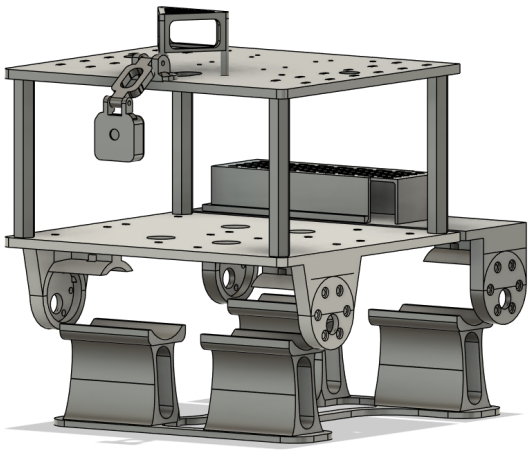
By elevating the chassis, the stand allowed the wheels to rotate freely in a "no-load" condition. This setup was extremely useful for:

- Calibrating the high-resolution encoders without the interference of ground friction.
- Validating the mapping of PWM signals to the MDD10A motor drivers.
- Identifying potential hardware asymmetries or wiring issues before conducting field tests.

The mechanical design follows an **ergonomic and minimalist approach**. A key feature of the structure is the integration of a hollow central grip. This design choice was not only functional, providing an easy point of handle for moving the robot and stand together, but also strategically **economical**. By incorporating this empty space, the total volume of 3D-printing filament was significantly reduced, leading to faster production times and lower material costs without compromising the structural integrity required to support the robot's weight.



Stand



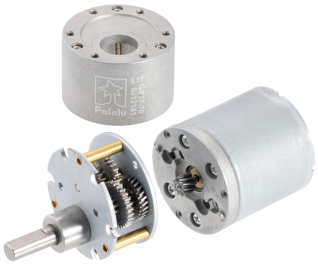
Omnicar and Stand assembled

4.2 Actuators, Sensors and Electronic System

The motion system is powered by four **Pololu 37Dx73L mm** metal gearmotors. These actuators are equipped with integrated 64 CPR quadrature encoders which, through the gear ratio, provide a high-resolution feedback of approximately **1200 counts per revolution**. This level of precision is fundamental for maintaining accurate odometry and ensuring stable trajectory tracking.

To manage the complex power requirements of both high-level processing and low-level actuation, the robot integrates a robust power distribution stage. A high-capacity **300W 20A DC-DC buck converter** is utilized to step down the battery voltage, providing a stabilized power rail for the most demanding components like encoders, **Raspberry Pi 5** and the **LIDAR** sensor.

To manage the complex power requirements of both high-level processing and low-level actuation, the robot's electrical architecture integrates a robust power distribution stage. The system is primary energized by a **Combo Battery 5000mAh 11.1V 3S1P** LiPo battery, chosen for its high energy density and discharge rate capabilities. To interface this source with the onboard electronics, a high-capacity **300W 20A DC-DC buck converter** is utilized to step down the battery voltage. This regulation provides a stabilized power rail for the most demanding components, including the high-resolution encoders, the **Raspberry Pi 5** central processing unit, and the **LIDAR** sensor, ensuring operational stability even during high-torque motor maneuvers.



Pololu Metal DC Gearmotors

DC-DC Buck Converter

Figure 4.6: Battery 5000mAh 11.1V 3S1P

The control architecture is divided into two main functional blocks:

- **Low-Level Control:** The core of this subsystem is an **ESP32-WROOM** microcontroller, see fig. 4.7, chosen for its dual-core architecture and dedicated hardware peripherals. The ESP32 handles the real-time execution of the control loop, utilizing its hardware timers for precise **PWM generation** and its **Pulse Counter (PCNT)** modules for efficient, high-frequency encoder reading [14].

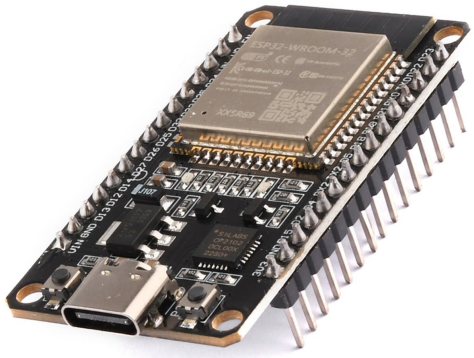
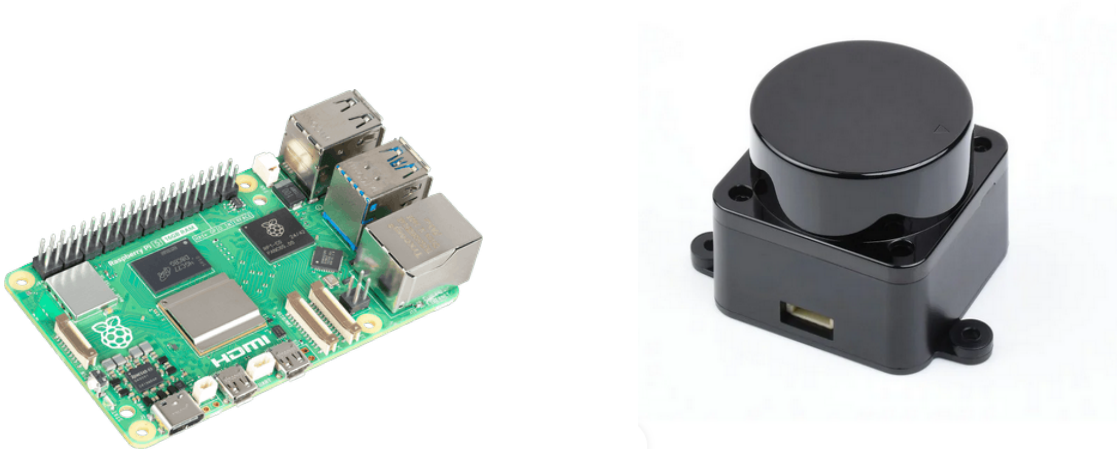


Figure 4.7: ESP32

- **Motor Actuation:** The microcontroller interfaces with two **MDD10A** dual-channel motor drivers, see fig. 4.8. This setup allows for independent control of each wheel's speed and direction through a PWM/DIR interface, creating a closed-loop system where the ESP32 simultaneously commands the motors and monitors wheel position.

Figure 4.8: Driver MDD10A

- **High-Level Navigation:** A **Raspberry Pi 5** [15], serves as the primary computational unit, processing data from the **LIDAR** to perform environment mapping, subsequently communicating target velocities to the ESP32. The main components of the upper layer are shown in Figure 4.9



Raspberry Pi Lidar

Figure 4.9: Upper layer main components

4.3 Electrical Design and Connection Architecture

The design of the entire electrical system for the OMNICAR platform was developed using the **EasyEDA online editor**. Thanks to the integrated component libraries, it was possible to model the power interfaces of the motor drivers and the logical connections of the microcontroller, ensuring a high degree of fidelity between the theoretical design and the final hardware.

The result of this phase is the **general electrical schematic** of the platform, illustrated in Figure 4.10, which serves as the blueprint for the entire control and power distribution system. The schematic is structured to isolate the high-current power lines intended for the motors from the 3.3V and 5V logic signals required for the control logic and sensors. This separation is crucial to prevent cross-talk and ensure the stability of the feedback signals from the encoders during the robot's maximum acceleration phases.

4.3.1 Physical Layout and Layer Organization

The physical organization of the hardware follows a layered logic to optimize space and weight distribution:

- **Lower Layer (Base):** This level constitutes the electromechanical heart of the robot. It houses the **ESP32 DevKit**, the four **MDD10A** drivers, and the power distribution system. Placing these at the base allows for a lower center of gravity

and reduces the length of the wiring to the motors, minimizing electromagnetic interference (EMI).

- **Upper Layer (Sensor Deck):** To ensure a field of view free from obstacles, the **Raspberry Pi 5**, the **LIDAR** sensor, and the **Pi Camera** are installed on the upper structure. This configuration protects the processing units from the heat generated by the drivers and provides an optimal perspective for navigation and vision algorithms.

4.3.2 ESP32 Pinout and Wiring Specification

The **ESP32-WROOM** microcontroller was selected for its high-performance architecture, capable of simultaneously managing non-linear control loops and high-frequency encoder readings. The connection configuration was designed to maximize the use of dedicated hardware peripherals, such as PWM (LEDC) and Pulse Counters (PCNT). The detailed mapping of these connections is provided in Table 4.1.

4.3.3 Pin Selection Justification

The pin assignment was guided by specific technical constraints to ensure system reliability:

1. **High-Resolution Feedback:** With encoders providing 1200 pulses per revolution, it was necessary to use pins compatible with the **PCNT** hardware peripheral. Pins 34-39 were specifically chosen for the rear encoders because, as "Input-Only" pins, they lack internal pull-up/pull-down circuitry that could interfere with the signal, thus guaranteeing digital integrity without the risk of output conflicts.

2. **PWM Generation:** The **LEDC** peripheral was utilized to control the MDD10A drivers. The assigned pins allow for independent, hardware-timed PWM signals, which are essential for maintaining a deterministic response in holonomic kinematics.

3. **Communication Integrity:** Predefined hardware **UART** pins were reserved for data exchange with the Raspberry Pi 5, supporting baud rates up to 115200 bps for real-time transmission of velocities and poses.

Table 4.1: ESP32 Complete Pinout for OMNICAR Motion Control

Function	**GPIO**	**Signal Type**	**Project Description**
Main Power Supply	VIN	Power (+5V)	Main DC input
Common Ground	GND	GND	Common return
Motor FL PWM	13	Output (LEDC)	Front-Left speed control
Motor FL DIR	12	Output	Front-Left direction
Motor FR PWM	14	Output (LEDC)	Front-Right speed control
Motor FR DIR	27	Output	Front-Right direction
Motor RL PWM	26	Output (LEDC)	Rear-Left speed control
Motor RL DIR	25	Output	Rear-Left direction
Motor RR PWM	33	Output (LEDC)	Rear-Right speed control
Motor RR DIR	32	Output	Rear-Right direction
Encoder FL (A/B)	16 / 17	Input (PCNT)	Front-Left feedback
Encoder FR (A/B)	5 / 18	Input (PCNT)	Front-Right feedback
Encoder RL (A/B)	19 / 21	Input (Only)	Rear-Left feedback
Encoder RR (A/B)	22 / 23	Input (Only)	Rear-Right feedback

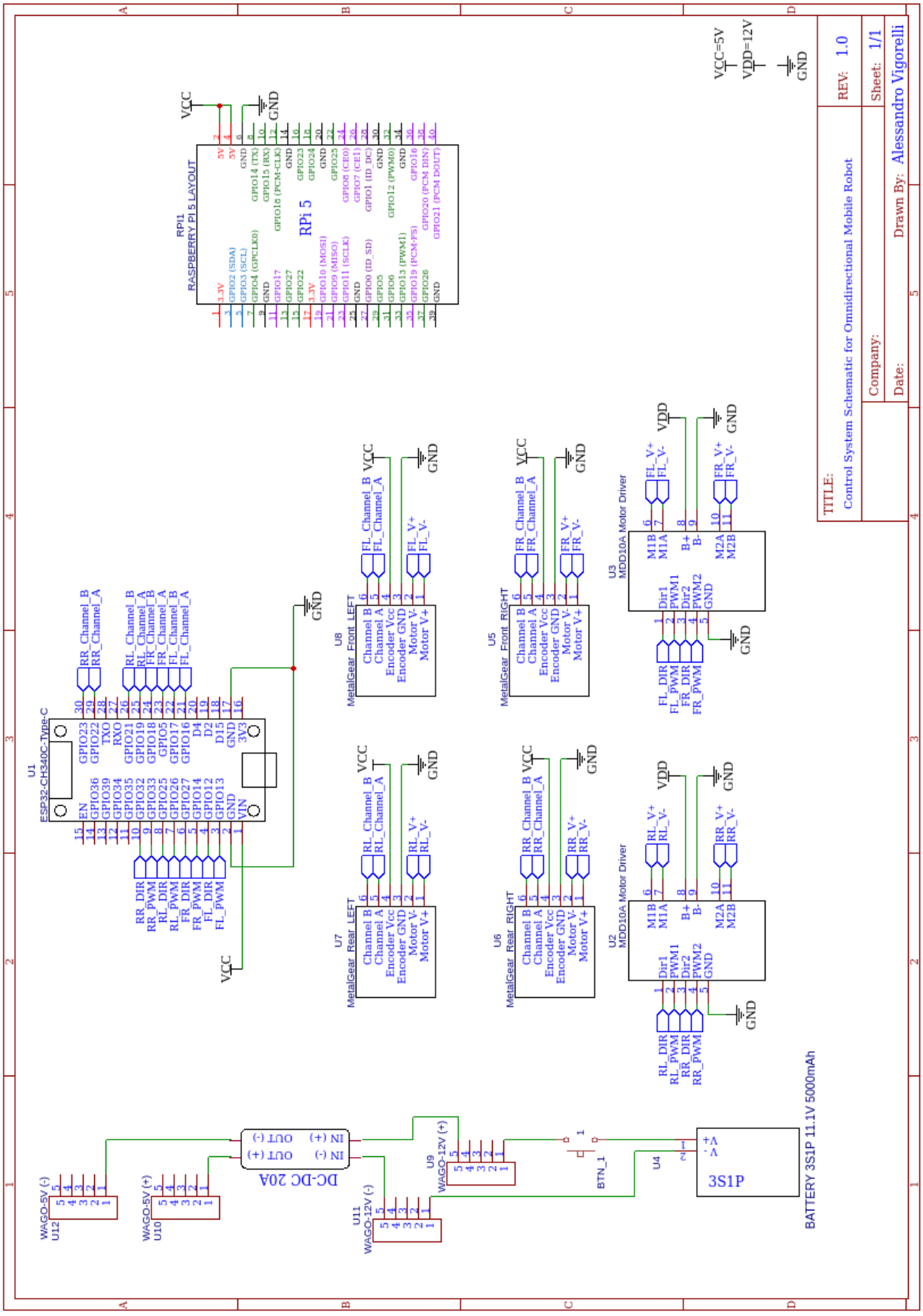


Figure 4.10: Detailed Connection Architecture designed in EasyEDA

Chapter 5

Physical Implementation and Assembly

This chapter details the transition from the digital design environment to the physical realization of the OMNICAR platform. The process encompasses the additive manufacturing of the structural components, the integration of the power electronics, and a rigorous multi-stage assembly and validation workflow.

5.1 Additive Manufacturing: 3D Printing Phase

The structural integrity of OMNICAR is primarily based on components manufactured through Fused Deposition Modeling (FDM) [16]. All custom parts were produced using a **Prusa MK3S+** 3D printer, known for its reliability and dimensional accuracy.

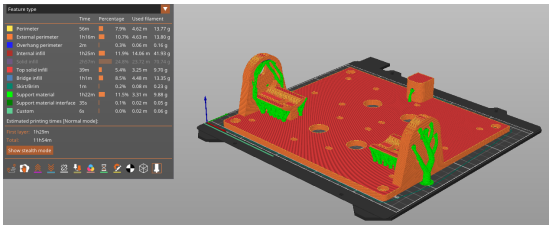
The material selected for the entire chassis and support system was **PLA (Polylactic Acid)**, chosen for its ease of printing, rigidity, and minimal warping. To balance structural strength with weight optimization, a standard **infill density of 15%** was applied to most components. However, for the primary structural elements—specifically the **Layer 1 Front**, **Layer 1 Back**, and **Layer 2**—the infill was increased to 25% to ensure higher mechanical resistance to the stresses generated by the four DC motors and the overall payload.

5.1.1 Printing Statistics and Material Consumption

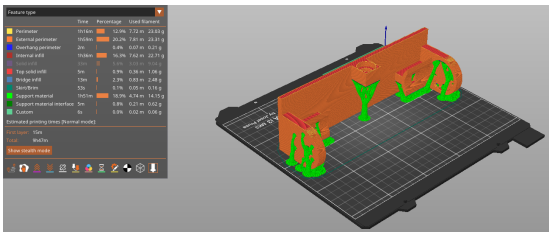
The manufacturing process was planned to optimize time and material usage. Table 5.1 summarizes the printing time and material consumption (in grams) for the main structural modules, as derived from the G-code analysis.

Table 5.1: OMNICAR 3D Printing Parameters and Statistics

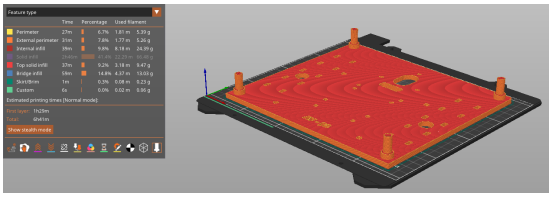
Component	**Printing Time**	**Material [g]**	**Infill [%]**
Layer 1 Front	11h 54m	115g	25%
Layer 1 Back	9h 47m	95g	25%
Layer 2	6h 41m	70g	25%
Battery Case and Support	7h 11m	55g	15%
Structural Pillars (Set)	6h 29m	88g	15%
Stand	19h 44m	160g	15%
TOTAL	**61h 46m**	**583g**	—



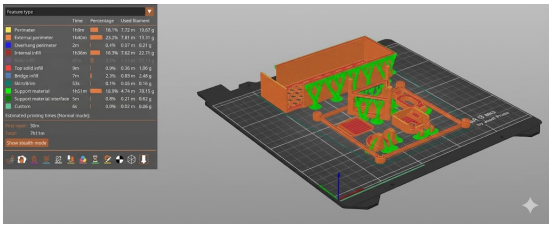
Front lower layer



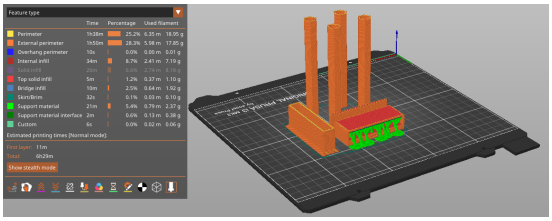
Back lower layer



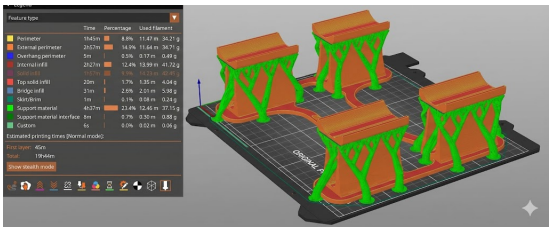
Upper Layer



Battery Case and Supports



Structural pillars and Wago Case



Robot Test Stand

Figure 5.1: Slicing preview of OMNICAR components in PrusaSlicer

5.2 Mechanical Assembly

The robot's construction follows a modular "bottom-up" methodology, designed to optimize internal volume distribution and ensure high maintainability. The structure is organized into two primary functional layers connected by rigid vertical pillars.

5.2.1 Layer 1: Traction and Power Distribution

The lower deck (Layer 1) serves as the structural foundation and houses the high-power components. During the initial assembly phase, the motors and core components were positioned without wiring to verify dimensional fits, see Figure 5.2.

- **Suspension and Chassis Coupling:** The connection between the front and rear sections is achieved through a structural bolt passing through flanged bearings. This assembly creates a pivot point that acts as a passive suspension system, allowing the frame to adapt to surface irregularities.
- **Actuation System:** Motor leads were extended by soldering custom wires insulated with heat-shrink tubing to reach the drivers efficiently.

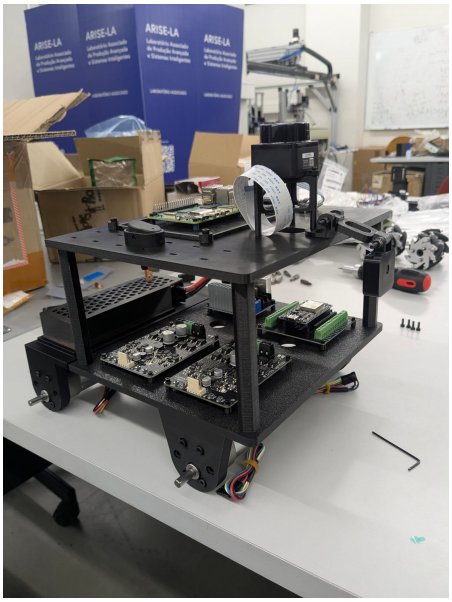
5.2.2 Power Management: DC-DC Conversion

To provide a stable logic supply, a **DC-DC 9A 300W Step-Down Buck Converter** was integrated. The module was calibrated by adjusting its internal multi-turn potentiometer. Although the nominal target was 5.2V to compensate for voltage drops under load, the tuning phase was critical to ensure the Raspberry Pi 5 receives a consistent voltage within its operational tolerances.

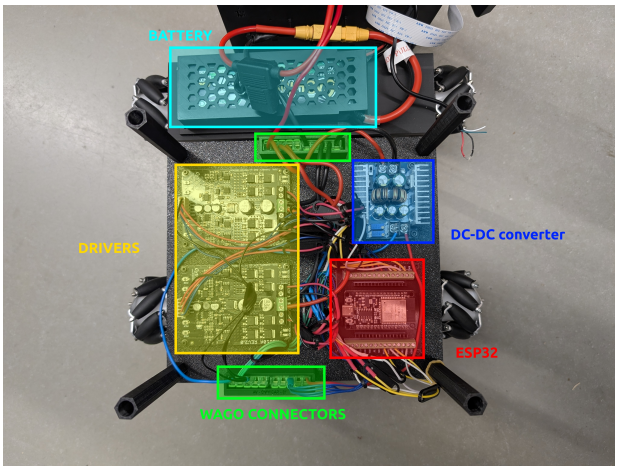
5.2.3 Layer 2: Logic and Navigation

The electronics deck (Layer 2) isolates sensitive logic components from electromagnetic interference (EMI).

- **Processing Unit:** The **Raspberry Pi 5** is mounted as the central controller.
- **Sensors:** The **Pi Camera** and **Lidar** provide an unobstructed 360° field of view.



Robot without wiring



Main component in the lower layer

Figure 5.2: Physical assembly of the OMNICAR lower layer

5.3 Assembly Workflow and System Validation

The implementation followed a rigorous chronological sequence to minimize the risk of electrical failure and ensure the correctness of the low-level control loops.

5.3.1 Wiring and Electrical Pre-testing

Following the layout of Layer 1 and Layer 2, the wiring was executed according to the previously defined electrical schematic. The circuit was powered by the LiPo battery, and the **DC-DC converter tuning** was performed as the first step.

The motor actuation was validated by connecting the motors to the MDD10A drivers and supplying 12V for power and 5V for logic. Initial functionality was confirmed **manually** via the physical test buttons on the drivers, ensuring the gearmotors responded correctly before involving the microcontroller. Subsequently, the DIR and PWM signal lines were routed to the **ESP32**.

5.3.2 Firmware Configuration and Signal Analysis

A preliminary firmware was developed using **PlatformIO** [17] to test the PWM generation. As illustrated in Figure 5.3, the ESP32 successfully generated a stable square wave with a 50% duty cycle, confirming the correct configuration of the LEDC hardware peripherals.

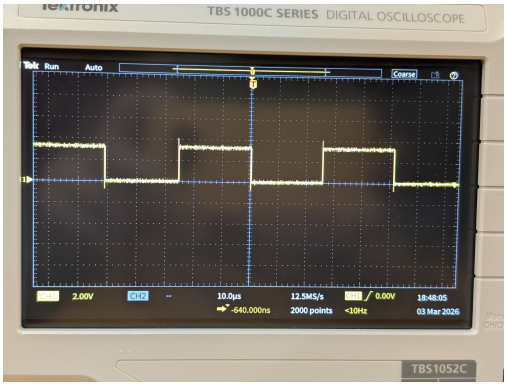


Figure 5.3: Oscilloscope capture of the PWM control signal (50% Duty Cycle)

5.3.3 Encoder Integration and Signal Conditioning

The final phase of the assembly involved the integration of the motor encoders. Given that the encoders were powered at 5V, it was necessary to ensure the output signals were compatible with the ESP32's 3.3V logic [14]. To achieve this, a custom signal conditioning board was realized by soldering **10kΩ pull-down resistors** on a prototyping PCB for all 8 encoder channels (A/B phases for 4 motors), see Figure 5.5. This setup, shown in Figure 5.4, effectively clamped the input signal to approximately **3.4V**.

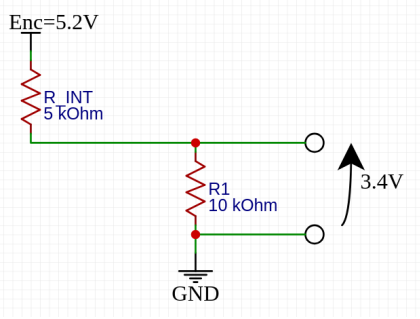


Figure 5.4: Pull-down resistor implementation for signal conditioning

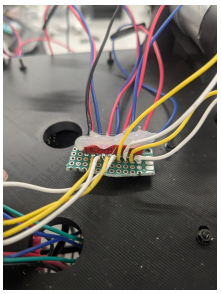
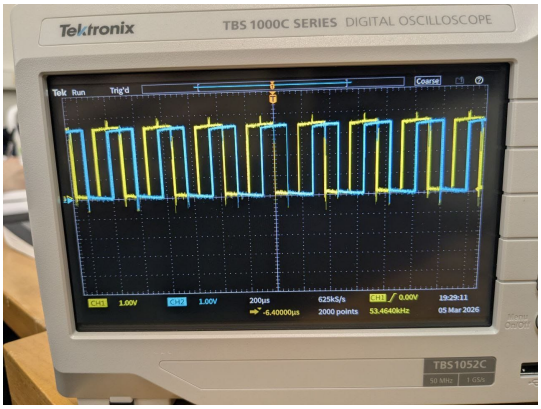


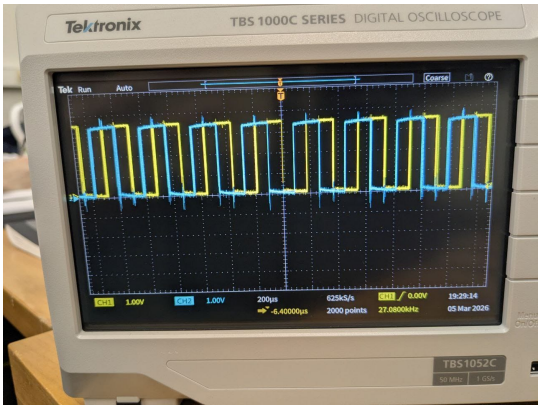
Figure 5.5: Real application of Pull-down resistor

Figure 5.6: Solution for Encoder signal

The resulting waveforms were verified via oscilloscope for both forward and backward rotations (Figure 5.7). The quadrature relationship between the signals remained consistent, providing the high-resolution feedback required for accurate odometry.



Forward rotation signal



Backward rotation signal

Figure 5.7: Conditioned encoder signals (approx. 3.4V peak-to-peak)

The completion of these hardware validation steps allowed for the commencement of the first dynamic field tests on the integrated platform, shown in Figure 5.8.

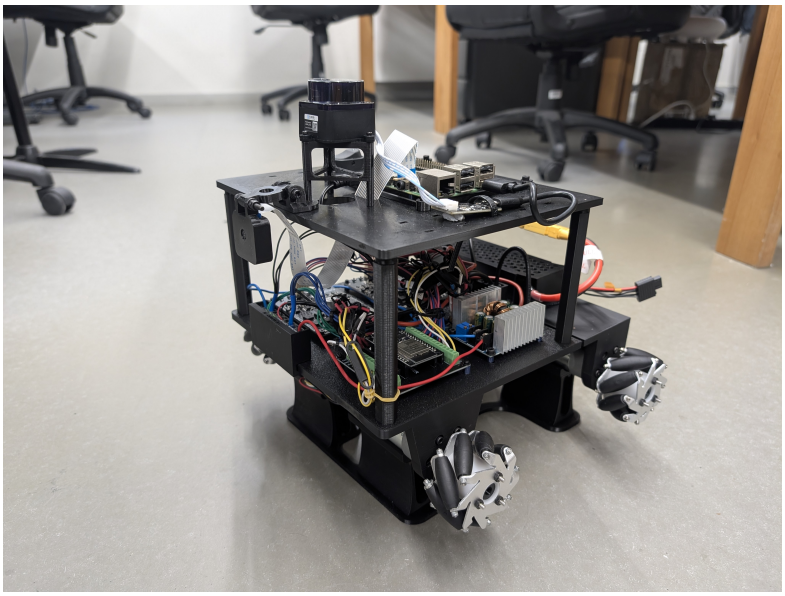


Figure 5.8: Fully assembled OMNICAR platform ready for field testing

Chapter 6

Firmware Development and Experimental Validation

6.1 Development Environment: ESP32 and PlatformIO

The OMNICAR project utilizes the ESP32 microcontroller as the core low-level control unit. Given the necessity to manage high-frequency tasks—such as reading four independent encoders and executing real-time control loops—the development environment played a critical role in ensuring system reliability. While the Arduino IDE is suitable for rapid prototyping, it lacks the advanced features required for complex, multi-file robotic projects. Consequently, the development was migrated to **PlatformIO**, an open-source ecosystem for IoT development integrated into Visual Studio Code.

The choice of PlatformIO was driven by several technical advantages essential for the OMNICAR's firmware architecture:

- **Modular Project Structure**: Unlike the flat file structure of Arduino, PlatformIO encourages a professional C++ organization. By separating declarations in header files (`.h`) and logic in implementation files (`.cpp`), the project achieved better encapsulation and significantly reduced compilation times through incremental builds.
- **Advanced Debugging and Monitoring**: The integrated serial monitor, configured at 115200 baud, allowed for real-time telemetry of motor speeds and PID performance. Furthermore, the environment's support for hardware debugging was pivotal during the initial validation of the encoder reading tasks.
- **Remote Deployment and Diagnostics (OTA)**: Since the OMNICAR is a mobile platform, frequent firmware updates via USB were impractical during dynamic field tests. PlatformIO's native support for **ArduinoOTA** (Over-The-Air) enabled wireless flashing of the ESP32 over the local network, allowing for immediate iterations. Furthermore, this setup facilitated the analysis of offline floor tests by

sharing log files through an integrated **Web Server** hosted on the ESP32's IP address, eliminating the need for physical access to the robot's internal hardware.

6.1.1 Configuration: the platformio.ini file

The `platformio.ini` file is the core of the project configuration. It allows for precise control over the compilation environment, library versions, and upload methods. Below is the specific setup used for the OMNICAR:

platformio.ini

```
[env:esp32dev]
platform = espressif32 @ 6.5.0
board = esp32dev
framework = arduino

; Over-the-Air update configuration
upload_protocol = espota
upload_port = 192.168.31.144    ; IP address ESP32

monitor_speed = 115200        ;BaudRate serial monitor
lib_deps =
    adafruit/Adafruit ADS1X15 @ ^2.5.0
    adafruit/Adafruit BusIO @ ^1.14.1
    bblanchon/ArduinoJson @ ^6.21.3

build_flags =
    -D CORE_DEBUG_LEVEL=3
    -D _USE_MATH_DEFINES

build_src_filter = +<*> -<PID.cpp>
```

As previously mentioned, the implementation of the `espota` protocol proved to be fundamental during the experimental phase. It enabled wireless firmware updates while the robot remained positioned on the floor, thereby eliminating the need for constant physical tethering.

6.2 Software Architecture and Project Structure

The firmware is developed using an Object-Oriented Programming (OOP) approach to manage the complexity of the four-wheeled omnidirectional system. The separation of declarations (`.h`) and implementations (`.cpp`) is fundamental to ensure modularity (module like motor, encoder, communication, control), allowing for independent testing of hardware components like motors and encoders.

The ESP32 project hierarchy, as organized within the PlatformIO environment, is structured as follows:

ESP32 Project Directory Structure

```
OmniCar_Control/[02]Code/Omnicar/
├── .pio/ ..................................... Build files and libraries
├── include/ ..................................... Header files (.h)
│   ├── connection.h ..................... WiFi and MQTT protocols
│   ├── log.h ..................................... Serial and network log
│   ├── motor.h ..................... Control interface (PWM/DIR)
│   ├── CtrlPID.h ..................................... PID control class
│   ├── robot.h ..................... Robot class aggregation
│   ├── robot_config.h ..................... Hardware pins and kinematics
│   └── encoder.h ..................... Encoder and speed estimation
├── lib/ ..................................... External managed libraries
│   ├── Adafruit_ADS1115
│   ├── Adafruit_BusIO
│   └── ArduinoJson
├── src/ ..................................... Implementation files (.cpp)
│   ├── main.cpp ..................................... Core logic and loops
│   ├── connection.cpp
│   ├── motor.cpp
│   ├── CtrlPID.cpp
│   ├── robot.cpp
│   └── encoder.cpp
└── platformio.ini ..................... Build configuration
```

6.2.1 Role of Main Components

To ensure a clear distinction between high-level logic and low-level hardware control, the following roles have been assigned to the primary modules:

- **Robot Class (`Robot.h/cpp`)**: Acts as the central hardware abstraction layer and orchestrator for the firmware. It encapsulates the state of the entire platform by aggregating instances of `Motor` objects and PID controllers. It is responsible for executing the main control loop at a deterministic frequency (e.g., 50Hz, 100Hz, 200Hz), updating the kinematic model, and synchronizing the flow of data between the low-level hardware (actuators/sensors) and the communication layer.
- **Motor & Encoders**: These low-level modules handle the direct interface with the hardware drivers. The Motor module translates velocity setpoints into PWM signals (using ESP32 peripherals or external drivers like MDD10A) to control the motors. The Encoder module processes signals—capturing ticks via interrupts or

dedicated hardware—to compute real-time angular position and velocity for the feedback loop.

- **Connection Handler**: Manages the communication bridge between the embedded controller (ESP32) and the high-level planner (Raspberry Pi/ROS). It implements the protocol stack to parse incoming command packets (e.g., velocity vectors) and serializes telemetry data (encoders, battery status). It supports multiple transport layers, such as high-speed Serial (UART) for real-time control or MQTT/WiFi for remote monitoring and logging.
- **Robot Config (`robot_config.h`)**: A centralized configuration header that serves as the "single source of truth" for the system. It contains all calibrated hardware constants—such as pin mappings, kinematic dimensions, gear ratios, and PID gains. This decoupling prevents hard-coded values in the logic files, facilitating easy tuning and adaptation to different hardware revisions without modifying the core source code.

6.3 Hardware Abstraction: robot_config.h

The `robot_config.h` file acts as a centralized configuration hub. It contains critical hardware parameters such as:

- **Pin Mapping**: GPIO assignments for PWM and DIR signals for the four MDD10A drivers.
- **Kinematics**: Wheel radius, gear ratios, and robot geometry (L and l distances).
- **Encoder Resolution**: Definition of the counts per revolution (CPR) to ensure accurate odometry.

The `robot_config.h` file acts as a centralized configuration hub, ensuring decoupling between hardware parameters and control logic. It contains critical system constants such as:

- **Pin Mapping**: GPIO assignments for the PWM and DIR (direction) peripherals and digital inputs for the encoder signals.
- **Kinematics**: Geometric definitions for the omnidirectional model, including wheel radius ($r = 0.06$ m), gear ratio ($N = 18.75$), and chassis half-dimensions ($L_x = 0.185$ m, $L_y = 0.220$ m).
- **Encoder Resolution**: Definition of the counts per revolution ($CPR = 64$) used to convert raw ticks into angular velocity.
- **PID Control Tuning**: Specifies the PI controller gains derived via *Internal Model Control* (IMC). The parameters are calculated based on an identified first-order plant model of the motors

6.4 Experimental Procedures and Main Loop

The firmware's operational core is encapsulated within the `main.cpp` file, which manages the real-time execution loop for the motion control algorithms.

6.4.1 System Initialization and Setup

The `setup()` function defines a common initialization phase for all experimental procedures. This routine configures the hardware peripherals and communication protocols required for robot operation. Include the initialization of the serial interface at 115,200 baud for telemetry, the configuration of the `Robot` object, and the setup of the Over-The-Air (OTA) update system.

The `robot.init()` method assigns the GPIO pins for the MDD10A motor drivers and read the encoder channels. A serial channel protocol is initialized to exchange data between the ESP32 and the Raspberry Pi 5.

main.cpp/setup

```
void setup() {
  // Inizializza la seriale per il monitoraggio
  Serial.begin(115200);
  serial_channels.init(processSerialPacket, serialWrite);
  // Configura il pulsante BOOT (GPIO 0)
  pinMode(0, INPUT_PULLUP);

  // --- WIFI, OTA, WEB SERVER SETUP ---
  setupWiFi();
  setupOTA();
  // setupWebServer(); //Web Server usato per scaricare i
     file di log durante le prove offline

  Serial.println("--- OMNICAR MOTOR TEST START ---");
  Serial.flush();
  // Inizializza il robot passando la funzione di callback
     per la seriale
  robot.init(serialWriteChannel);
  Serial.println("Robot initialized.");

  // Reset signal
  serialWriteChannel('r', 0);
  // Initialization
  current_micros = micros();
  previous_micros = current_micros;
  last_motor_update_millis = millis();
}
```

6.4.2 Encoder Validation and Tick Counting

The initial experimental phase consisted of a manual rotation test performed on a selected motor to calibrate and validate the encoder resolution. Although the hardware specifications indicated a nominal resolution of 1200 pulses per revolution (ppr), this empirical verification was essential to ensure that the software parameters defined in `robot_config.h` accurately reflected the physical behavior of the system. This procedure eliminated potential discrepancies between the theoretical datasheet and the real-time pulse counting logic, providing a reliable foundation for the first-order velocity control loop.

main.cpp/loop_manuale

```
void loop() {
  uint32_t dt = 0; // Variabile delta time per robot.update
  int idx_mot = 0;
  Serial.println("   TEST MANUALE ENCODER (Ruota a mano)/\n
     Inserisci indice motore (0-3) per iniziare.");

  while (Serial.available() == 0) {
    delay(10);
  }

  idx_mot = Serial.parseInt();
  // Pulisci il buffer seriale (rimuovi newline)
  while (Serial.available()) { Serial.read(); }

  if (idx_mot >= 0 && idx_mot < kNumMot) {
    Serial.printf("--> MONITORAGGIO MOTORE %d ATTIVO\n",
       idx_mot);
    Serial.println("    Ruota la ruota manualmente.");

    robot.setMotorPWM(idx_mot, 0);

    // Loop finch  non si riceve input seriale
    while (Serial.available() == 0) {
      robot.update(dt);
      Serial.printf("MOT[%d] | Ticks: %d | Odo: %d\n",
         idx_mot, robot.enc[idx_mot].tick, robot.enc[idx_mot
         ].odo);
    }
  } else {
    Serial.printf("Indice %d non valido! Inserire 0, 1, 2 o
       3.\n", idx_mot);
  }
}
```

6.4.3 Open-Loop Response and First-Order Approximation

Individual motors were driven at specific PWM duty cycles to validate the assumption of a **first-order system** model [18, 19]. To derive this model, we consider the electromechanical equations of a DC motor in the Laplace domain (s-domain):

$$\begin{cases} U(s) = (R + sL)I(s) + K\Omega(s) \\ sJ\Omega(s) = KI(s) - B\Omega(s) - T_q \end{cases} \quad \begin{cases} U(s) = (R + sL)I(s) + K\Omega(s) \\ KI(s) = (sJ + B)\Omega(s) + T_q \end{cases} \tag{6.1}$$

where $U(s)$ is the input voltage, $I(s)$ the armature current, $\Omega(s)$ the angular velocity, and T_q represents the load torque.

Linearization and Order Reduction

It is important to note that a linear transfer function $\frac{\Omega(s)}{U(s)}$ cannot be directly deduced from the equations above because the term T_q renders the expression affine rather than linear. To achieve a linear representation, we assume $T_q = 0$, with the intention of compensating for this physical nonlinearity later. Under this assumption, the system can be rearranged as:

$$\frac{\Omega(s)}{U(s)} = \frac{K}{(R + sL)(sJ + B)} \tag{6.2}$$

resulting in a second-order system. In this scenario, the electrical pole ($s = -R/L$) is typically much faster (located much further left in the complex plane) than the mechanical pole ($s = -B/J$). Consequently, the high-frequency electrical dynamics can be neglected, leading to the final first-order approximation:

$$G(s) = \frac{\Omega(s)}{U(s)} \approx \frac{K_{gain}}{sJ + B} = \frac{A}{\tau s + 1} \tag{6.3}$$

6.4.4 Double Step Test for PID Tuning

As specified in the `robot_config.h` file, the nominal control frequency was established at 50 Hz, although the system performance at higher frequencies was also evaluated in specific test cases. Prior to the formal identification of the system parameters, a preliminary test was conducted on a single motor under no-load conditions. The resulting angular velocity $\omega(t)$, measured in rad/s, is illustrated in Figure 6.1.

The experimental procedure involved a double-step input: the first step applied a PWM duty cycle of 2047 (representing approximately half of the maximum available thrust), while the second step reached the full resolution of 4095. The steady-state angular velocities observed during these tests align closely with idealized theoretical values, peaking at approximately 60 rad/s.

Notably, the open-loop response under no-load conditions show marginal noise and reaches the target velocity with high precision. The purely asymptotic profile of the curve

provides visual confirmation that the electrical dynamics—associated with the second pole of the system—are effectively negligible compared to the mechanical time constant. This empirical evidence further justifies the adoption of a first-order approximation for the velocity control loop.

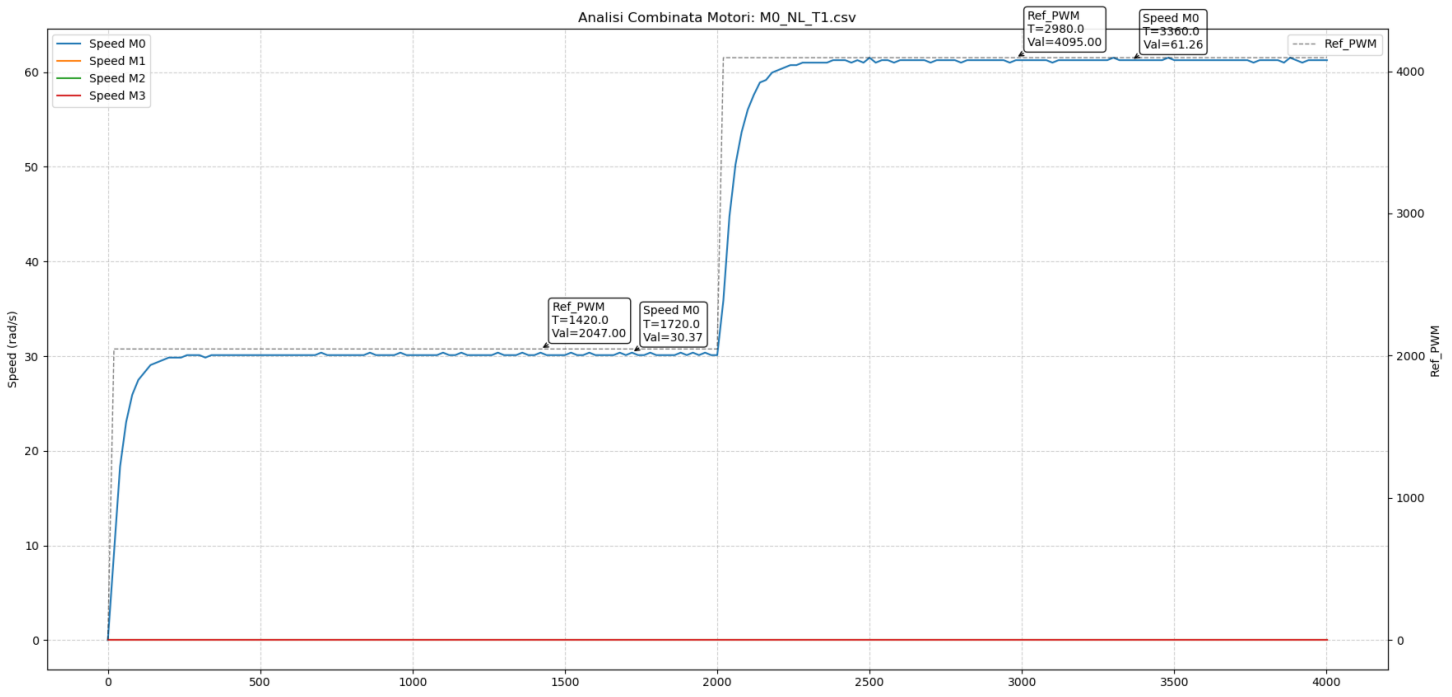


Figure 6.1: First Test PWM in 50Hz

A double-step input was applied to all four motors simultaneously during a test with load (robot on the floor). By measuring the steady-state speed we can calculating the gain and the time constant (τ).

As illustrated in Figures 6.2, 6.3, and 6.4, specific safety PWM thresholds were established for the step response tests: a first step at 750 and a second at 1500. These duty cycle values correspond approximately to angular velocities of 10 rad/s and 20 rad/s, respectively.

In this phase, isolating the individual motor responses was fundamental to detecting any potential mechanical or electrical anomalies. The experimental data confirms that all motors exhibit consistent behavior, characterized by a decoupled and uniform response across the assembly.

Based on the analysis of the collected data, the following system parameters were identified. First, the PWM signals were converted into their equivalent voltage values using the following relation:

$$V_{pwm} = \frac{PWM_{ref}}{PWM_{max}} \cdot V_{battery} \tag{6.4}$$

Given a maximum resolution $PWM_{max} = 4095$ and a measured battery voltage $V_{battery} = 11.9$ V, we obtain:

$$V_{pwm,750} \approx 2.18 \text{ V}, \quad V_{pwm,1500} \approx 4.36 \text{ V} \tag{6.5}$$

The steady-state gain K was then calculated as the ratio between the variation in angular velocity and the variation in input voltage:

$$K = \frac{\Delta\omega}{\Delta V_{in}} = \frac{\omega_2 - \omega_1}{V_{pwm,1500} - V_{pwm,750}} = 5.04 \tag{6.6}$$

To determine the time constant τ, we identified the angular velocity value corresponding to 63.2% of the transient transition between the first and second step:

$$\omega_\tau = \omega_1 + 0.632 \cdot (\omega_2 - \omega_1) = 16.9 \text{ rad/s} \tag{6.7}$$

By locating ω_τ on the experimental temporal plot, the resulting time constant was estimated to be approximately $\tau \approx 90$ ms.

Figure 6.2: Open-loop no-load test (MA_NL) at a sampling frequency of 50 Hz

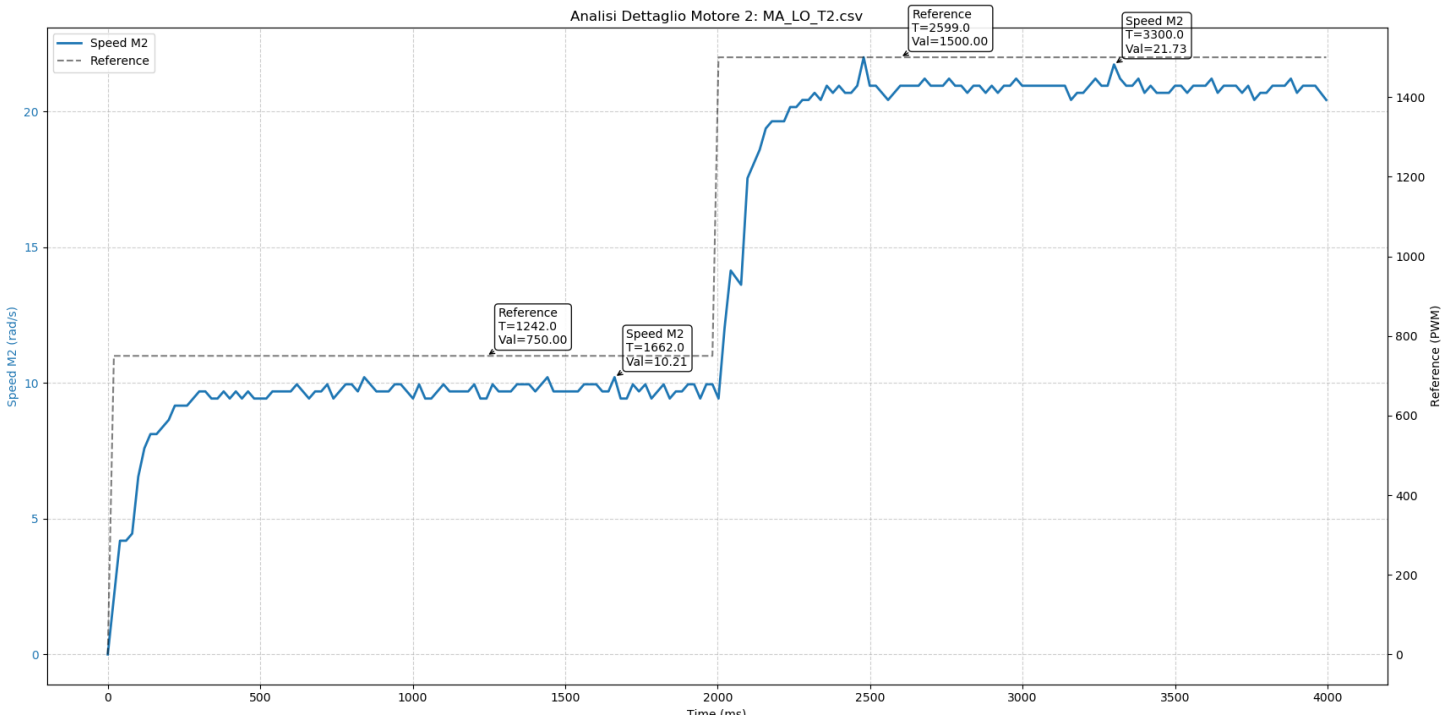


Figure 6.3: Open-loop velocity profile for Motor 2 at 50 Hz sampling rate

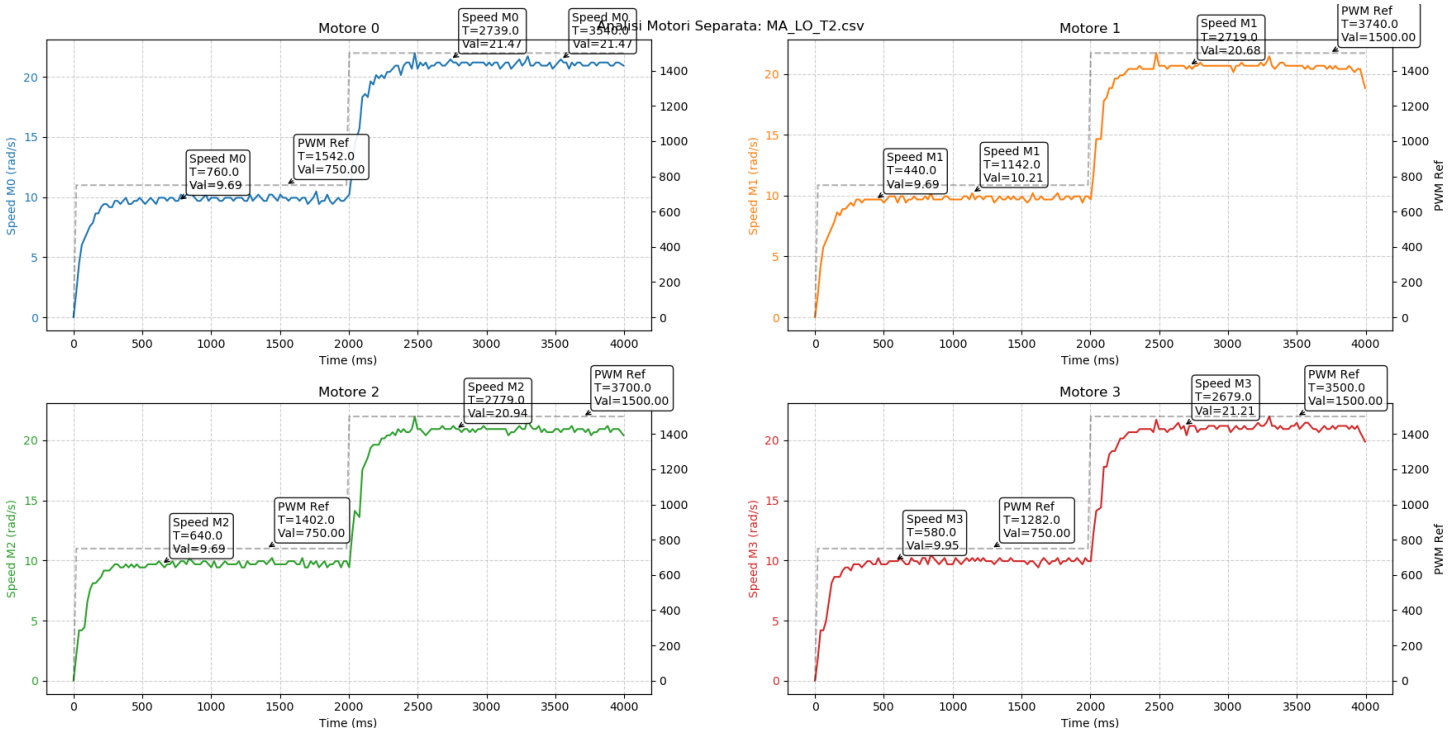


Figure 6.4: Detailed view of the open-loop response at 50 Hz for system identification

As shown in Figure 6.5. Experimental analysis demonstrated that increasing the control frequency beyond 50 Hz significantly degrades the Signal-to-Noise Ratio (SNR) in velocity estimation. This phenomenon is primarily attributable to the reduction in numerical resolution—specifically, the quantization error of encoder ticks within shorter sampling periods—and temporal jitter affecting the `kMotCtrlTimeUs` parameter.

At higher frequencies, interrupt management on the ESP32 introduces variable latencies that lead to asynchronous tick accumulation, resulting in periodic noise spikes. To ensure the stability of the control algorithms and prevent the amplification of these disturbances through derivative terms, the low-level control loop was maintained at 50 Hz. This configuration provides the optimal compromise between temporal determinism and signal resolution.

Subsequent phases of the project will involve use of a dedicated communication link with the Raspberry Pi. In this setup, the raw encoder signals will be transmitted directly and evaluated without the onboard conversion to radians. This approach is designed to assess the signal integrity and the effectiveness of processing when operating at sampling frequencies exceeding 50 Hz.

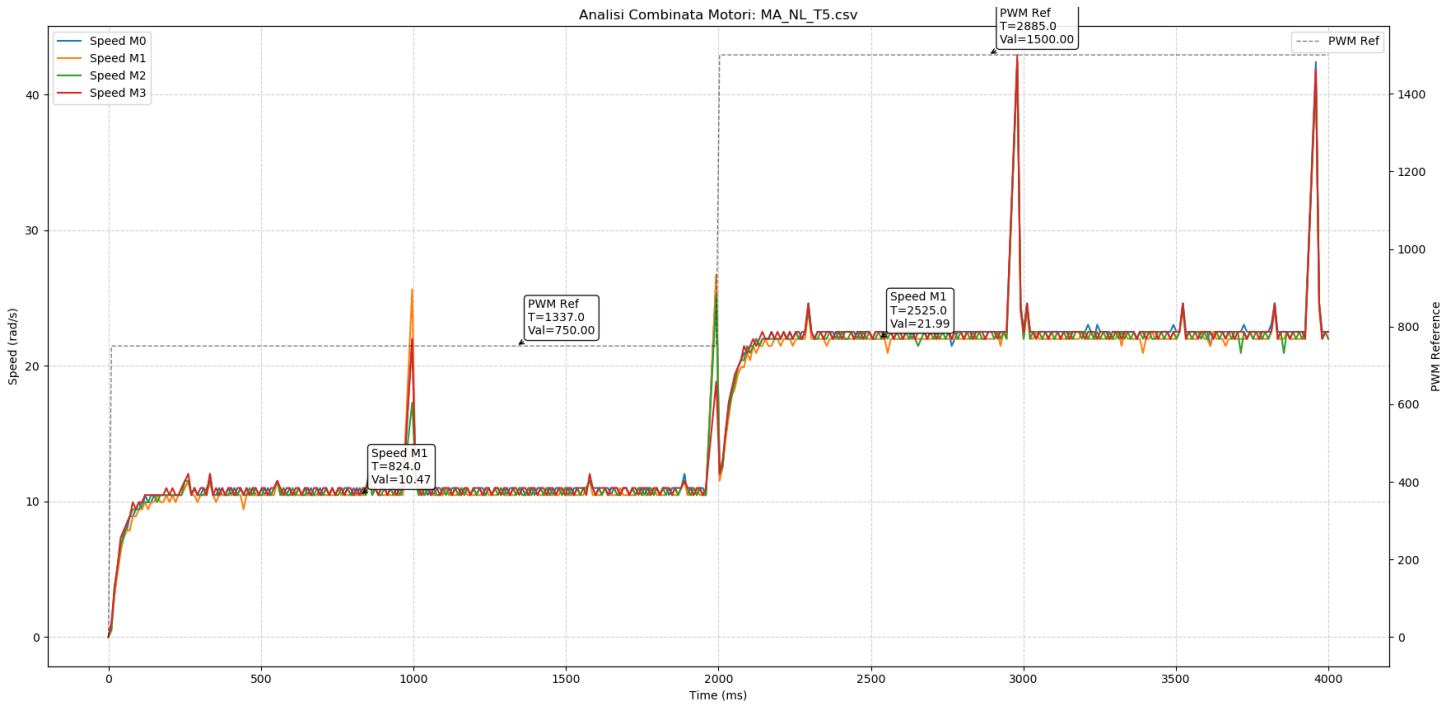


Figure 6.5: Open-loop test results at 100 Hz showing the initial linear noise increase

main.cpp/loop_tuning

```
void loop() {
  ArduinoOTA.handle();
  server.handleClient();
  handleOfflineTest(); // Controlla pressione tasto

  // Gestione Countdown 5 secondi dopo press BOOT
  if (is_boot_test_pending) {
    if (millis() - boot_test_start_millis > 5000) {
      is_boot_test_pending = false;

      // AVVIO TEST E LOG
      logFile = SPIFFS.open("/log.csv", "w");
      if (logFile) {
        tuner.setOutput(&logFile);
        //start(maschera motori attivi, pwm_max, preheat time
            , step1_time, step2_time)
        tuner.start(0b1111, 1500, 0, 2000, 2000);
      }
    }
  }

  // Se il tuner    attivo, esegui il suo ciclo di update (
     non bloccante)
  if (tuner.isRunning()) {
    // Controllo ABORTO: Se premi 'x'
    // (Solo se connesso via seriale)
    if (Serial.available() > 0) {
      char c = Serial.peek(); // Guarda il carattere senza
          rimuoverlo subito
      if (c == 'x' || c == 'X') {
        Serial.read(); // Rimuovi dal buffer
        tuner.stop();
        return; // Ricomincia il loop
      }
    }
    tuner.update();
  }
  // Chiusura del file log
  else if (logFile) {
    logFile.close();
    tuner.setOutput(&Serial);
    Serial.println("\n--- LOG SALVATO SU SPIFFS ---");
    Serial.println("Usa il comando 'r' per scaricare i dati."
        );
  }
  // Altrimenti attendi comandi seriali
  else {
    handleSerialInput();
  }
}
```

6.4.5 Closed-Loop

The transition from open-loop duty cycle commands to a regulated velocity setpoint was achieved through a Proportional-Integral (PI) controller. Since the plant is modeled as a stable first-order system with no zeros, a PI architecture is sufficient to ensure a null steady-state error for step references $R(s) = \frac{1}{s}$.

Controller Design and Pole-Zero Cancellation

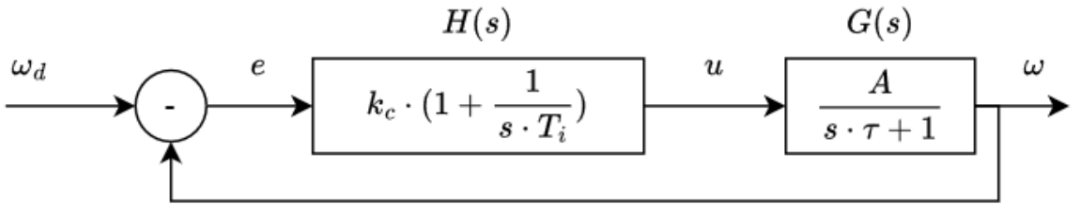


Figure 6.6: Closed-Loop system - Block diagram

Consider the controlling architecture presented in Figure 6.6 The controller $H(s)$ is defined by the following transfer function:

$$H(s) = k_c \left(1 + \frac{1}{T_i s}\right) = k_c \frac{T_i s + 1}{T_i s} \tag{6.8}$$

where k_c represents the proportional gain and T_i the integral time constant. To simplify the closed-loop dynamics, the design choice $T_i = \tau$ was adopted. This allows the controller's zero to cancel the plant's pole [20, 19], resulting in the open-loop transfer function:

$$G(s)H(s) = k_c \frac{A}{\tau s + 1} \cdot \frac{\tau s + 1}{\tau s} = \frac{k_c A}{\tau s} \tag{6.9}$$

The resulting closed-loop transfer function remains a first-order system with unity gain ($A_{cl} = 1$):

$$T(s) = \frac{G(s)H(s)}{1 + G(s)H(s)} = \frac{1}{\tau_{cl} s + 1}, \quad \text{with } \tau_{cl} = \frac{\tau}{k_c A} \tag{6.10}$$

Feedforward Compensation and Real-world Constraints

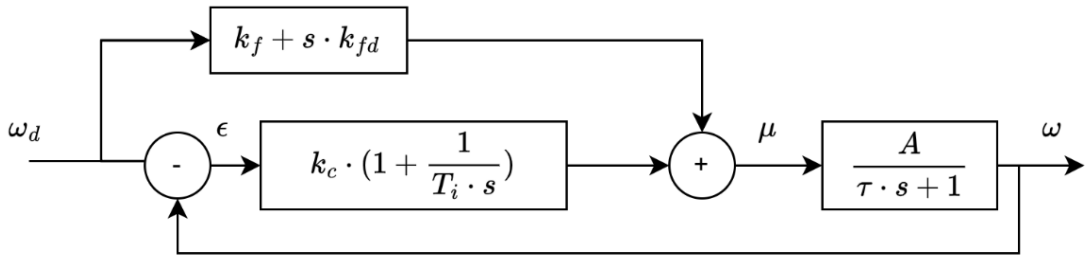


Figure 6.7: Closed-loop control architecture with feedforward - Block diagram

Consider the controlling architecture presented in Figure 6.7 To enhance the system's reactivity beyond the feedback loop's limits, a Feedforward (FF) action $F(s)$ was considered. Ideally, the FF controller should invert the plant dynamics ($F(s) = 1/G(s)$) to achieve $F(s)G(s) = 1$. However, a full inversion $\frac{\tau s+1}{A}$ would be non-proper, requiring a derivative term k_d that is physically unrealizable and highly sensitive to noise. Therefore, a simplified feedforward gain was implemented as the inverse of the steady-state gain:

$$k_f = \frac{1}{A} \tag{6.11}$$

This allows the system to reach the setpoint in finite time by providing the theoretical steady-state voltage required, while the PI controller manages parameter uncertainties and disturbances.

Experimental Validation and Tuning Strategy

Initial tuning was performed using a baseline configuration where the desired closed-loop time constant was set to the open-loop value ($\tau_{cl} = \tau$).

robot_config.h/

```
// PI Gains (Derived via IMC Tuning)
const float kMotCtrlTauCl = kMotModelTau / 1.0f;
// Desired time constant for the closed-loop (s)
const float kMotCtrlKcKp = kMotModelTau / (kMotCtrlTauCl +
    kMotModelLag);
// Tuning: Kc_PI * Kp_plant
const float kMotCtrlKc = (kMotModelTau / kMotCtrlTauCl) /
    kMotModelKp;
// PI proportional gain (V / rad.s^(-1))
const float kMotCtrlTi = kMotModelTau;
// PI integration time (s)
const float kMotCtrlKf = 1.0f / kMotModelKp;
// Feed-Forward gain
```

As shown in Fig. 6.8, this configuration led to an overshoot of approximately 20% and significant jitter at high frequencies (200 Hz). This behavior is consistent with the theoretical risks of aggressive feedforward and quantization errors in discrete-time systems.

To ensure stability without overshoot, a conservative tuning was finalized by doubling the closed-loop time constant ($\tau_{cl} = 2\tau$) and disabling the feedforward action ($k_f = 0$).

$$\tau_{cl} = 2 \cdot \tau \implies k_c = \frac{\tau}{2\tau A} = \frac{1}{2A} \tag{6.12}$$

The results, validated at 50 Hz (Fig. 6.10), demonstrate a purely asymptotic response. This configuration successfully balances the convergence speed with the necessity of a high Signal-to-Noise Ratio (SNR) in velocity estimation.

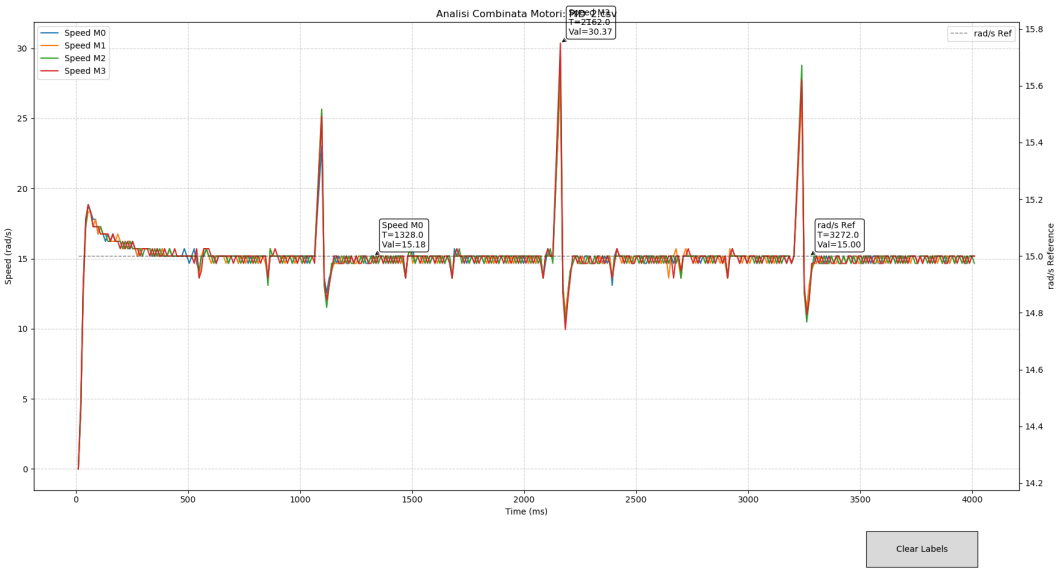


Figure 6.8: Full closed-loop PI test at 15 rad/s (200 Hz) with Feedforward compensation

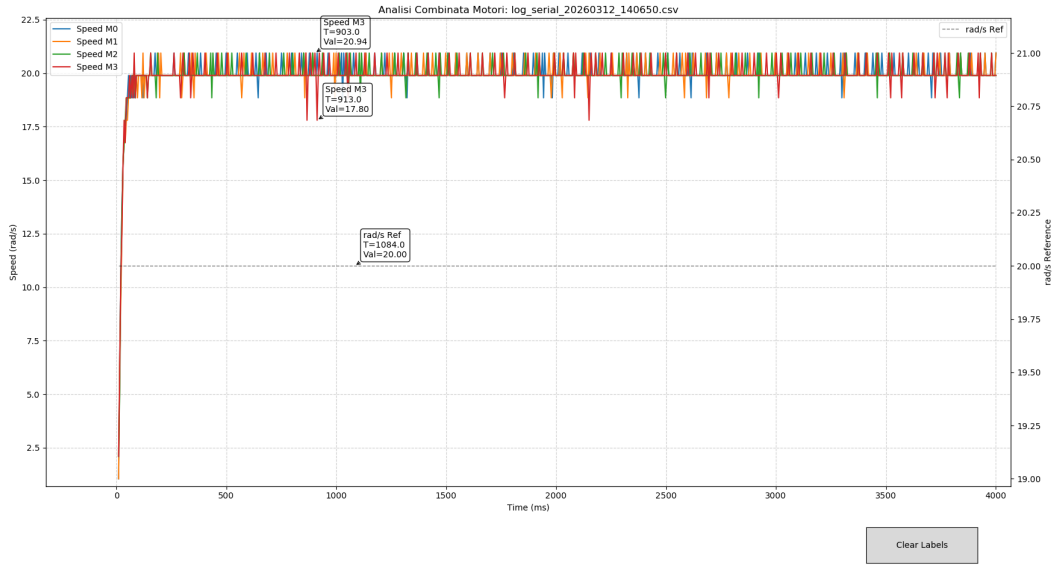


Figure 6.9: PI closed-loop response at 20 rad/s (100 Hz) showing increased measurement jitter

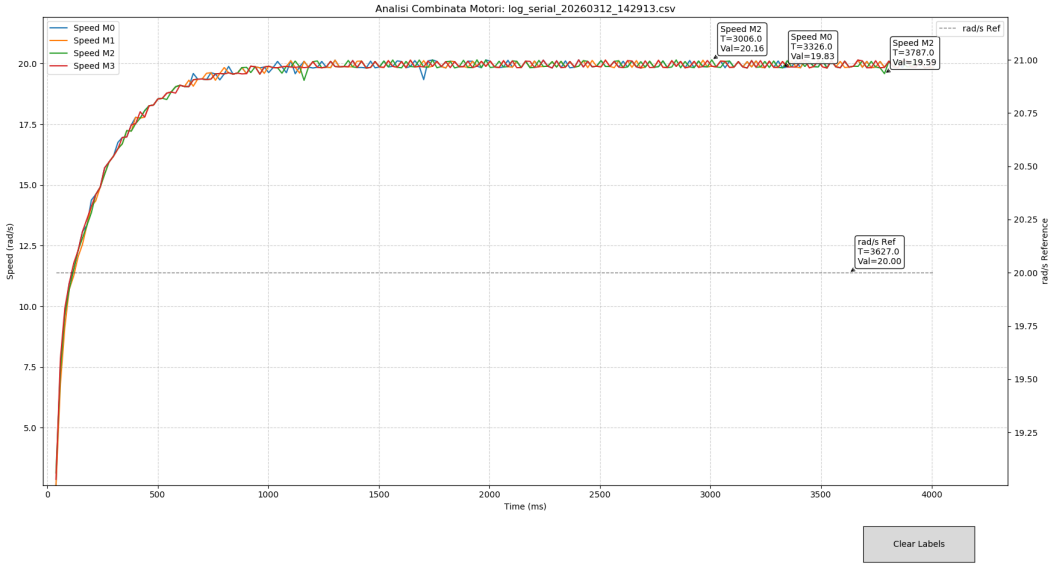


Figure 6.10: PI closed-loop tracking at 20 rad/s (50 Hz) without Feedforward

Chapter 7

Omnicar Platform, ROS Integration and High-Level Software Architecture

7.1 High-Level Software Architecture: From ROS1 Noetic to ROS2 Jazzy

In this thesis, the previous ROS1-based architecture was progressively migrated to ROS2 Jazzy. The objective was not only to update the software stack, but also to obtain a structure that could be tested more easily, extended with new modules, and maintained on the Raspberry Pi 5.

The transition to ROS2 was particularly relevant for this platform because the high-level software has to coordinate several modules: the serial driver, the odometry node, the localization pipeline, the lidar interface, and the motion-control nodes. In this context, the **Data Distribution Service** based communication layer of ROS2 provides a more suitable basis for distributed execution and future extensions than the previous centralized **ROS Master**.

The hardware upgrade to the **Raspberry Pi 5** was also an important part of the migration. Compared with the **Raspberry Pi 3** used in the previous version, the new board offers more computational margin for running several ROS2 nodes at the same time. This is especially useful for odometry computation, localization, lidar data processing, and high-level control tasks, where delays or CPU saturation can directly affect the quality of the robot behaviour.

7.1.1 Migration and Local Implementation

From a practical point of view, the migration required rebuilding the software stack around the ROS2 workflow and checking each module separately before running the complete robot. This was necessary because the driver, odometry, localization, and

control nodes depend on each other, but failures in one layer can easily mask problems in the others.

During the migration, I chose to keep the project inside a dedicated ROS2 workspace on the Raspberry Pi instead of mixing the packages with system-level software. This made the development process easier to control, especially when rebuilding packages, changing configuration files, or testing only a subset of the nodes.

- **Dependency Isolation**: A local workspace allows better control of project-specific dependencies and reduces conflicts with system-wide libraries and packages.
- **Package organization**: The workspace structure helps organize drivers, interfaces, launch files, and configuration files in a clear and maintainable way.
- **Reproducibility**: The complete software can be deployed more easily on different Raspberry Pi boards while preserving the same internal package organization.
- **Future Extensions**: ROS2 makes future extensions easier, such as improved localization, additional sensors, and more advanced control features.

The migration was not limited to a simple syntax update. Some technical points required particular attention during the transition:

- the change from the ROS1 `catkin` build system to the ROS2 `ament_cmake` and `colcon` workflow;
- the adaptation of legacy custom messages and services to the ROS2 interface package `sdpo_drivers_interfaces`;
- the conversion of launch files to the ROS2 Python-based launch system;
- the verification of topic names, node names, parameter files, and TF frame conventions;
- the preservation of compatibility with the serial protocol used by the ESP32 firmware.

For this reason, the migration was performed progressively. I first focused on the ROS2 driver, because it represented the mandatory bridge between the Raspberry Pi and the ESP32 firmware. Once this communication layer was stable, the remaining modules could be migrated and tested one at a time.

7.1.2 Workspace Structure

The active ROS2 workspace of the project is `omnicar_ws`. It follows the standard ROS2 and `colcon` organization, where the compiled artifacts are separated from the source code. In particular, the directories `build/`, `install/`, and `log/` contain the compilation results and runtime environment, while the `src/` directory contains the software packages under development.

Each package keeps the standard ROS2 files, namely `CMakeLists.txt` and `package.xml`, which define the build targets, dependencies, and package metadata.

The separation between header files (`.h`) and source files (`.cpp`) remains important for code organization and maintainability. Header files define the interfaces of the software components, while source files implement the internal logic. This structure simplifies development and supports incremental compilation.

The `config` directories use `.yaml` files to store parameters such as communication settings, odometry constants, and localization configuration. This choice makes the system easier to configure and tune without modifying the source code.

The following directory tree shows the simplified structure of the active ROS2 workspace:

omnicar_ws: Simplified Structure

```
omnicar_ws/
├── bags/ ..... Recorded ROS2 bag files from controller and robot tests
├── Test/ ..... Offline analysis workspace for experimental validation
│   ├── test_code/ ..... Python scripts and configuration files for bag analysis
│   └── test_results/ ..... Generated plots, metrics and summaries from test runs
└── src/ ..... ROS2 source workspace containing robot packages and support libraries
    ├── 5dpo_ratf_driver-main/ ..... Main ROS2 robot driver package
    │   ├── include/ ..... Header files for robot and driver classes
    │   ├── src/ ..... ROS2 source files, including driver and tuning nodes
    │   ├── launch/ ..... ROS2 launch files, including bringup
    │   └── sh/ ..... Support scripts and legacy tuning utilities
    ├── sdpo_driver_omnijoy/ ..... ROS2 joystick teleoperation node
    │   ├── src/ ..... Joystick command processing logic
    │   └── launch/ ..... Launch file for Logitech F710 teleoperation
    ├── sdpo_drivers_interfaces/ ..... ROS2 custom messages and services
    │   ├── msg/ ..... Motor reference and encoder feedback messages
    │   └── srv/ ..... Custom service definitions
    ├── sdpo_ros_odom/ ..... ROS2 odometry computation package
    │   ├── include/ ..... Odometry class headers
    │   ├── src/ ..... Odometry implementation
    │   ├── config/ ..... Kinematic and wheel parameters
    │   └── launch/ ..... Odometry launch files
    ├── sdpo_ratf_ros_localization/ ..... ROS2 localization module
    │   ├── include/ ..... Localization and EKF headers
    │   ├── src/ ..... Localization implementation
    │   ├── config/ ..... Localization parameters
    │   └── launch/ ..... Localization launch file
    ├── ldlidar_stl_ros/ ..... ROS2 lidar integration package
    │   ├── src/ ..... Lidar publishing and listening nodes
    │   └── config/ ..... Lidar configuration files
    ├── 5dpo_serial_port-main/ ..... Asynchronous serial communication library
    │   ├── include/ ..... Serial communication headers
    │   └── src/ ..... Async serial implementation
    ├── serial_communication_channels-main/ ..... Serial channel protocol library
    │   ├── include/ ..... Protocol headers
    │   └── src/ ..... Protocol implementation
    └── sdpo_motion_control/ ..... High-level ROS2 motion-control package
        ├── config/ ..... Controller parameter files and trajectory/path settings
        ├── src/ ..... Controller node implementations for go-to-point and path following
        ├── launch/ ..... Launch files for controller tests
        └── msg/ ..... Custom debug messages published by the control nodes
```

7.1.3 Experimental Validation: ROS2 Integration and Control Chain

This phase of the software integration focused on validating the communication and control chain of the OmniCar platform in the ROS2 environment. These tests were intended to verify the correctness of serial communication with the ESP32 controller, the consistency of encoder feedback, joystick commands, odometry, and localization modules inside the migrated ROS2 architecture [8, 21].

Test 1: Encoder Monitoring and Open-Loop Velocity Control

The first test was intentionally simple, because before testing odometry or control it was necessary to verify that the Raspberry Pi could receive reliable encoder data from the ESP32.

Operational Procedure After enabling access to the serial device, the main driver node, `sdpo_ratf_driver`, was launched. This node is the central communication interface between the ROS2 graph and the ESP32 firmware. In the current implementation, it subscribes to the `motors_ref` topic and publishes encoder and digital state feedback through the topics `motors_enc`, `switch_1_state`, and `switch_2_state`.

To validate the feedback chain, the `motors_enc` topic was monitored during runtime while the wheels were rotated manually. This check confirmed the correct reception of encoder increments, the absence of packet loss, and the correct transmission of angular speed and ticks-per-revolution information from the embedded layer to ROS2.

For actuation validation, a constant reference was sent on the `motors_ref` topic using the custom message type `sdpo_drivers_interfaces/msg/MotRefArray`. In this way, it was possible to verify the complete command path from ROS2 to the ESP32 firmware.

The test confirmed that the node `sdpo_ratf_driver` acts as the central bridge of the low-level architecture. It guarantees bidirectional data exchange with the embedded controller while providing a stable interface for the higher- level ROS2 nodes.

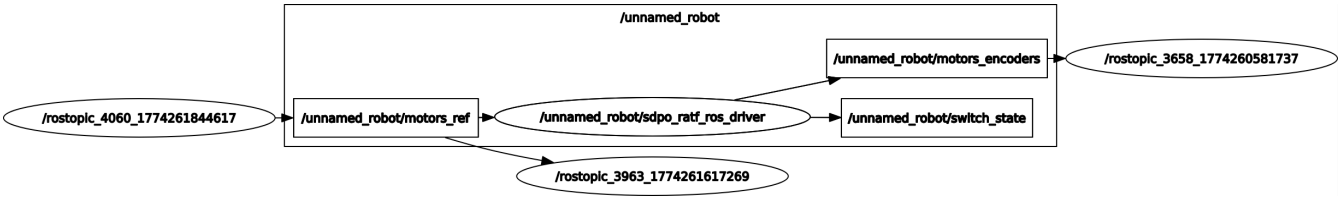


Figure 7.1: rqt_graph showing the active nodes and topics during the encoder test, including reference velocity and measured encoder feedback.

Commands to use for Test 1

```
CMD1
  ros2 launch sdpo_ratf_driver sdpo_ratf_driver.launch.py

CMD2
  ros2 topic echo /unnamed_robot/motors_enc

CMD3
  ros2 topic pub -r 20 /unnamed_robot/motors_ref
     sdpo_drivers_interfaces/msg/MotRefArray "{ang_speed_ref:
      [{ref: 5.0}, {ref: 5.0}, {ref: 5.0}, {ref:5.0}]}"
  ros2 topic pub -r 20 /unnamed_robot/motors_ref
     sdpo_drivers_interfaces/msg/MotRefArray "{ang_speed_ref:
      [{ref: 0.0}, {ref: 0.0}, {ref: 0.0}, {ref:0.0}]}"
```

Test 2: Manual Control using a Joystick Controller

The second validation phase concerned the integration of a joystick controller in the ROS2 environment, with the aim of verifying manual control of the platform and the correct interaction between high-level user commands and low- level motor actuation.

The teleoperation chain is organized into three stages:

- **Joystick Acquisition**: The node `joy_linux_node`, launched through the package `joy_linux`, acquires the state of the Logitech F710 controller and publishes it on the topic `joy` as `sensor_msgs/msg/Joy`.
- **User Command Interpretation**: The node `sdpo_driver_omnijoy` subscribes to `joy` and converts the joystick inputs into a velocity command of type `geometry_msgs/msg/Twist`, published on the topic `cmd_vel`. This node applies deadman logic and scaling factors for linear and angular motion.
- **Conversion into Wheel References**: The node `sdpo_ros_odom_cmd_vel` subscribes to `cmd_vel`, applies the inverse kinematics of the four-wheel omnidirectional platform, and publishes the resulting wheel references on `motors_ref`. In addition, it publishes the scaled robot-level reference on `cmd_vel_ref`.

At this point, the node `sdpo_ratf_driver` receives the `motors_ref` message and transmits the corresponding angular speed references to the ESP32 firmware through the serial communication protocol. This result is important, because it shows that the migrated ROS2 control chain is modular and separated into functional layers, as shown in Figure 7.2. The joystick node manages the user input, the kinematic conversion node computes the wheel references, and the main driver handles communication with the embedded controller. This separation improves clarity, maintainability, and testability of the overall system.

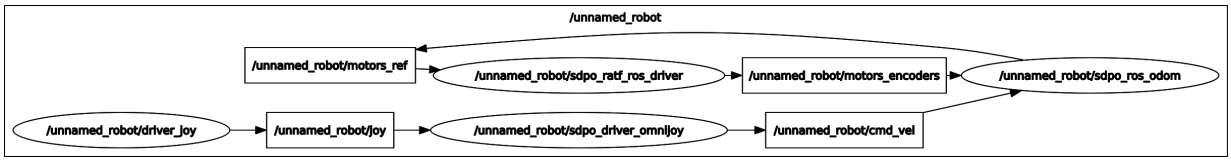


Figure 7.2: rqt_graph showing the active nodes and topics during the joystick test

Commands to use for Test 2

```
CMD 1
    ros2 launch sdpo_ratf_driver omnicar_joy_control.launch.
        py

CMD 2
    ros2 topic echo /unnamed_robot/odom
```

Test 3: Localization and Lidar Integration

After validating the joystick teleoperation chain, the next step was to verify that the odometry node produced a coherent estimate of the robot motion. The robot was moved manually or through joystick commands while the estimated pose and TF frames were observed in RViz. The comparison with the real motion showed that the direction of motion and frame alignment were consistent, although small discrepancies appeared due to wheel slip, odometric drift, and modelling errors.

The localization function is managed by the node `sdpo_ratf_ros_localization`, which subscribes to `odom` and, when enabled, also to `laser_scan_point_cloud`. It publishes the estimated robot pose on the topic `pose`, the detected beacons on `beacons`, and the reference map beacons as `map_beacons`. Moreover, it broadcasts the TF transform between the frames `map` and `odom`.

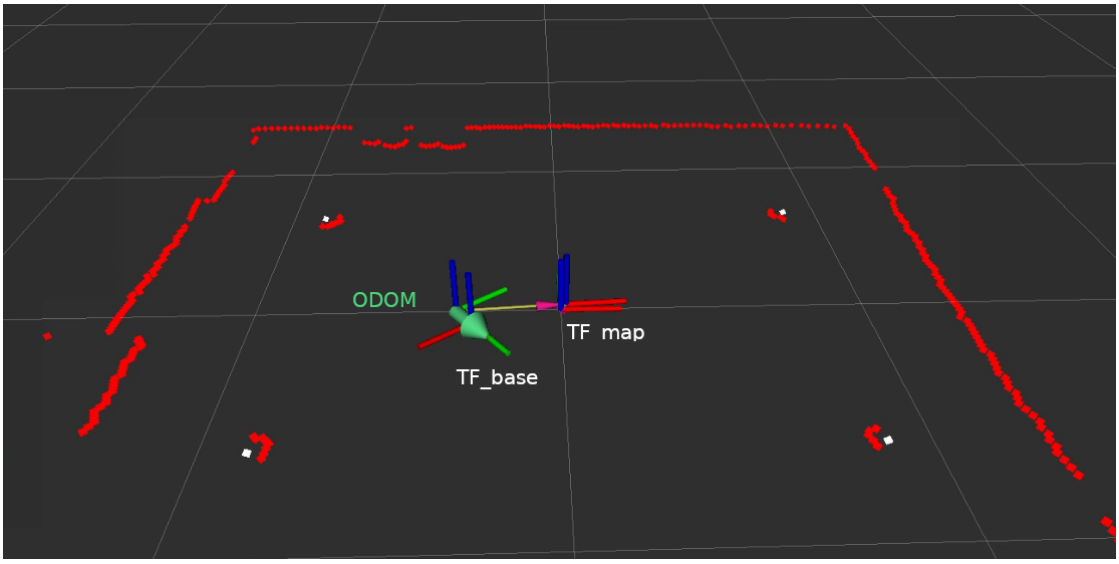
This architecture allows localization to refine the odometric estimate by combining wheel-based motion prediction with external observations derived from the lidar subsystem. The localization process can be observed in Fig. 7.3, where the RViz visualization shows the lidar scan in red and the detected beacons in white, providing a direct representation of the perception layer used for pose estimation.

The TF tree adopted in this configuration follows the standard chain
`map` → `odom` → `base_footprint` → `laser`.

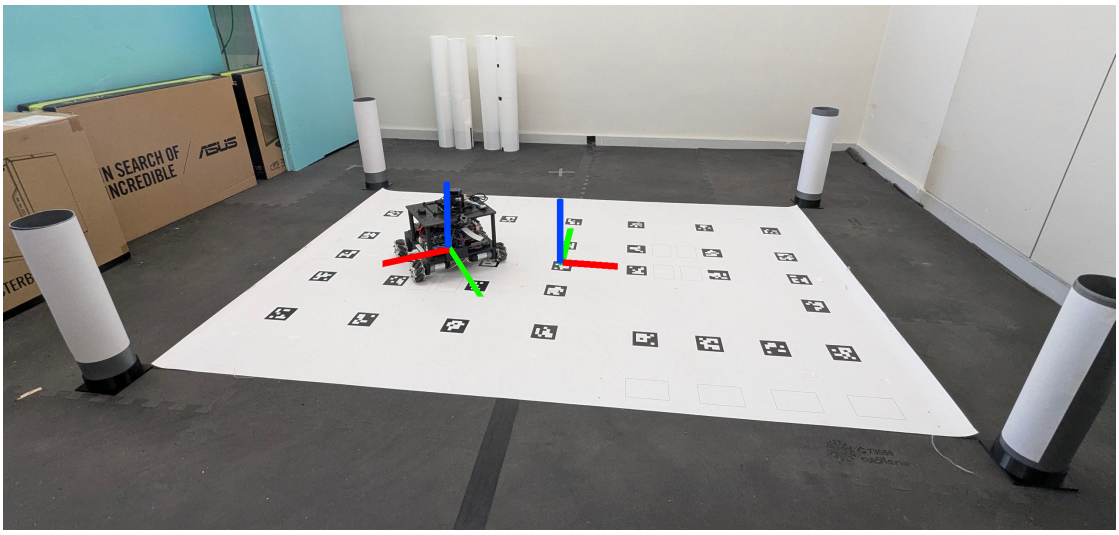
In this structure, the `map` frame represents the global reference, `odom` provides a continuous but drifting local estimate, `base_footprint` defines the robot base frame, and `laser` corresponds to the lidar sensor frame. This hierarchical organization ensures consistency between localization, odometry, and sensor measurements [13, 1].

The lidar integration is provided by the package `ldlidar_stl_ros`, which publishes scan data for the perception layer. In the current migration process, the lidar is an important component because it supports the localization pipeline and extends the ROS2 stack beyond basic low-level control.

The comparison between the real experiment and the RViz visualization shows that the odometry provides a consistent estimate of the robot motion. The trajectory and frame alignment are coherent, while small differences are due to odometric drift, wheel slip, and modeling errors. For the following Test, the same laucher used in the previous test is valid.



RViz odometry and TF visualization



Real experimental setup

Figure 7.3: Comparison between the real robot experiment and the corresponding odometry visualization in RViz.

Test 4: Waypoint Tracking Validation with the `go_to_point` Controller

This test represented the first closed-loop high-level motion-control experiment performed within the migrated ROS2 architecture. In this phase, the package `sdpo_motion_control` was used to command the robot toward a sequence of planar target poses, with the objective of evaluating the behaviour of the `go_to_point` controller, the consistency of the odometric feedback, and the overall quality of the integrated control chain.

This test was carried out after correcting an inconsistency in the `map`–`odom` transformation inside the localization chain. In the previous implementation, the rotation between `map` and `odom` was computed correctly, but the translation term was combined with the robot EKF yaw instead of the relative `map`–`odom` rotation. This produced a frame inconsistency. After this correction, the waypoint experiments became coherent and repeatable.

Test configuration For controller tuning and validation, six reference poses were assigned sequentially. Their spatial arrangement and execution order are shown in Figure 7.4. The same six target positions are used in the plots showing the reference trajectory, the actual robot trajectory, and the corresponding tracking errors:

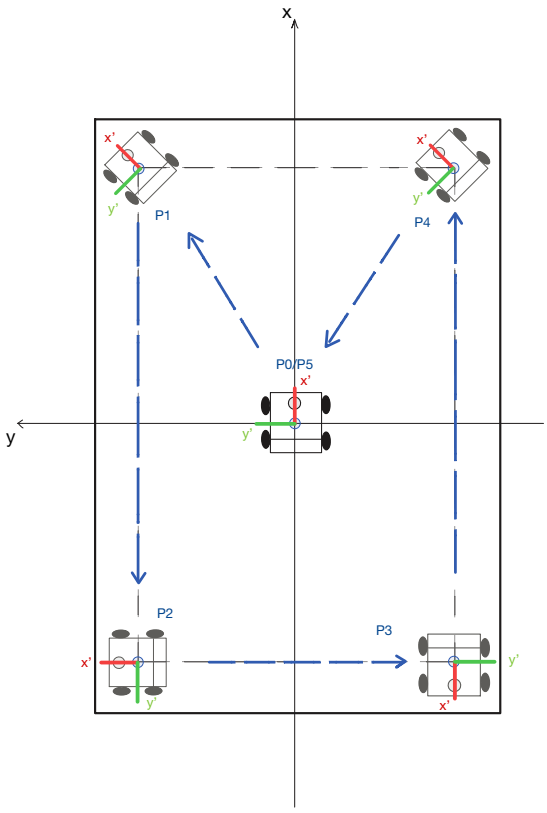


Figure 7.4: diagram of the test executed

- (P0) {x: 0.0, y: 0.0, z: 0.0}
- (P1) {x: 0.6, y: 0.45, z: 0.78}
- (P2) {x: -0.6, y: 0.45, z: 1.56}
- (P3) {x: -0.6, y: -0.45, z: 3.14}
- (P4) {x: 0.6, y: -0.45, z: 0.78}
- (P5) {x: 0.0, y: 0.0, z: 0.0}

The sequence was chosen to force the robot to move in different directions of the workspace, including diagonal displacements where odometric errors and wheel slip are more visible, ending with a return to the origin.

Observed behaviour The experiments showed that the `go_to_point` controller was able to drive the robot through the assigned sequence of reference poses shown in Figure 7.5 and to reach all the commanded poses while maintaining stable behaviour. Motions along the main axes were generally more accurate.

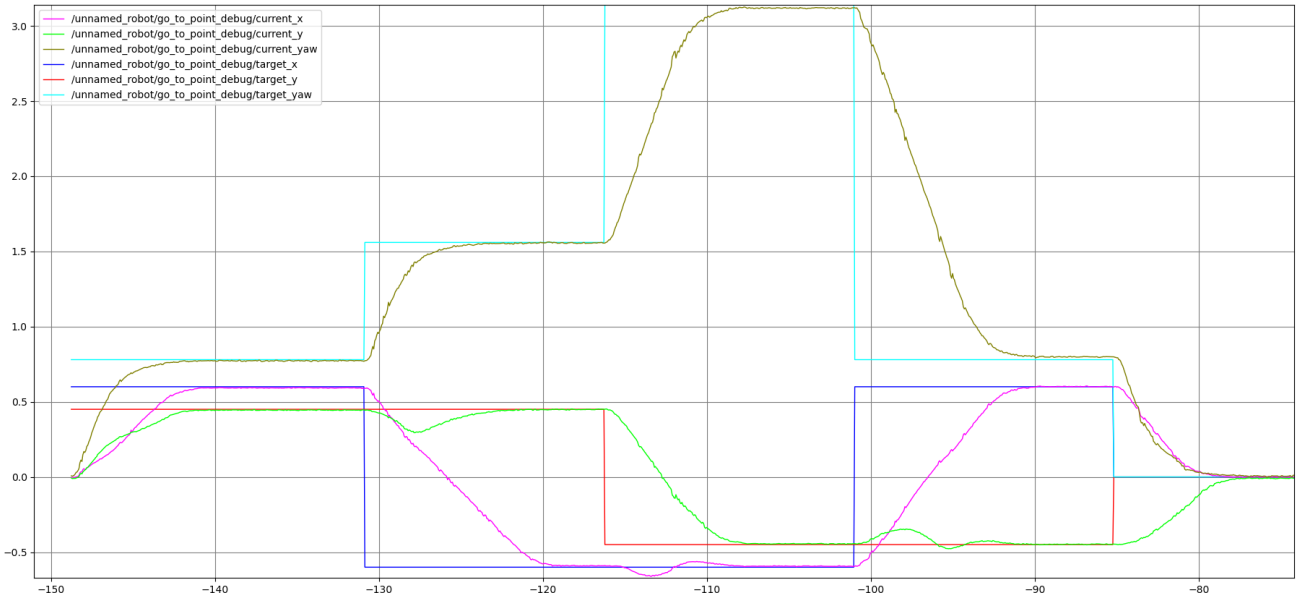


Figure 7.5: Position control results.

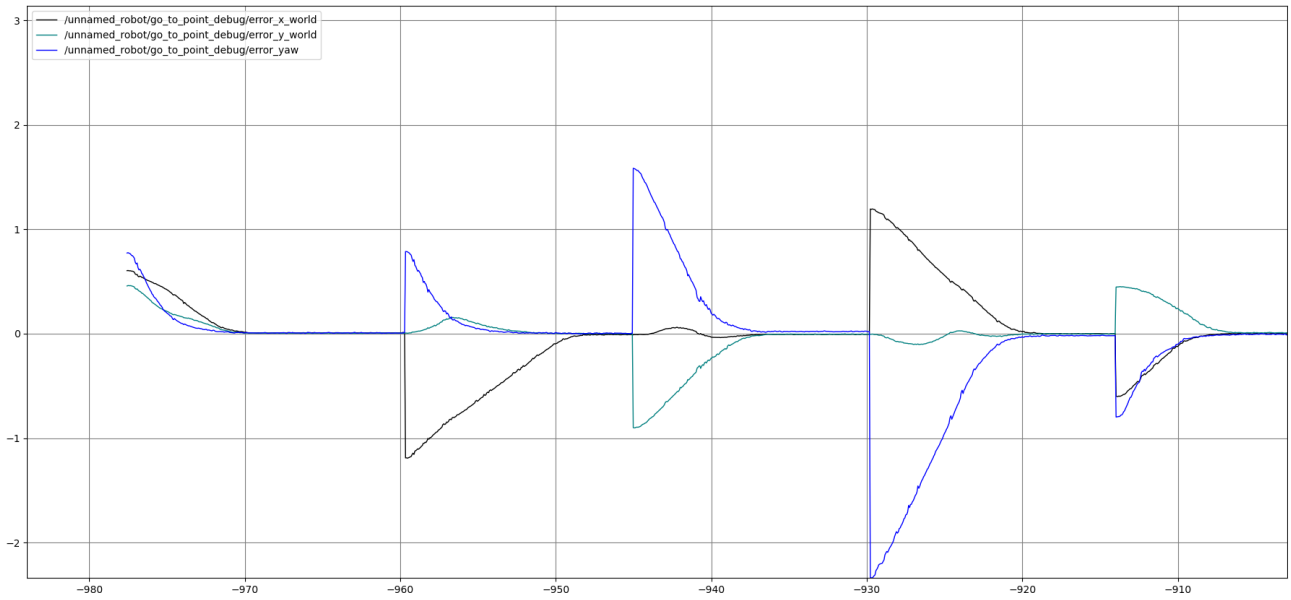


Figure 7.6: Position error results.

The comparison between the commanded target pose and the estimated current pose, together with the plots of the tracking error, highlighted that the control loop remained stable during all tests, but also confirmed the presence of a residual positioning error, as shown in Figure 7.6. Over the waypoint sequence, the largest observed deviation was on the order of 2 cm on each axis. This discrepancy was not mainly associated with instability of the controller itself, but rather with the accumulated estimation error of the pose used by the controller.

The results were particularly useful because they helped separate the effect of control tuning from the effect of state estimation. In fact, the tests confirmed that the basic control chain `goal` → `controller` → `cmd_vel` → `motors_ref` → robot is functionally correct, while the main limitation at this stage is the residual drift and bias of the pose estimate over longer or diagonal displacements.

Discussion of the results From a control point of view, these experiments represent the first successful validation of waypoint regulation in the migrated ROS2 architecture. The robot was able to execute repeated planar repositioning tasks using the new `sdpo_motion_control` package, and the obtained behaviour is already sufficient to consider the controller as a valid baseline for further tuning and for future trajectory-tracking developments.

At the same time, the tests showed that improved precision will require not only controller tuning, but also a more accurate calibration of the localization and odometry chain, especially for diagonal motions and global return-to-origin tasks. For this reason, Test 4 was an important milestone: it validated the complete waypoint control loop in

ROS2 and clearly identified the remaining gap between acceptable closed-loop operation and higher-accuracy autonomous navigation.

7.1.4 Final Considerations on the Integrated ROS2 Chain

From a control perspective, the `go_to_point` validation was carried out within a cascaded control architecture [2, 19], where the inner loop was already introduced in the previous chapters.

At the embedded level, the ESP32 firmware implements the low-level wheel-velocity regulation loop, including the motor control logic discussed in Chapter 6. This inner loop tracks wheel-speed references and provides the actuation layer required by the ROS2 motion stack. As a result, the high-level controller running on the Raspberry Pi can be designed at the robot-motion level, in terms of planar pose regulation.

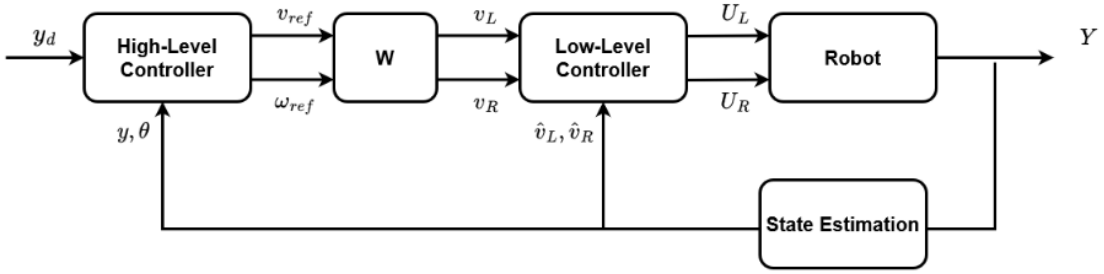


Figure 7.7: Controller Architecture

Motion-control architecture adopted for OMNICAR as a two-layer cascade consistent with the firmware and ROS2 integration described in the previous chapters.

- **High-level controller**: receives the estimated robot pose (x, y, θ) from the state-estimation pipeline and computes robot-level reference velocities for motion tasks (initially waypoint regulation, later path-following scenarios). These references are then mapped to wheel-level commands through the kinematic transformation block W.
- **Low-level controller**: implemented on the ESP32, receives wheel-velocity references and applies closed-loop motor control to track them through PWM/DIR actuation, using encoder feedback for regulation.

This architecture, shown in Fig. 7.7, provides a clear separation of responsibilities: high-level motion generation in ROS2 and fast embedded actuation at firmware level. In the present chapter, the high-level scenario is limited to a `go_to_point` task with a P-only outer loop.

In the ROS2 experiments presented in Test 4, the high-level outer loop was intentionally kept simple and implemented as a **P-only controller** on the planar coordinates

(x, y, ψ). Three proportional actions were used to generate the commanded robot velocities from position and heading errors, while integral and derivative terms were disabled in this phase.

The tuning was performed progressively by starting from moderate proportional gains (around $K_p \approx 0.8$, then adjusted per axis) and increasing them until a satisfactory compromise between responsiveness and oscillation margin was obtained. This approach was chosen to obtain a robust baseline controller and to isolate the effect of the localization and odometry chain on the final positioning performance.

Experimental results confirmed that this P-only outer loop is sufficient to achieve stable waypoint reaching and repeatable closed-loop behavior for the considered trajectories. Residual errors, especially during diagonal motions and return-to-origin phases, were mainly associated with state-estimation drift and model mismatch rather than with instability of the proportional control law itself.

This test should therefore be interpreted as the first high-level control scenario in the migrated ROS2 architecture: a deliberately simple and transparent baseline. In the next chapter, more advanced high-level control scenarios will be analyzed.

Chapter 8

Motion Control Design and Experimental Evaluation

The high-level control loop is implemented in ROS2 inside the package `sdpo_motion_control`. Its role is to compute planar velocity commands for the omnidirectional robot from the measured pose and from the desired path. The output of this layer is a body-frame velocity command:

$$\mathbf{u} = \begin{bmatrix} v_x & v_y & \omega \end{bmatrix}^T, \tag{8.1}$$

where v_x is the linear velocity component along the robot x_b-axis, v_y is the linear velocity component along the robot y_b-axis, and ω is the yaw rate around the vertical axis. Therefore, v_x and v_y are not two independent wheel speeds, but the two planar components of the robot velocity in the body frame [1, 2]. The robot pose in the global frame is defined as:

$$\mathbf{x} = \begin{bmatrix} x & y & \psi \end{bmatrix}^T, \tag{8.2}$$

where x and y are the robot position and ψ is the yaw angle. The first two controllers are geometric and non-optimizing, they do not directly solve an optimization problem. Instead, they use geometric quantities such as path projection, path tangent, cross-track error, and along-track error. The third one (MPCC) is optimization-based.

Three controllers are analysed:

- the **Moving Reference proportional controller**, implemented as `moving_reference_p`;
- the **Linear Segment controller**, implemented as `linear_segment`.
- the **Nonlinear Model Predictive Contouring Controller**, implemented as `nonlinear_mpcc`.

For each controller, the control law, the main parameters, the test setup, the quantitative results, and the experimental figures are reported.

8.1 Mathematical Model of the OMNICAR Platform

8.2 Evaluation Protocol and Metrics

The experiments are evaluated using the recorded path-following datasets stored in `Test/test_results/`. The main performance indicators are computed from the sampled tracking errors [19]. For a generic error signal e_i, with N samples, we can calculate:
- The root mean square errot is:

$$\mathrm{RMSE}_e = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_i^2}. \tag{8.3}$$

- The mean absolute error is:

$$\mathrm{MAE}_e = \frac{1}{N}\sum_{i=1}^{N} |e_i|. \tag{8.4}$$

- The maximum absolute error is:

$$e_{\max} = \max_{i=1,\dots,N} |e_i|. \tag{8.5}$$

The path progression ratio is used to understand how much of the reference path was covered during the test:

$$r_\gamma = \frac{\gamma_{\text{final}}}{L_{\text{path}}}, \tag{8.6}$$

where γ_{final} is the final curvilinear abscissa reached by the reference or projection point, and L_{path} is the total path length. A value close to one means that the robot completed almost the whole path. The velocity saturation ratios are also reported:

$$r_{\text{sat},v_x}, \quad r_{\text{sat},v_y}, \quad r_{\text{sat},\omega}. \tag{8.7}$$

These values indicate the fraction of samples in which the corresponding velocity command reached its imposed limit.

8.3 Moving Reference Proportional Controller

8.3.1 Control Principle

The Moving Reference controller tracks a virtual point that moves along the path [1]. The path is described by a parametric curve:

$$\mathbf{p}_r(\gamma) = \begin{bmatrix} x_r(\gamma) \\ y_r(\gamma) \end{bmatrix}, \tag{8.8}$$

where γ is the path coordinate. The reference point advances according to:

$$\dot{\gamma} = \mathrm{sat}\left(v_d + k_\gamma e_\parallel, \dot{\gamma}_{\min}, \dot{\gamma}_{\max}\right), \tag{8.9}$$

where v_d is the desired path speed, k_γ is the progression gain, and $e_{\|}$ is the along-track error. The saturation function limits the path progression between $\dot{\gamma}_{\min}$ and $\dot{\gamma}_{\max}$. The position error in the global frame is:

$$\mathbf{e}_w = \mathbf{p}_r(\gamma) - \mathbf{p}, \qquad \mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix} \tag{8.10}$$

Since the robot receives velocity commands in its body frame, the error is rotated using the yaw angle:

$$\mathbf{e}_b = \mathbf{R}^T(\psi)\mathbf{e}_w, \tag{8.11}$$

with:

$$\mathbf{R}(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{bmatrix} \tag{8.12}$$

The proportional control law is:

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} k_x e_{b,x} \\ k_y e_{b,y} \end{bmatrix} \tag{8.13}$$

The heading is controlled independently with a proportional law:

$$\omega = k_\psi e_\psi, \tag{8.14}$$

where:

$$e_\psi = \mathrm{wrap}_{[-\pi,\pi]}(\psi_r - \psi)\,, \qquad e_\psi \in [-\pi, \pi] \tag{8.15}$$

$\mathrm{wrap}_{[-\pi,\pi]}$ maps any angle to its principal value in $[-\pi, \pi]$, so that the heading controller uses the shortest angular error.
Finally, the command is saturated to respect the velocity limits:

$$v_x \in [-v_{x,\max}, v_{x,\max}], \quad v_y \in [-v_{y,\max}, v_{y,\max}], \quad \omega \in [-\omega_{\max}, \omega_{\max}] \tag{8.16}$$

This controller is simple and robust, but its performance strongly depends on the balance between the path progression speed and the feedback gains. If the virtual point moves too fast, the robot follows the path with a delay. If it moves too slowly, the robot can track accurately but the test becomes slower.

8.3.2 Test Parameters and Results

Test legend:

- **MR-R01**: `PF_2026-04-27_RECT_MAP_MREFP_V01_R01`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.1 and 8.2.

- **MR-R01**: `PF_2026-04-27_RECT_MAP_PONLY_V01_R01`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.3 and 8.4.

- **P-R02**:

  `PF_2026-04-27_RECT_MAP_PONLY_V01_R02`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.5 and 8.6.

Table 8.1: Moving Reference controller: main test parameters.

Parameter	**MR-R01**	**P-R01**	**P-R02**
Path	RECT	RECT	RECT
Frame	MAP	MAP	MAP
Controller mode	`moving_reference_p`	`moving_reference_p`	`moving_reference_p`
Heading mode	fixed	fixed	fixed
v_d [m/s]	0.35	0.35	0.35
k_γ	0.50	0.50	0.50
$\dot{\gamma}_{\min}$ [m/s]	0.10	0.10	0.10
$\dot{\gamma}_{\max}$ [m/s]	0.35	0.35	0.35
Control rate [Hz]	50	50	50
kp_x	0.50	0.80	0.50
kp_y	0.60	0.90	0.50
kp_{yaw}	0.50	0.50	0.50
$v_{x,\max}$ [m/s]	0.40	0.40	0.40
$v_{y,\max}$ [m/s]	0.40	0.40	0.40
$\omega_{\max}$ [rad/s]	0.35	0.35	0.35

Table 8.2: Moving Reference controller: experimental results.

Metric	**MR-R01**	**P-R01**	**P-R02**
Duration [s]	30.56	16.12	16.28
Samples	975	515	514
Mean sample time [s]	0.0314	0.0314	0.0317
Real path length [m]	8.1990	4.9195	4.7319
Final γ [m]	1.5230	0.6813	2.9163
Path progress ratio r_γ	0.3626	0.1622	0.6944
RMSE_x [m]	0.0509	0.0455	0.0556
RMSE_y [m]	0.1021	0.0930	0.0609
RMSE_ψ [rad]	0.0141	0.0148	0.0147
$\text{RMSE}_{e_\parallel}$ [m]	0.0701	0.0629	0.0772
$\text{MAE}_{e_\parallel}$ [m]	0.0593	0.0548	0.0704
$\max \lvert e_\parallel \rvert$ [m]	0.1163	0.1124	0.1190
$\text{IAE}_{e_\parallel}$	1.8162	0.8878	1.1433
$\max \lvert e_\psi \rvert$ [rad]	0.0344	0.0447	0.0304
Saturation ratio v_x	0.00	0.00	0.00
Saturation ratio v_y	0.00	0.00	0.00
Saturation ratio ω	0.00	0.00	0.00

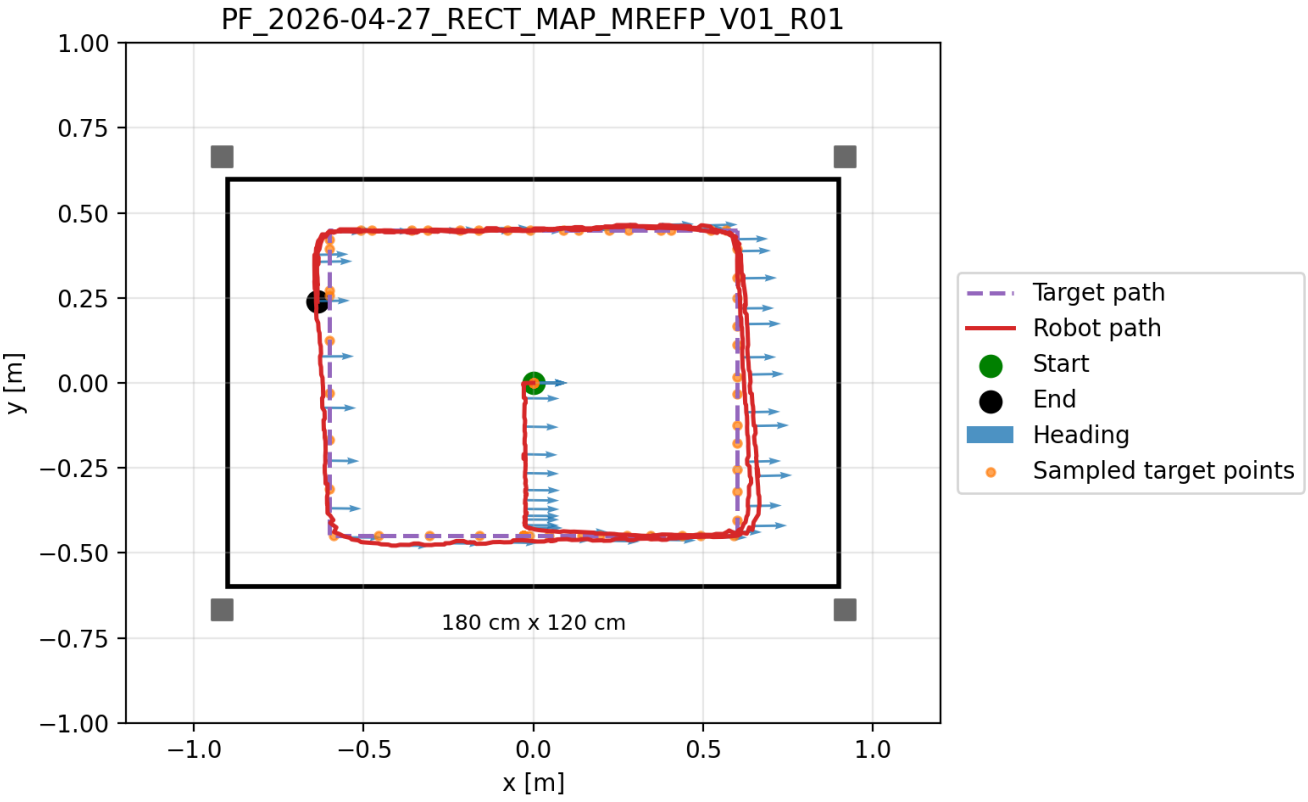


Figure 8.1: XY map for the MR_R01 scenario.

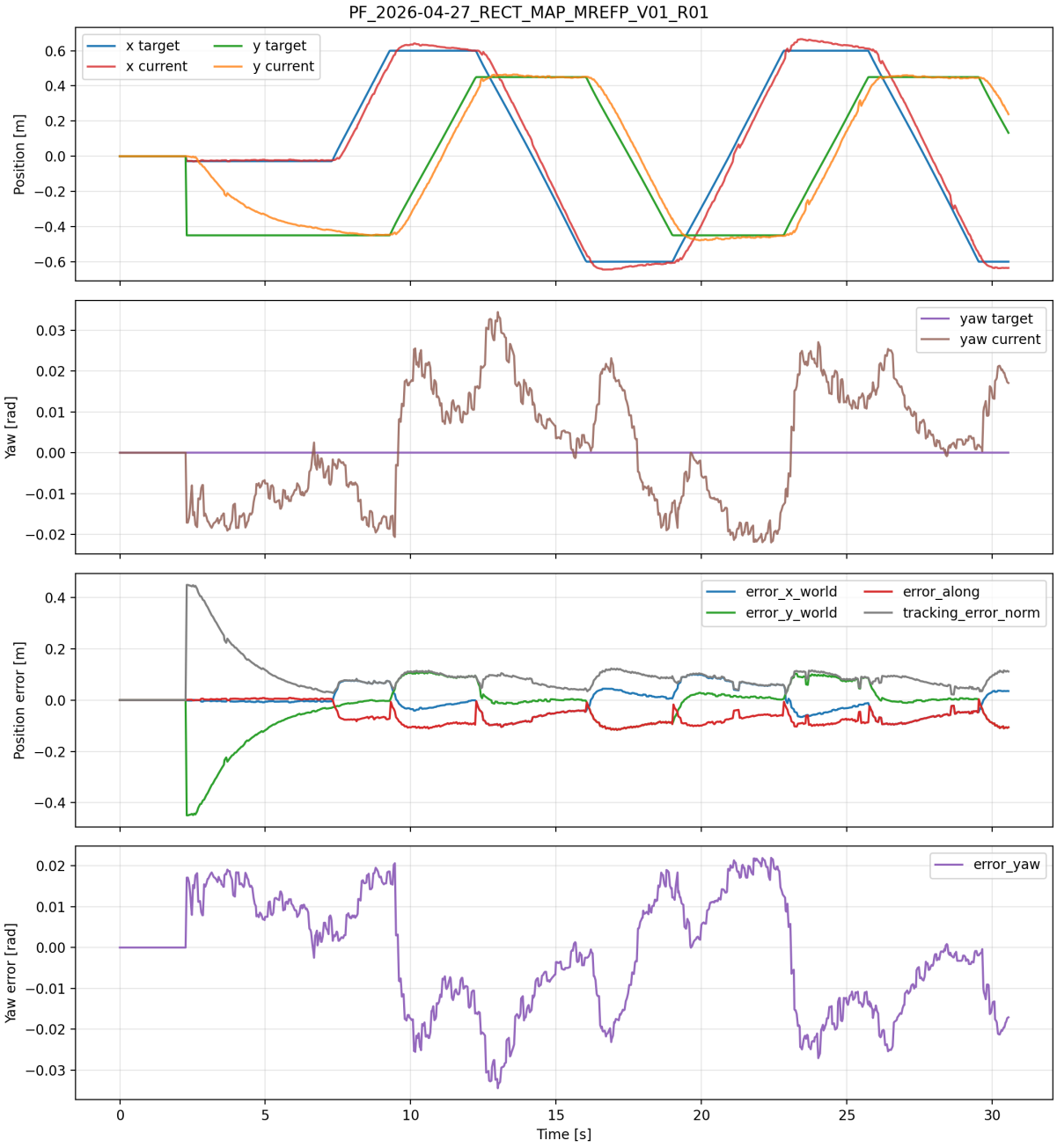


Figure 8.2: Target tracking and error results for the MR_R01 scenario.

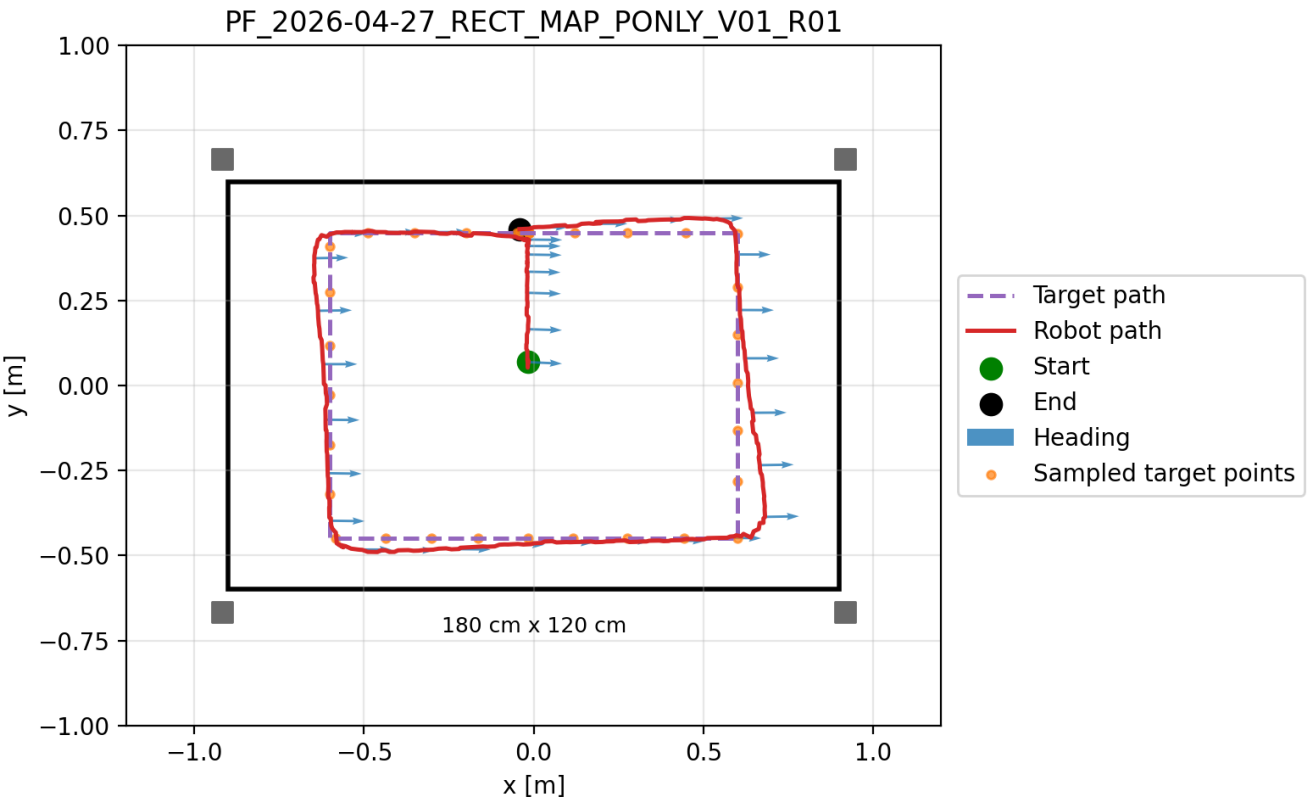


Figure 8.3: XY map for the P_R01 scenario.

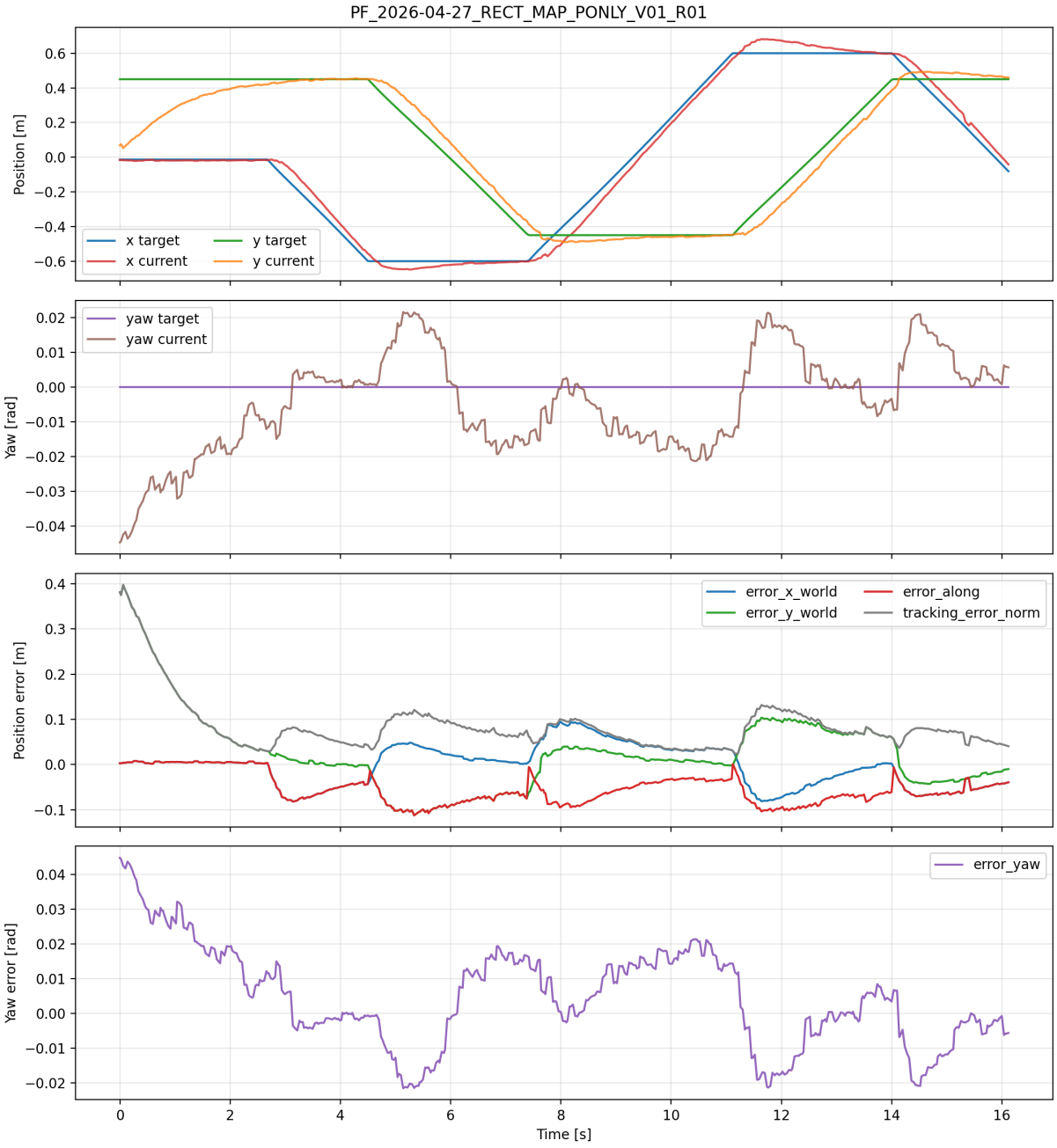


Figure 8.4: Target tracking and error results for the P_R01 scenario.

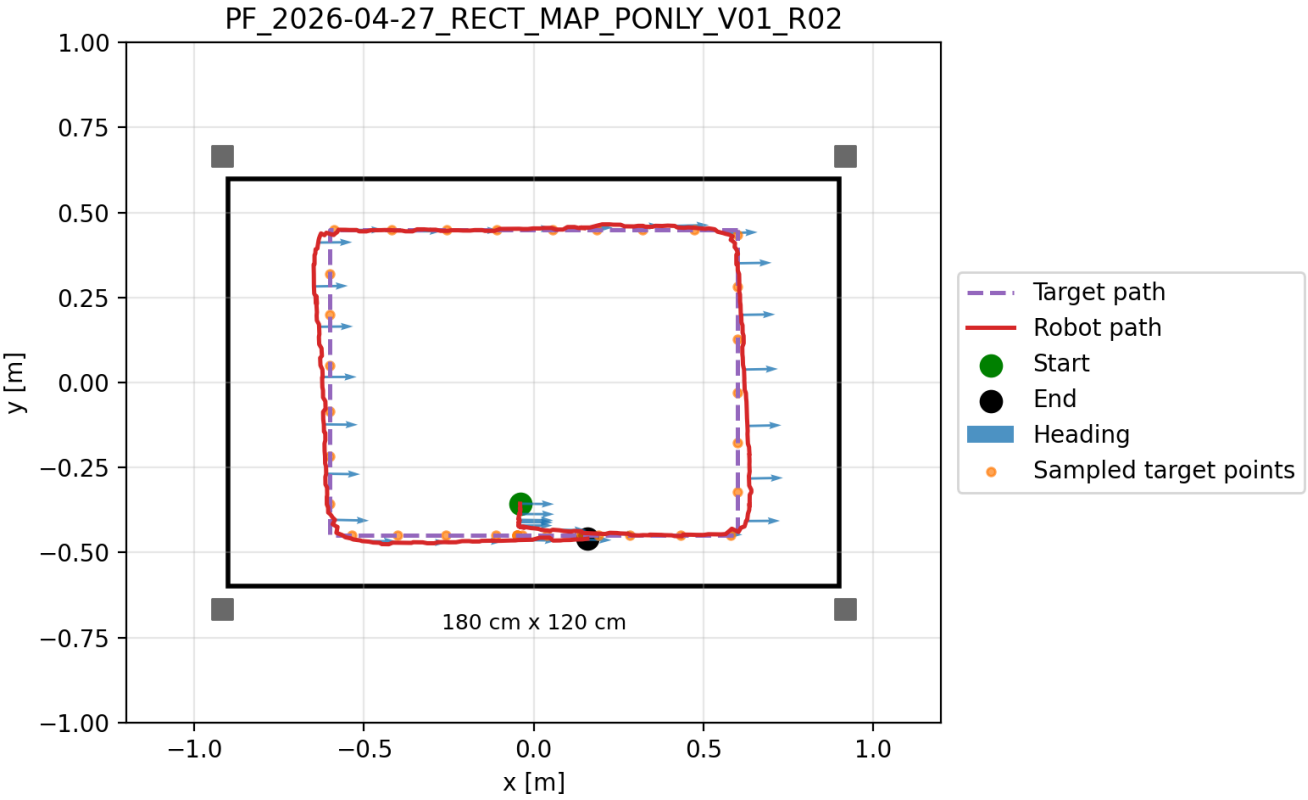


Figure 8.5: XY map for the P_R02 scenario.

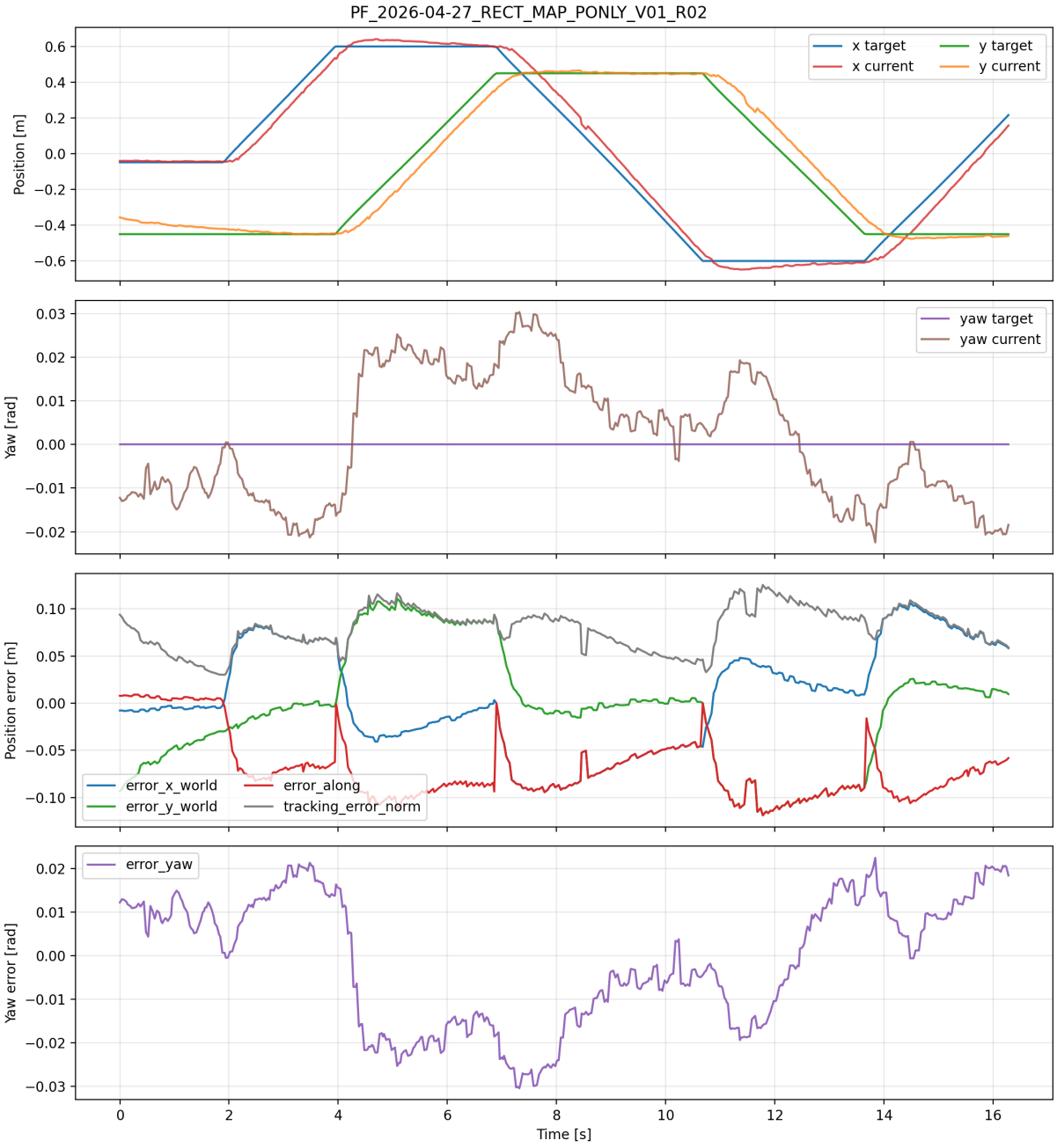


Figure 8.6: Target tracking and error results for the P_R02 scenario.

Commands to use moving_reference

```
CMD 1
    ros2 launch sdpo_ratf_driver omnicar_bringup.launch.py

CMD 2
    ros2 launch sdpo_motion_control path_follower_map.launch.
       py \
    robot_id:=<ROBOT_ID> \
    control_frame_id:=<ROBOT_ID>/map \
    heading_mode:=<HEADING_MODE> \
    path_file:="<PATH_FILE>" \
    record_bag:=true \
    test_name:=<TEST_NAME> \
    controller_mode:=<CONTROLLER_MODE> \
    bag_directory:="<BAG_DIRECTORY>"
```

Example values for moving_reference command

```

<ROBOT_ID>          = unnamed_robot
<HEADING_MODE>      = fixed
<PATH_FILE>         = /home/c2sr/Omnicar_Control_Ros2/[02]Code
   /omnicar_ws/src/splines/FEUP/FEUP_rounded_rect_eight.csv
<TEST_NAME>         = PF_2026-04-28_RECT_MAP_MREFP_V01_R01
<CONTROLLER_MODE>   = moving_reference_p
<BAG_DIRECTORY>     = /home/c2sr/Omnicar_Control_Ros2/[02]Code
   /omnicar_ws/bags
```

Commands to analyze test records

```
python3 "<ANALYSIS_SCRIPT>" \
    --bag "<BAG_PATH>" \
    --test-id <TEST_ID> \
    --path-name <PATH_NAME> \
    --frame <REFERENCE_FRAME> \
    --controller-name <CONTROLLER_NAME> \
    --controller-version <CONTROLLER_VERSION> \
    --path-file "<PATH_FILE>" \
    --config "<ANALYSIS_CONFIG>" \
    --output-root "<OUTPUT_ROOT>" \
    --debug-topic /<ROBOT_ID>/path_follower_debug \
    --force
```

Example values for test analysis command

```
<ANALYSIS_SCRIPT>     = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_code/scripts/
   analyze_path_following_bag.py

<BAG_PATH>            = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/bags/PF_2026-04-29
   _INFIN_MAP_LINEAR_V01_R10

<TEST_ID>             = PF_2026-04-28_RECT_MAP_MREFP_V01_R01
<PATH_NAME>           = INFIN
<REFERENCE_FRAME>     = MAP
<CONTROLLER_VERSION>  = V01

<PATH_FILE>           = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/src/splines/FEUP/FEUP_rounded_rect_eight.
   csv

<ANALYSIS_CONFIG>     = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_code/config/analysis_config.yaml

<OUTPUT_ROOT>         = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_results

<ROBOT_ID>            = unnamed_robot
```

8.4 Linear Segment Controller

8.4.1 Control Principle

The Linear Segment controller uses the local geometry of the path [1]. At each control step, the robot position is projected onto the current path segment. Given two consecutive path points:

$$\mathbf{p}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix}, \qquad \mathbf{p}_{i+1} = \begin{bmatrix} x_{i+1} \\ y_{i+1} \end{bmatrix}, \tag{8.17}$$

the segment vector is:

$$\mathbf{s}_i = \mathbf{p}_{i+1} - \mathbf{p}_i \tag{8.18}$$

The unit tangent vector is:

$$\mathbf{t}_i = \frac{\mathbf{s}_i}{\|\mathbf{s}_i\|} \tag{8.19}$$

The normal vector is obtained by rotating the tangent by 90°:

$$\mathbf{n}_i = \begin{bmatrix} -t_{i,y} \\ t_{i,x} \end{bmatrix} \tag{8.20}$$

The projection coefficient is:

$$\alpha = \frac{(\mathbf{p} - \mathbf{p}_i)^T \mathbf{s}_i}{\mathbf{s}_i^T \mathbf{s}_i} \tag{8.21}$$

To keep the projected point inside the segment, α is limited:

$$\alpha_c = \mathrm{sat}(\alpha, 0, 1) \tag{8.22}$$

The closest point on the segment is:

$$\mathbf{p}_c = \mathbf{p}_i + \alpha_c \mathbf{s}_i \tag{8.23}$$

The position error is:

$$\mathbf{e} = \mathbf{p}_c - \mathbf{p} \tag{8.24}$$

This error is decomposed into an along-track component and a lateral component:

$$e_{\parallel} = \mathbf{e}^T \mathbf{t}_i, \qquad e_{\perp} = \mathbf{e}^T \mathbf{n}_i \tag{8.25}$$

The desired velocity in the world frame is:

$$\mathbf{v}_w = v_d \mathbf{t}_i + k_{\parallel} e_{\parallel} \mathbf{t}_i + k_{\perp} e_{\perp} \mathbf{n}_i \tag{8.26}$$

The first term moves the robot along the path. The second term corrects the longitudinal error. The third term corrects the lateral error and is the main term used to reduce the

cross-track deviation. Since the robot command is expressed in body frame, the world-frame velocity is rotated using:

$$\mathbf{v}_b = \mathbf{R}^T(\psi)\mathbf{v}_w \tag{8.27}$$

Therefore:

$$v_x = v_{b,x}, \qquad v_y = v_{b,y} \tag{8.28}$$

The yaw motion is controlled separately:

$$\omega = k_\psi \, \mathrm{wrap}_{[-\pi,\pi]}(\psi_r - \psi), \qquad \omega \in [-\pi, \pi] \tag{8.29}$$

This formulation is useful for an omnidirectional robot because longitudinal motion and lateral correction can be commanded at the same time. For this reason, the robot does not need to align its body with the path before reducing the lateral error.

8.4.2 Test Parameters

Test legend:

- **LIN-01**: `PF_2026-04-28_INFIN_MAP_LINEAR_V01_R01`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.7 and 8.8.
- **LIN-02**: `PF_2026-04-29_INFIN_MAP_LINEAR_V01_R02`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.9 and 8.10.
- **LIN-03**: `PF_2026-04-29_INFIN_MAP_LINEAR_V01_R07`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.11 and 8.12.
- **LIN-04**: `PF_2026-04-29_INFIN_MAP_LINEAR_V01_R08`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.13 and 8.14.
- **LIN-05**: `PF_2026-04-29_INFIN_MAP_LINEAR_V01_R10`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.15 and 8.16.

Table 8.3: Linear Segment controller: configuration parameters of the selected tests.

Parameter	**LIN-01**	**LIN-02**	**LIN-03**	**LIN-04**	**LIN-05**
Path	INFIN	INFIN	INFIN	INFIN	INFIN
Frame	MAP	MAP	MAP	MAP	MAP
Heading mode	fixed	fixed	fixed	fixed	fixed
v_d [m/s]	0.35	0.20	0.30	0.40	0.50
k_γ	0.20	0.20	0.20	0.30	0.40
$\dot{\gamma}_{\min}$ [m/s]	0.10	0.05	0.05	0.05	0.05
$\dot{\gamma}_{\max}$ [m/s]	0.35	0.20	0.20	0.30	0.30
$k_\parallel$	0.00	0.40	1.00	1.00	1.00
$k_\perp$	0.80	3.00	5.00	5.00	8.00
Control rate [Hz]	50	50	50	50	50

Table 8.4: Linear Segment controller: experimental results of the selected tests.

Metric	**LIN-01**	**LIN-02**	**LIN-03**	**LIN-04**	**LIN-05**
Duration [s]	23.80	39.20	32.28	27.39	22.58
Samples	753	1268	1020	853	716
Mean sample time [s]	0.0316	0.0309	0.0317	0.0321	0.0316
Real path length [m]	7.7631	7.7922	9.6978	10.7535	11.6237
Final γ [m]	2.0687	4.4860	2.0970	2.1181	2.1832
Path progress ratio r_γ	0.4542	0.9849	0.4604	0.4650	0.4793
RMSE$_x$ [m]	0.0720	0.0125	0.0172	0.0270	0.0396
RMSE$_y$ [m]	0.0586	0.0113	0.0148	0.0228	0.0252
RMSE$_\psi$ [rad]	0.0139	0.0082	0.0120	0.0187	0.0324
RMSE$_{e_\parallel}$ [m]	0.0028	0.0003	0.0005	0.0005	0.0009
MAE$_{e_\parallel}$ [m]	0.0009	0.00005	0.00008	0.00010	0.00020
$\max\lvert e_\parallel\rvert$ [m]	0.0217	0.0037	0.0049	0.0057	0.0104
IAE$_{e_\parallel}$	0.0206	0.0020	0.0025	0.0027	0.0044
$\max\lvert e_\psi\rvert$ [rad]	0.0340	0.0264	0.0310	0.0464	0.0745
Saturation ratio v_x	0.0000	0.0000	0.0186	0.0668	0.2304
Saturation ratio v_y	0.0000	0.0000	0.0461	0.0961	0.1955
Saturation ratio ω	0.0000	0.0000	0.0000	0.0000	0.0000

The experimental validation analyzes the impact of varying desired speeds (v_d) and feedback gains ($k_\parallel, k_\perp$) on the Linear Segment controller. All tests were conducted on an "infinity" path within the MAP frame, utilizing a fixed-heading mode. The primary objective is to evaluate the trade-off between path progression, tracking accuracy, and control effort.

Baseline and Tuning Impact (LIN-01) The baseline configuration, **LIN-01** ($v_d = 0.35$ m/s), employs a weak feedback action with $k_{\parallel} = 0$ and $k_{\perp} = 0.8$. This setup results in the poorest tracking performance among the selected runs:

$$\text{RMSE}_x = 0.0720 \text{ m}, \quad \text{RMSE}_y = 0.0586 \text{ m}, \quad \text{RMSE}_{e_{\parallel}} = 0.0028 \text{ m}$$

Since saturation ratios remain at zero, the high tracking error is not due to physical actuator limits but is directly attributable to the insufficient feedback gains, particularly the absence of longitudinal compensation ($k_{\parallel} = 0$).

Optimal Performance (LIN-02) The most balanced results are achieved in **LIN-02**. By reducing the speed to $v_d = 0.20$ m/s and increasing the gains to $k_{\parallel} = 0.4$ and $k_{\perp} = 3.0$, the tracking errors decrease significantly:

$$\text{RMSE}_x = 0.0125 \text{ m}, \quad \text{RMSE}_y = 0.0113 \text{ m}, \quad \text{RMSE}_{e_{\parallel}} = 0.0003 \text{ m}$$

This configuration provides high path completion and negligible error without encountering velocity saturation, establishing a robust benchmark for reliable autonomous navigation.

High-Speed Scenarios and Actuator Limits (LIN-03 to LIN-05) As the desired speed increases, a clear degradation in performance and an increase in control effort are observed:

- **LIN-03 ($v_d = 0.30$ m/s):** Despite maintaining good local accuracy, the path progress ratio drops to $r_{\gamma} = 0.4604$. The controller begins to approach the actuation limits, with saturation ratios reaching 1.86% on v_x and 4.61% on v_y.
- **LIN-04 ($v_d = 0.40$ m/s):** The system nears its operational limits. Position errors and heading divergence increase ($\text{RMSE}_{\psi} = 0.0187$ rad), while saturation becomes more frequent ($r_{\text{sat},v_y} \approx 9.61\%$).
- **LIN-05 ($v_d = 0.50$ m/s):** In the fastest test, the controller hits significant saturation ($\approx 23\%$ on v_x). This prevents the full application of corrective commands, leading to the highest observed errors:

$$\text{RMSE}_x = 0.0396 \text{ m}, \quad \max|e_{\psi}| = 0.0745 \text{ rad}$$

Concluding Remarks The results demonstrate a direct correlation between target velocity and tracking stability. While **LIN-02** represents the ideal operational point for the current tuning, the high-speed tests highlight the physical and algorithmic boundaries of the system. To maintain accuracy at higher velocities, future developments should consider adaptive path progression or the introduction of feedforward terms to mitigate the lag induced by high-gain feedback during saturation.

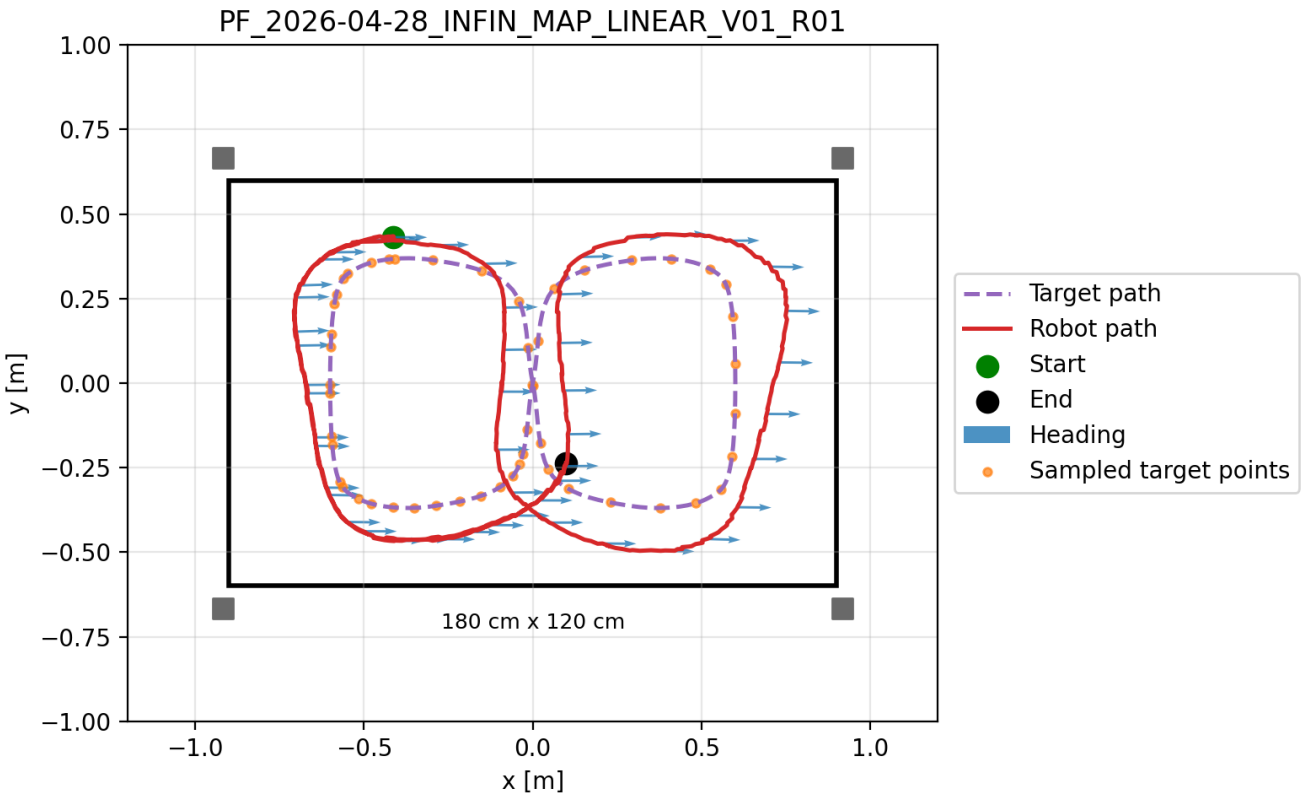


Figure 8.7: XY map for the LIN_01 scenario.

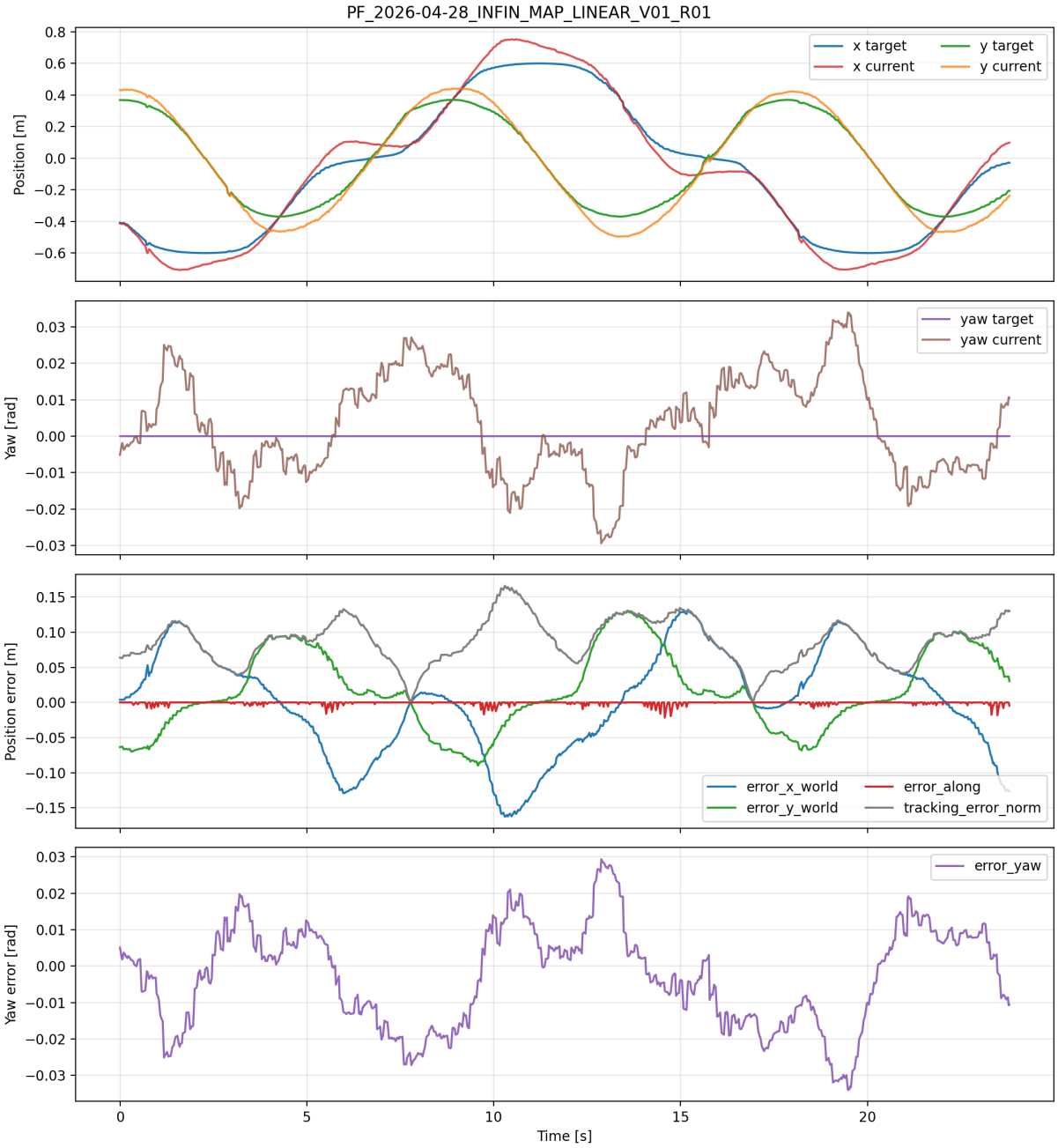


Figure 8.8: Target tracking and error results for the LIN_01 scenario.

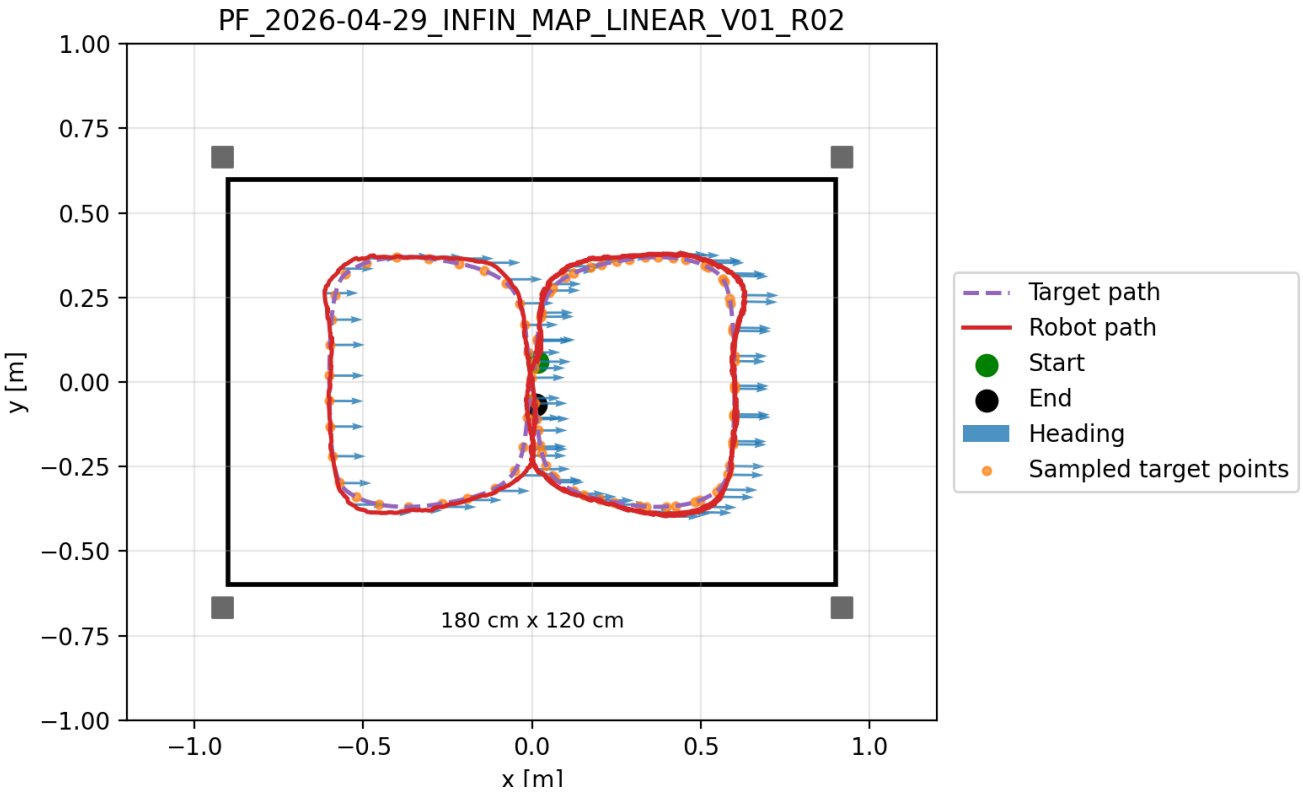


Figure 8.9: XY map for the LIN_02 scenario.

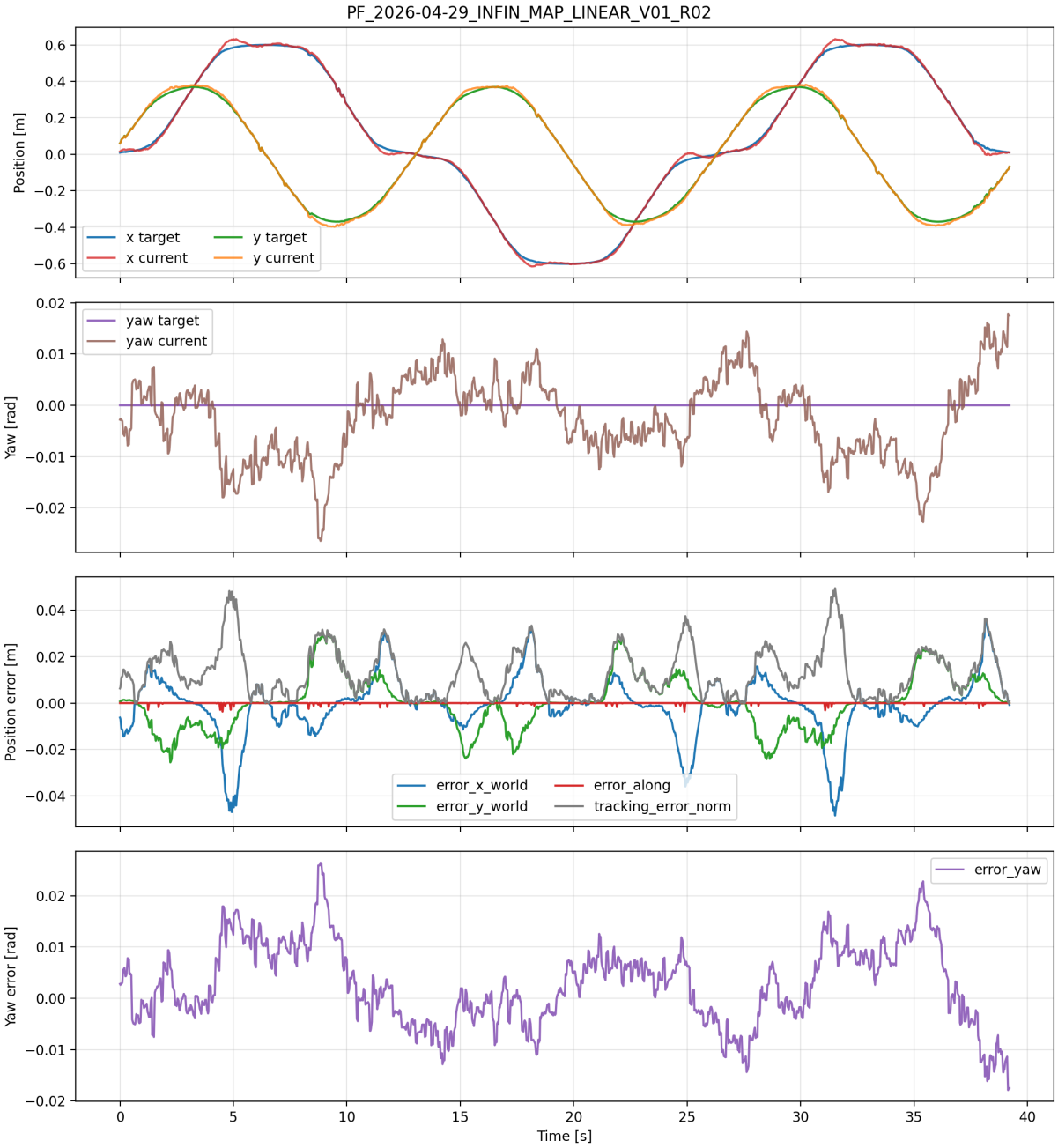


Figure 8.10: Target tracking and error results for the LIN_02 scenario.

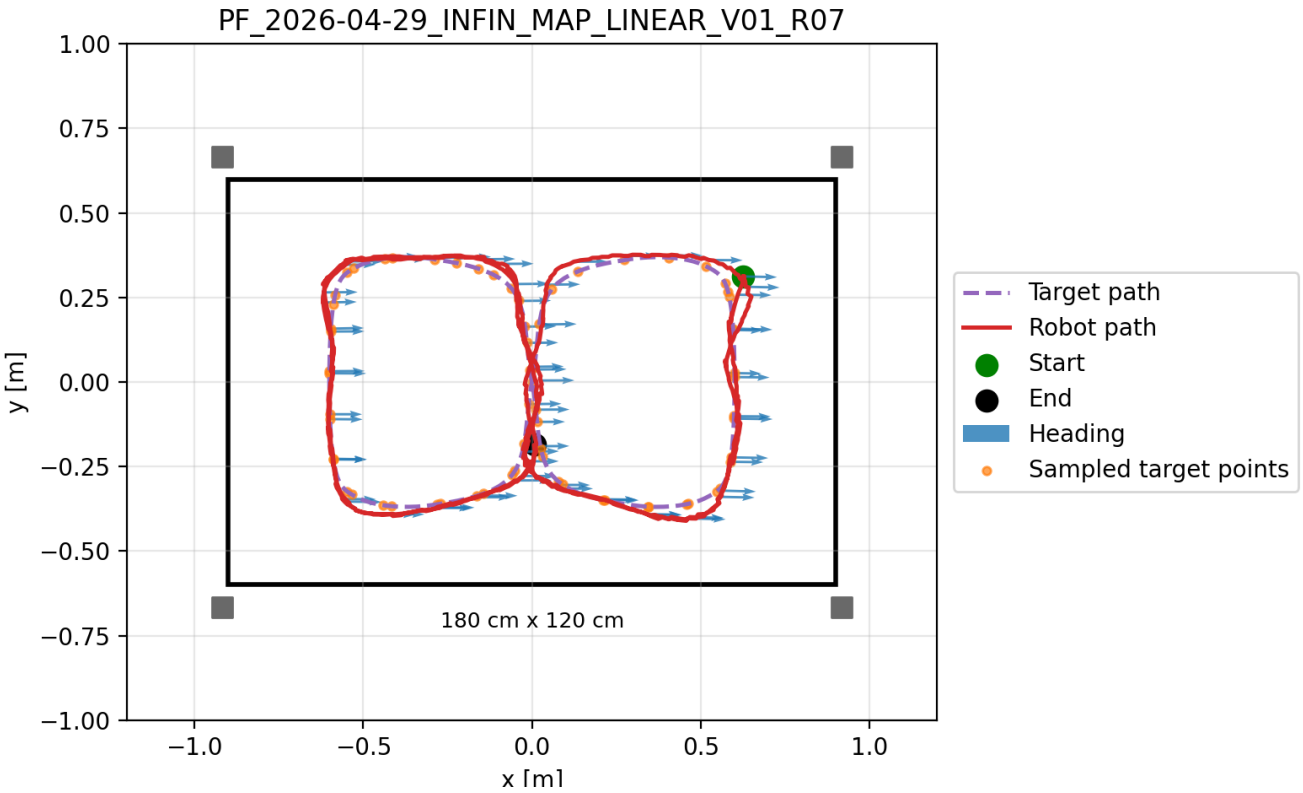


Figure 8.11: XY map for the LIN_03 scenario.

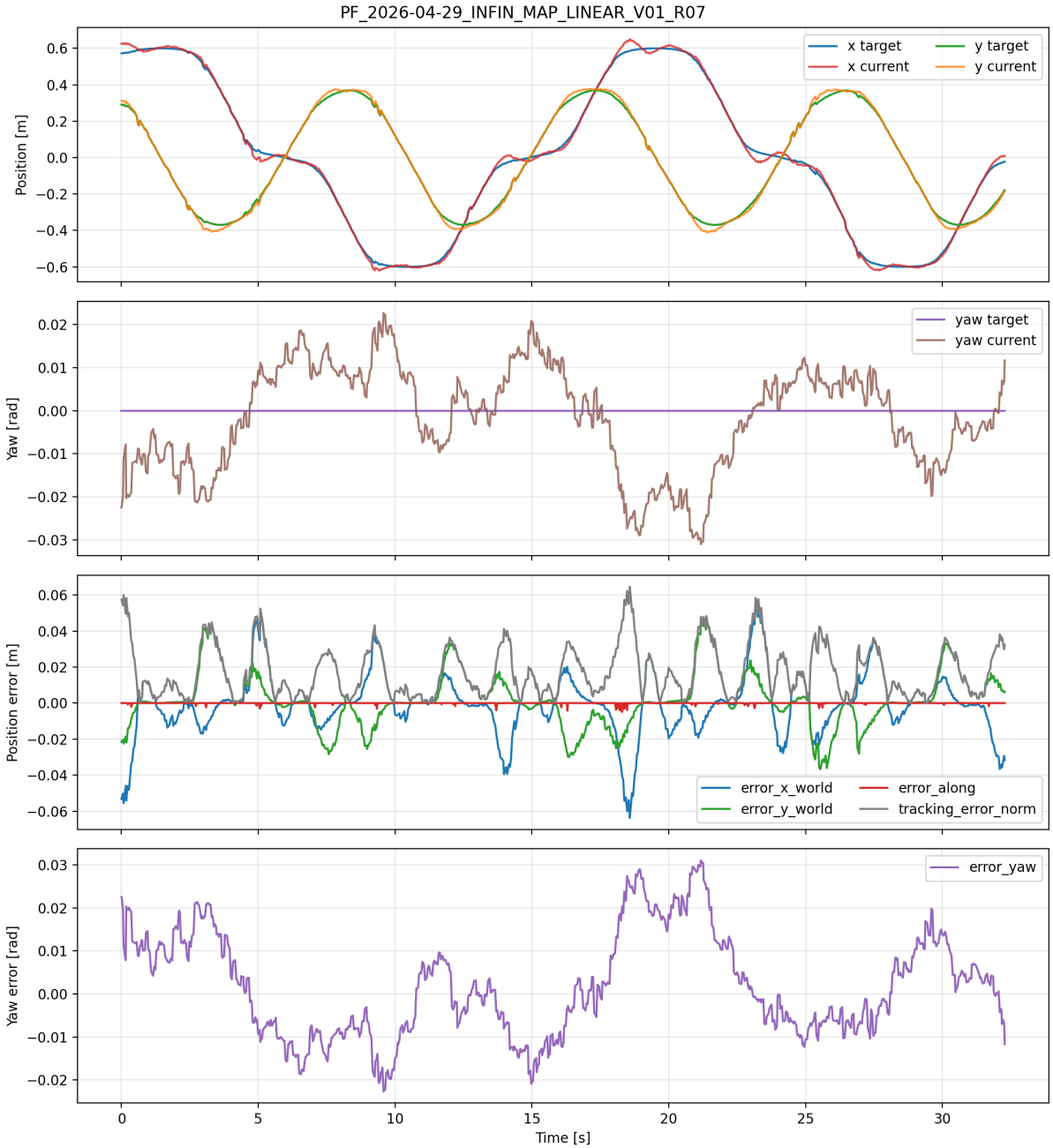


Figure 8.12: Target tracking and error results for the LIN_03 scenario.

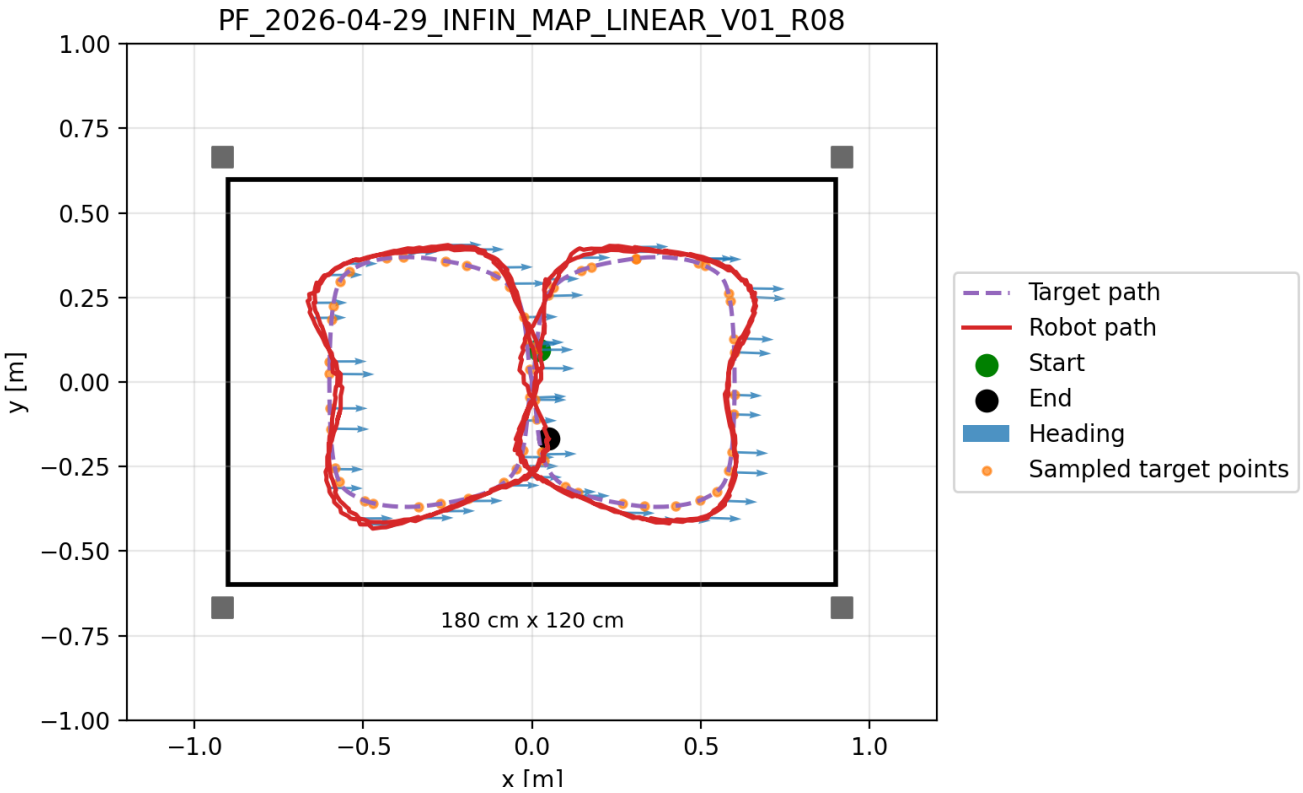


Figure 8.13: XY map for the LIN_04 scenario.

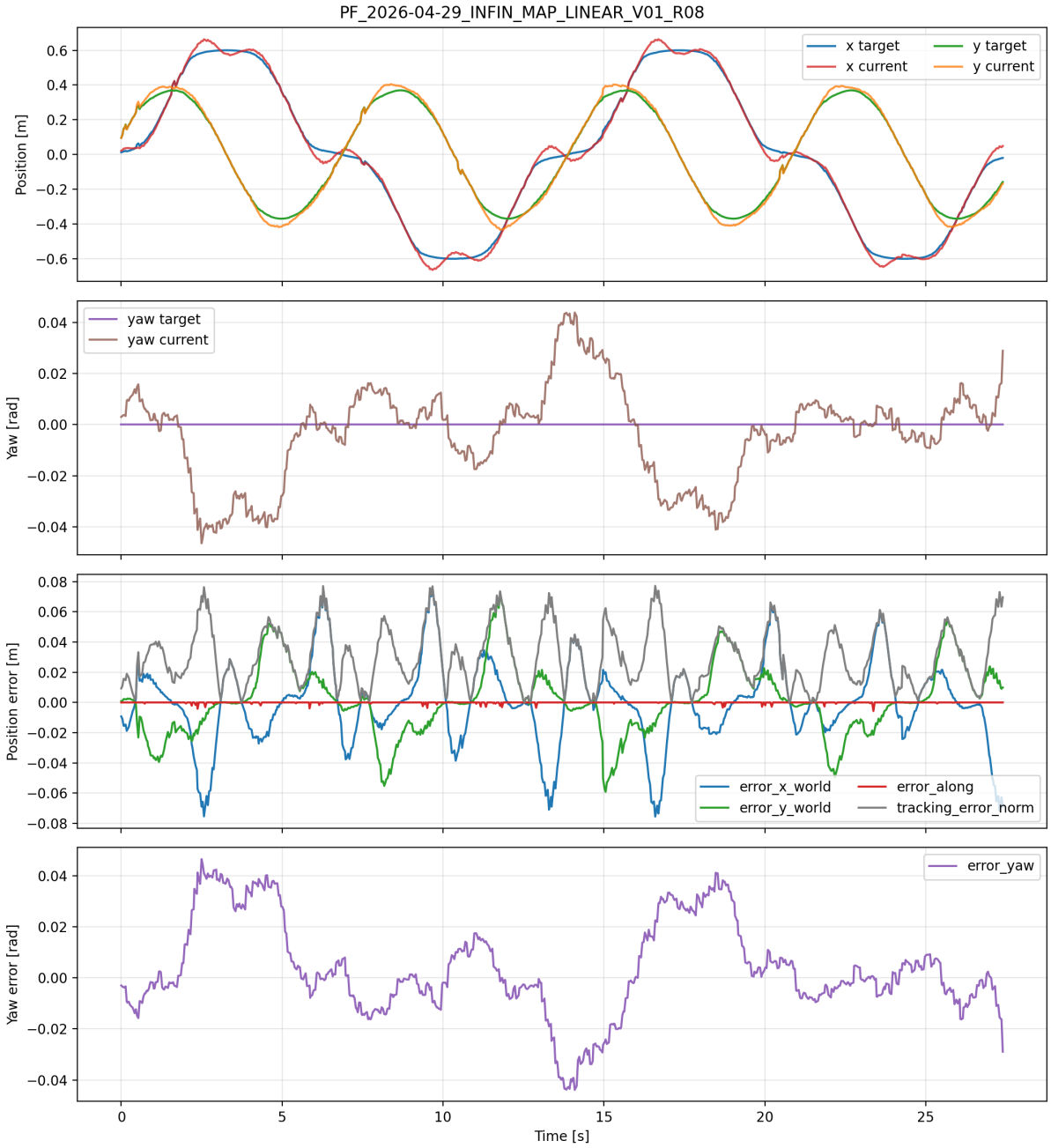


Figure 8.14: Target tracking and error results for the LIN_04 scenario.

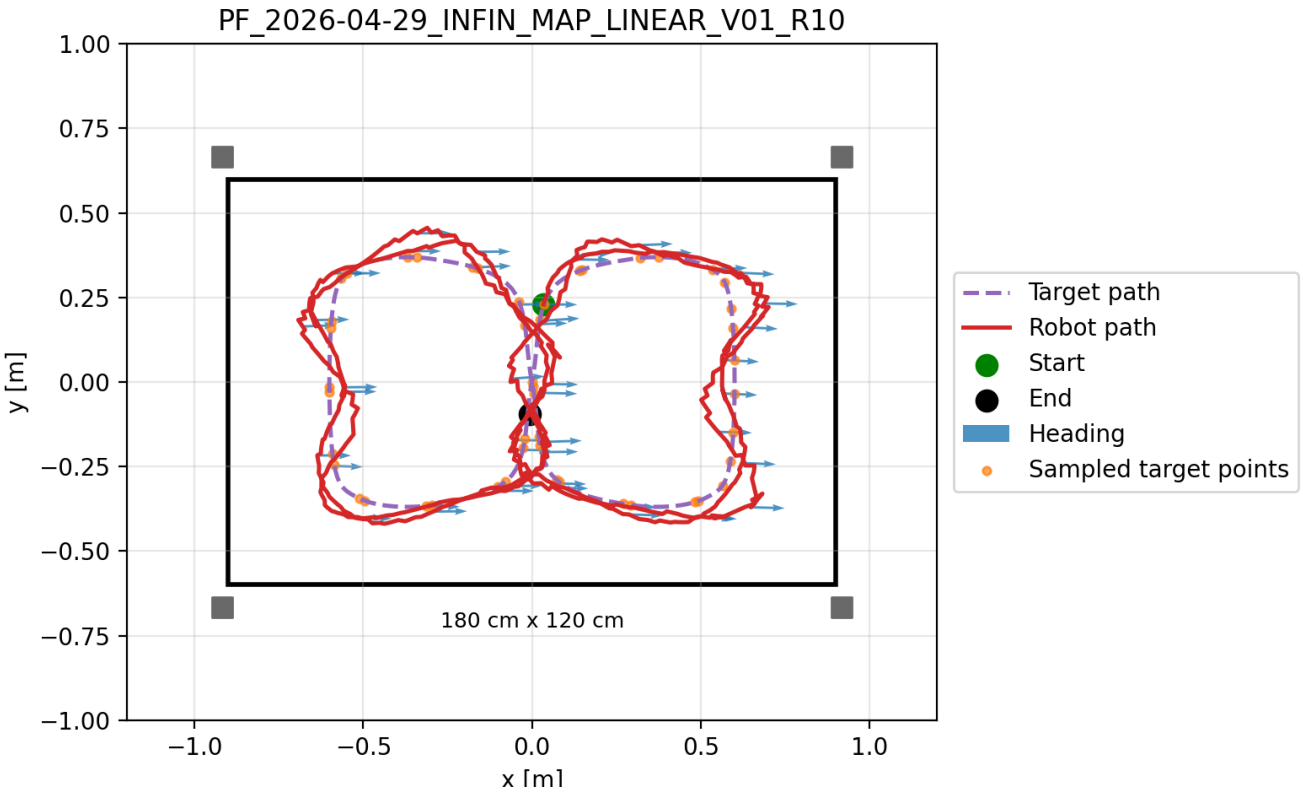


Figure 8.15: XY map for the LIN_05 scenario.

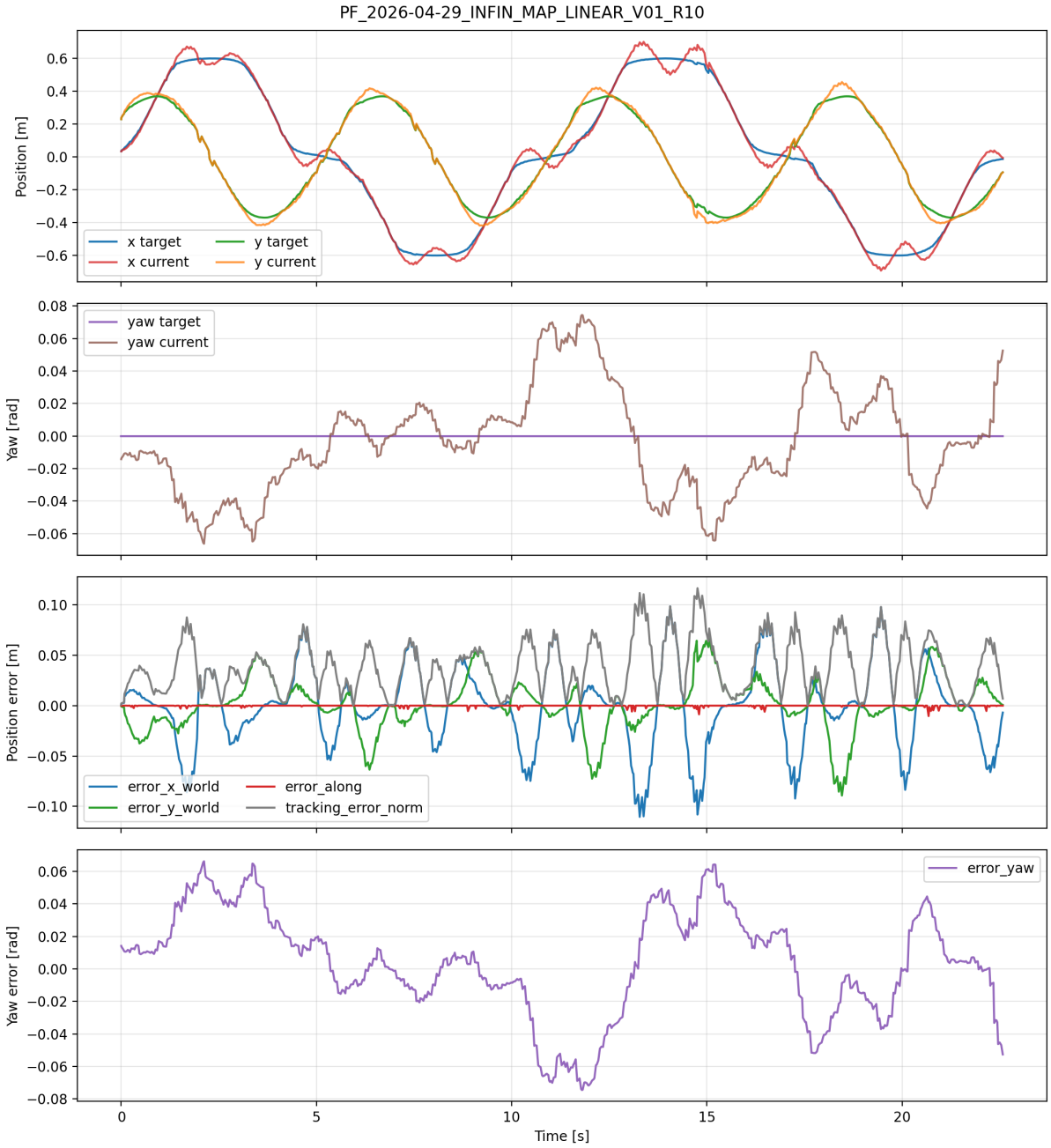


Figure 8.16: Target tracking and error results for the LIN_05 scenario.

Commands to use moving_reference

```
CMD 1
    ros2 launch sdpo_ratf_driver omnicar_bringup.launch.py

CMD 2
    ros2 launch sdpo_motion_control path_follower_map.launch.
       py \
    robot_id:=<ROBOT_ID> \
    control_frame_id:=<ROBOT_ID>/map \
    heading_mode:=<HEADING_MODE> \
    path_file:="<PATH_FILE>" \
    record_bag:=true \
    test_name:=<TEST_NAME> \
    controller_mode:=<CONTROLLER_MODE> \
    bag_directory:="<BAG_DIRECTORY>"
```

Example values for moving_reference command

```

<ROBOT_ID>          = unnamed_robot
<HEADING_MODE>      = fixed
<PATH_FILE>         = /home/c2sr/Omnicar_Control_Ros2/[02]Code
   /omnicar_ws/src/splines/FEUP/FEUP_rounded_rect_eight.csv
<TEST_NAME>         = PF_2026-04-29_INFIN_MAP_LINEAR_V01_R10
<CONTROLLER_MODE>   = linear_segment
<BAG_DIRECTORY>     = /home/c2sr/Omnicar_Control_Ros2/[02]Code
   /omnicar_ws/bags
```

Commands to analyze test records

```
python3 "<ANALYSIS_SCRIPT>" \
    --bag "<BAG_PATH>" \
    --test-id <TEST_ID> \
    --path-name <PATH_NAME> \
    --frame <REFERENCE_FRAME> \
    --controller-name <CONTROLLER_NAME> \
    --controller-version <CONTROLLER_VERSION> \
    --path-file "<PATH_FILE>" \
    --config "<ANALYSIS_CONFIG>" \
    --output-root "<OUTPUT_ROOT>" \
    --debug-topic /<ROBOT_ID>/path_follower_debug \
    --force
```

Example values for test analysis command

```
<ANALYSIS_SCRIPT>     = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_code/scripts/
   analyze_path_following_bag.py

<BAG_PATH>            = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/bags/PF_2026-04-29
   _INFIN_MAP_LINEAR_V01_R10

<TEST_ID>             = PF_2026-04-29
   _INFIN_MAP_LINEAR_V01_R10
<PATH_NAME>           = INFIN
<REFERENCE_FRAME>     = MAP
<CONTROLLER_VERSION>  = V01

<PATH_FILE>           = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/src/splines/FEUP/FEUP_rounded_rect_eight.
   csv

<ANALYSIS_CONFIG>     = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_code/config/analysis_config.yaml

<OUTPUT_ROOT>         = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_results

<ROBOT_ID>            = unnamed_robot
```

8.5 Nonlinear Model Predictive Contouring Controller

This section describes the predictive controller used in the experimental tests. The objective of the controller is not only to reduce the instantaneous Cartesian error with respect to a moving point, but to choose a velocity command by considering the future evolution of the robot over a short horizon. This is the main difference with the previous geometric controllers: the command is selected by balancing path accuracy, path progression, command effort and command smoothness in the same optimization problem.

The formulation is inspired by MPC trajectory-tracking approaches for omnidirectional mobile robots [9, 10, 22]. In those approaches a model of the robot is used to predict the future motion, and an optimal control problem is solved at each control period.
The proposed controller is a nonlinear kinematic Model Predictive Contouring Controller. The prediction model is kinematic because it propagates the robot pose from commanded body-frame velocities, without explicitly modelling wheel dynamics, actuator delays, or motor forces. It is nonlinear because the mapping between body-frame velocities and world-frame motion depends on the robot heading through trigonometric terms, and because the contouring formulation evaluates path-dependent quantities along the curvilinear coordinate. Smooth parametric references, such as B-Spline trajectories, are particularly suitable for MPC because the reference can be evaluated at any point of the prediction horizon and the local path geometry can be used inside the cost function. In this work the reference is treated as a geometric path rather than as a purely time-indexed trajectory. The progression along the path is represented by the curvilinear coordinate γ, so the controller can separate lateral path error from advancement along the curve.

The controller follows the same cascade architecture used in the rest of the ROS2 stack. The high-level node receives the estimated pose in the `map` frame, evaluates the reference path, computes a planar velocity command and publishes a `geometry_msgs/Twist` message on `cmd_vel`. The lower ROS2 driver and the ESP32 firmware then convert this velocity reference into wheel commands and motor voltages.

8.5.1 Geometric Path Representation

The reference path is represented by a sequence of points loaded from a CSV file. The implementation stores the cumulative distance along the polyline, which is used as the path coordinate γ. Given γ, the function `sampleAt` finds the corresponding segment, interpolates the reference position and computes the local tangent:

```
PathSample sampleAt(double gamma) const
{
  ...
  sample.x = a.x + ratio * (b.x - a.x);
  sample.y = a.y + ratio * (b.y - a.y);
  sample.tangent_x = (b.x - a.x) / segment_length;
```

```
  sample.tangent_y = (b.y - a.y) / segment_length;
  ...
}
```

The tangent vector is therefore

$$\mathbf{t}(\gamma) = \begin{bmatrix} t_x(\gamma) & t_y(\gamma) \end{bmatrix}^T, \tag{8.30}$$

while the normal vector is obtained as

$$\mathbf{n}(\gamma) = \begin{bmatrix} -t_y(\gamma) & t_x(\gamma) \end{bmatrix}^T. \tag{8.31}$$

This same geometric representation is used by the Linear Segment controller and is the natural basis for the MPCC formulation, because contouring and lag errors are defined with respect to the tangent and normal directions of the path.

8.5.2 Predictive Model

The predicted state is defined as

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \psi_k & \gamma_k \end{bmatrix}^T, \tag{8.32}$$

where (x_k, y_k, ψ_k) is the predicted robot pose and γ_k is the predicted curvilinear coordinate along the path. The control vector is

$$\mathbf{u}_k = \begin{bmatrix} v_{x,k} & v_{y,k} & \omega_k & \dot{\gamma}_k \end{bmatrix}^T. \tag{8.33}$$

The first three variables are the body-frame velocity command sent to the robot, whereas $\dot{\gamma}_k$ controls the virtual advancement of the reference point along the path.

The prediction model is the discrete-time kinematic model of a holonomic mobile robot:

$$\begin{aligned} x_{k+1} &= x_k + \Delta t \left(\cos\psi_k \, v_{x,k} - \sin\psi_k \, v_{y,k}\right), \\ y_{k+1} &= y_k + \Delta t \left(\sin\psi_k \, v_{x,k} + \cos\psi_k \, v_{y,k}\right), \\ \psi_{k+1} &= \mathrm{wrap}_{[-\pi,\pi]}(\psi_k + \Delta t \, \omega_k), \\ \gamma_{k+1} &= \gamma_k + \Delta t \, \dot{\gamma}_k, \end{aligned} \qquad \psi_{k+1} \in [-\pi, \pi]. \tag{8.34}$$

This model is intentionally simple: it assumes that the omnidirectional platform can track the commanded planar velocity directly. Therefore, wheel dynamics, slip, actuator dead zones and low-level loop delays are not part of the prediction model. In the experimental tests the prediction step is $\Delta t = 0.08$ s. The horizon is usually $N = 12$, corresponding to a prediction window of approximately 0.96 s, while the final test uses $N = 14$, corresponding to approximately 1.12 s.

8.5.3 Contouring and Lag Errors

At each predicted step the path is sampled at γ_k. Let $\mathbf{p}_k = [x_k \; y_k]^T$ be the predicted robot position and $\mathbf{p}_r(\gamma_k)$ the sampled reference point. The position error is projected onto the tangent and normal directions:

$$e_{l,k} = (\mathbf{p}_k - \mathbf{p}_r(\gamma_k))^T \, \mathbf{t}(\gamma_k), \qquad e_{c,k} = (\mathbf{p}_k - \mathbf{p}_r(\gamma_k))^T \, \mathbf{n}(\gamma_k). \tag{8.35}$$

The lag error e_l measures whether the predicted robot position is ahead of or behind the virtual reference point along the path. The contouring error e_c measures the lateral deviation from the path and is the most important quantity for geometric path following. The heading error is

$$e_{\psi,k} = \mathrm{wrap}_{[-\pi,\pi]}(\psi_r(\gamma_k) - \psi_k)\,, \qquad e_{\psi,k} \in [-\pi, \pi]. \tag{8.36}$$

The same decomposition can be seen in the available path-following code used for the Linear Segment controller. The controller computes the tangent, builds the normal vector, and projects the Cartesian error onto both directions:

```
const double normal_x = -command.target.tangent_y;
const double normal_y = command.target.tangent_x;
const double error_along =
  dx_world * command.target.tangent_x + dy_world * command.target.tangent_y;
const double error_lateral = dx_world * normal_x + dy_world * normal_y;
```

For the MPCC, these same geometric quantities are evaluated not only at the current pose, but at every predicted state along the horizon.

8.5.4 Cost Function

The stage cost minimized along the horizon is composed of tracking, progress, effort and smoothness terms:

$$\begin{aligned} J_k =& q_c e_{c,k}^2 + q_l e_{l,k}^2 + q_\psi e_{\psi,k}^2 - q_p \dot{\gamma}_k \\ &+ r_{v_x} v_{x,k}^2 + r_{v_y} v_{y,k}^2 + r_\omega \omega_k^2 + r_{\dot{\gamma}} \dot{\gamma}_k^2 \\ &+ r_{\Delta v_x} \Delta v_{x,k}^2 + r_{\Delta v_y} \Delta v_{y,k}^2 + r_{\Delta\omega} \Delta\omega_k^2 + r_{\Delta\dot{\gamma}} \Delta\dot{\gamma}_k^2. \end{aligned} \tag{8.37}$$

The terms q_c, q_l and q_ψ penalize geometric tracking errors. The negative term $-q_p\dot{\gamma}_k$ rewards advancement along the path. The r terms penalize large velocity commands, while the r_Δ terms penalize command increments. Penalizing increments is important in practice because it reduces abrupt changes in `cmd_vel` without forcing the absolute velocity to be unnecessarily small.

8.5.5 Implementation Structure

The ROS2 path-following node is organized around a common control pipeline: receive the pose, compute a `ControlCommand`, and publish the final `Twist`. In the public repository version, the dispatcher selects between `linear_segment` and `moving_reference_p`:

```
void runFollow(const rclcpp::Time & stamp, double dt)
{
  ControlCommand command = controller_mode_ == "linear_segment" ?
    computeLinearSegmentCommand() :
    computeMovingReferenceCommand(dt);
  publishCommand(stamp, dt, command);
}
```

The MPCC controller follows the same interface: it must return a `ControlCommand` containing the selected body-frame velocities and the predicted path advancement. Conceptually, the dispatcher becomes:

```
if (controller_mode_ == "linear_segment") {
  command = computeLinearSegmentCommand();
} else if (controller_mode_ == "moving_reference_p") {
  command = computeMovingReferenceCommand(dt);
} else if (controller_mode_ == "nonlinear_mpcc") {
  command = computeNonlinearMpccCommand(dt);
}
```

The sampled MPCC implementation can be summarized by the following pseudocode:

```
best_cost = +infinity
best_command = previous_command

for vx in sampled_vx:
  for vy in sampled_vy:
    for omega in sampled_omega:
      for gamma_dot in sampled_gamma_dot:
        x_pred = current_x
        y_pred = current_y
        psi_pred = current_yaw
        gamma_pred = current_gamma
        cost = 0

        for k = 0 ... N-1:
          sample = sampleAt(gamma_pred)
          t = sample.tangent
```

```
    n = [-t.y, t.x]

    e_l = dot([x_pred,y_pred] - sample.position, t)
    e_c = dot([x_pred,y_pred] - sample.position, n)
    e_psi = wrapToPi(sample.yaw - psi_pred)

    cost += stageCost(e_c, e_l, e_psi,
                      vx, vy, omega, gamma_dot,
                      previous_command)

    x_pred += dt * (cos(psi_pred)*vx - sin(psi_pred)*vy)
    y_pred += dt * (sin(psi_pred)*vx + cos(psi_pred)*vy)
    psi_pred = wrapToPi(psi_pred + dt * omega)
    gamma_pred += dt * gamma_dot

  cost += terminalCost(e_c, e_l)

  if cost < best_cost:
    best_cost = cost
    best_command = [vx, vy, omega, gamma_dot]
```

After the best candidate is selected, it is published through the same output stage used by the other controllers. The current node adds feedforward and feedback terms, saturates the velocities and publishes `cmd_vel`:

```
cmd.linear.x = clamp(cmd_vx_raw, -max_linear_x_, max_linear_x_);
cmd.linear.y = clamp(cmd_vy_raw, -max_linear_y_, max_linear_y_);
cmd.angular.z = clamp(cmd_w_raw, -max_angular_z_, max_angular_z_);
pub_cmd_vel_->publish(cmd);
```

This implementation choice avoids the need for an external nonlinear optimizer and is deterministic, but the result depends on the sampling resolution. A coarse grid is computationally cheap but may produce quantized commands, whereas a dense grid improves the approximation at the cost of a higher computation time.

8.5.6 Limitations and Possible Improvements

The main limitations of this first predictive formulation are:

- the prediction model neglects wheel dynamics, actuator delay, wheel slip, friction and the transient response of the low-level velocity loop;
- the command is selected on a finite grid, so the solution is only an approximation of the continuous optimum;

- the same candidate command is kept constant during the whole rollout, which reduces computation time but limits the richness of the predicted control sequence;
- the controller depends directly on the pose estimate, so localization drift appears as contouring and lag error;
- obstacle constraints are not included, therefore the controller is a path-following controller and not a complete local planner.

Future work should focus on a continuous nonlinear solver, optimization of a full command sequence instead of a constant command, curvature-dependent velocity limits, adaptive progress weights, explicit actuator-delay compensation and uncertainty-aware localization handling. The current implementation is therefore best interpreted as an experimental predictive baseline: it shows how tracking, progress and smoothness can be traded off explicitly, but it still requires further tuning before it can be considered superior to a simpler and well-tuned geometric controller in all metrics.

8.5.7 Test Parameters and Results

Test legend:

- **MPC-01**: `PF_2026-05-11_INFIN_MAP_MPCC_V01_R01`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.17 and 8.18.
- **MPC-02**: `PF_2026-05-11_INFIN_MAP_MPCC_V01_R02`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.19 and 8.20.
- **MPC-03**: `PF_2026-05-11_INFIN_MAP_MPCC_V01_R03`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.21 and 8.22.
- **MPC-04**: `PF_2026-05-11_INFIN_MAP_MPCC_V01_R04`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.23 and 8.24.
- **MPC-05**: `PF_2026-05-11_INFIN_MAP_MPCC_V01_R05`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.25 and 8.26.
- **MPC-06**: `PF_2026-05-11_INFIN_MAP_MPCC_V01_R06`.
  The corresponding XY trajectory and tracking results are shown in Figures 8.27 and 8.28.

The following parameters are common to all tests: path `INFIN`, reference frame `MAP`, controller mode `nonlinear_mpcc`, fixed heading mode.

Table 8.5: Main characteristic parameters of the selected MPCC tests.

Parameter	**MPC-01**	**MPC-02**	**MPC-03**	**MPC-04**	**MPC-05**	**MPC-06**
Main configuration	Baseline	Slow/stable	Aggressive	Very aggressive	Aggressive curves	Extreme
$v_{x,\max}$ [m/s]	0.25	0.18	0.35	0.50	0.60	0.70
$v_{y,\max}$ [m/s]	0.25	0.18	0.35	0.50	0.60	0.70
$\omega_{\max}$ [rad/s]	0.45	0.35	0.60	0.60	0.60	0.80
$\dot{\gamma}_{\max}$ [m/s]	0.30	0.20	0.40	0.40	0.40	0.70
$a_{x,\max}$ [m/s^2]	0.60	0.40	0.90	0.90	0.90	1.80
$a_{y,\max}$ [m/s^2]	0.60	0.40	0.90	0.90	0.90	1.80
$\dot{\omega}_{\max}$ [rad/s^2]	1.20	0.80	1.60	1.60	1.60	2.50
$\dot{\gamma}_{\max}$ [m/s^2]	0.80	0.50	1.20	1.20	1.20	2.50
q_p	0.40	0.25	0.60	0.60	0.90	1.60
q_c	20	20	20	20	20	18
q_l	2	2	2	2	2	1
q_ψ	1.0	1.0	1.0	1.0	1.0	0.6
r_{v_x}, r_{v_y}	0.15	0.15	0.15	0.15	0.15	0.07
r_ω	0.10	0.10	0.10	0.10	0.10	0.05
$r_{\dot{\gamma}}$	0.05	0.05	0.05	0.05	0.05	0.02
$r_{\Delta v_x}, r_{\Delta v_y}$	0.50	0.50	0.50	0.50	0.70	0.25
$r_{\Delta\omega}$	0.20	0.20	0.20	0.20	0.20	0.10
$r_{\Delta\dot{\gamma}}$	0.10	0.10	0.10	0.10	0.10	0.30
N	12	12	12	12	12	14
$\dot{\gamma}$ samples	5	5	5	5	5	7

Table 8.6: Compact notation used for vector parameters.

Code	**Values**
A	$(r_{v_x}, r_{v_y}, r_\omega, r_{\dot{\gamma}}) = (0.15, 0.15, 0.10, 0.05)$
B	$(r_{v_x}, r_{v_y}, r_\omega, r_{\dot{\gamma}}) = (0.07, 0.07, 0.05, 0.02)$
C	$(r_{\Delta v_x}, r_{\Delta v_y}, r_{\Delta\omega}, r_{\Delta\dot{\gamma}}) = (0.50, 0.50, 0.20, 0.10)$
D	$(r_{\Delta v_x}, r_{\Delta v_y}, r_{\Delta\omega}, r_{\Delta\dot{\gamma}}) = (0.25, 0.25, 0.10, 0.30)$
S1	$(v_x, v_y, \omega, \dot{\gamma})_{\text{samples}} = (5, 5, 5, 5)$
S2	$(v_x, v_y, \omega, \dot{\gamma})_{\text{samples}} = (5, 5, 5, 7)$

Table 8.7: Nonlinear MPCC controller: experimental results of the selected tests.

Metric	**MPC-01**	**MPC-02**	**MPC-03**	**MPC-04**	**MPC-05**	**MPC-06**
Duration [s]	54.46	32.39	50.40	29.53	35.39	21.53
Samples	818	487	757	444	532	324
Mean sample time [s]	0.0667	0.0667	0.0667	0.0667	0.0667	0.0667
Real path length [m]	9.5826	7.1290	7.0083	6.8210	9.4391	6.2836
Final γ [m]	2.1353	4.5252	4.5341	2.2472	2.2663	4.4428
Path progress ratio r_γ	0.4688	0.9935	0.9955	0.4934	0.4976	0.9754
RMSE_x [m]	0.0202	0.0272	0.0153	0.0291	0.0368	0.0677
RMSE_y [m]	0.0317	0.0440	0.0228	0.0487	0.0583	0.0934
$\mathrm{RMSE}_{\|e\|}$ [m]	0.0376	0.0517	0.0274	0.0568	0.0689	0.1154
Mean $\|e\|$ [m]	0.0333	0.0463	0.0253	0.0479	0.0614	0.1085
Max $\|e\|$ [m]	0.0851	0.0970	0.0693	0.1323	0.1467	0.2102
RMSE_ψ [rad]	0.0067	0.0092	0.0046	0.0108	0.0139	0.0294
$\mathrm{RMSE}_{e_\parallel}$ [m]	0.0366	0.0503	0.0268	0.0552	0.0668	0.1104
$\mathrm{MAE}_{e_\parallel}$ [m]	0.0321	0.0445	0.0244	0.0458	0.0589	0.1038
$\max \lvert e_\parallel \rvert$ [m]	0.0851	0.0964	0.0693	0.1318	0.1455	0.2032
$\mathrm{IAE}_{e_\parallel}$	1.7487	1.4376	1.2276	1.3506	2.0827	2.2351
$\max \lvert e_\psi \rvert$ [rad]	0.0413	0.0538	0.0151	0.0376	0.0473	0.0710
Saturation ratio v_x	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Saturation ratio v_y	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
Saturation ratio ω	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

The six MPCC tests show a clear progression in tuning. **MPC-01** uses the most conservative velocity limits and progress weight. It remains stable and has no command saturation, but it covers less than half of the path ($r_\gamma = 0.4688$). **MPC-02** and **MPC-03** increase the path progression weight and the allowed velocity limits. Both runs almost complete the path, with $r_\gamma = 0.9935$ and $r_\gamma = 0.9955$, respectively. Among all MPCC tests, **MPC-03** provides the best compromise in terms of tracking accuracy, with $\mathrm{RMSE}_x = 0.0153\,\mathrm{m}$, $\mathrm{RMSE}_y = 0.0228\,\mathrm{m}$, and the lowest yaw error $\mathrm{RMSE}_\psi = 0.0046\,\mathrm{rad}$.

MPC-04 and **MPC-05** do not improve the tracking behaviour. Even though the actuator limits are increased, the path progression remains close to half of the path and the position errors increase. This suggests that simply raising the available command range is not sufficient: the cost weights and the prediction behaviour must remain balanced. **MPC-06** is the most aggressive configuration. It uses a longer horizon, larger velocity limits, lower effort penalties, and a much higher progress reward. The robot again covers almost the entire path ($r_\gamma = 0.9754$), but the tracking error increases substantially, reaching $\mathrm{RMSE}_{\|e\|} = 0.1154$ m. This run is useful because it identifies the practical limit of this first MPCC tuning: encouraging too much progression produces faster motion but degrades the geometric quality of the trajectory.

8.5.8 Experimental Figures

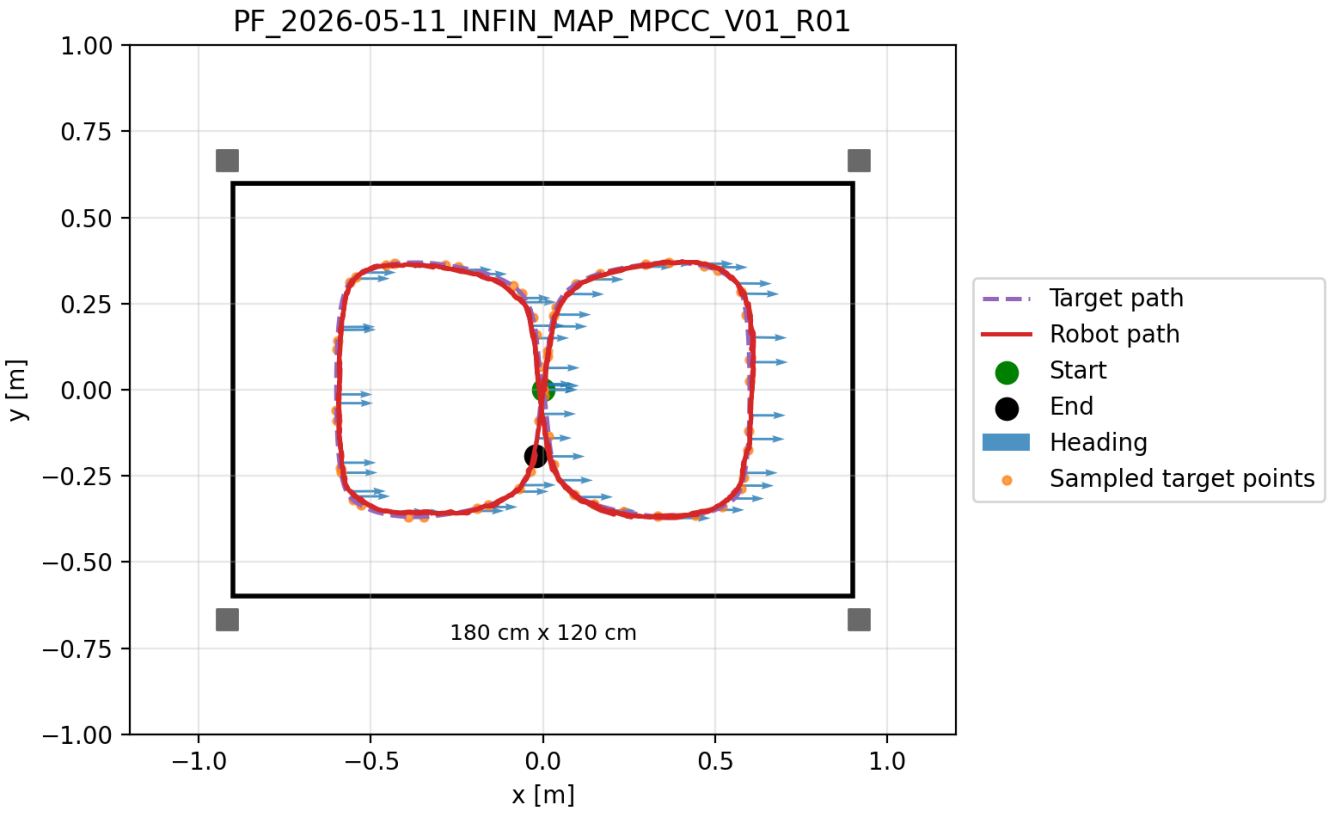


Figure 8.17: XY map for the MPC_01 scenario.

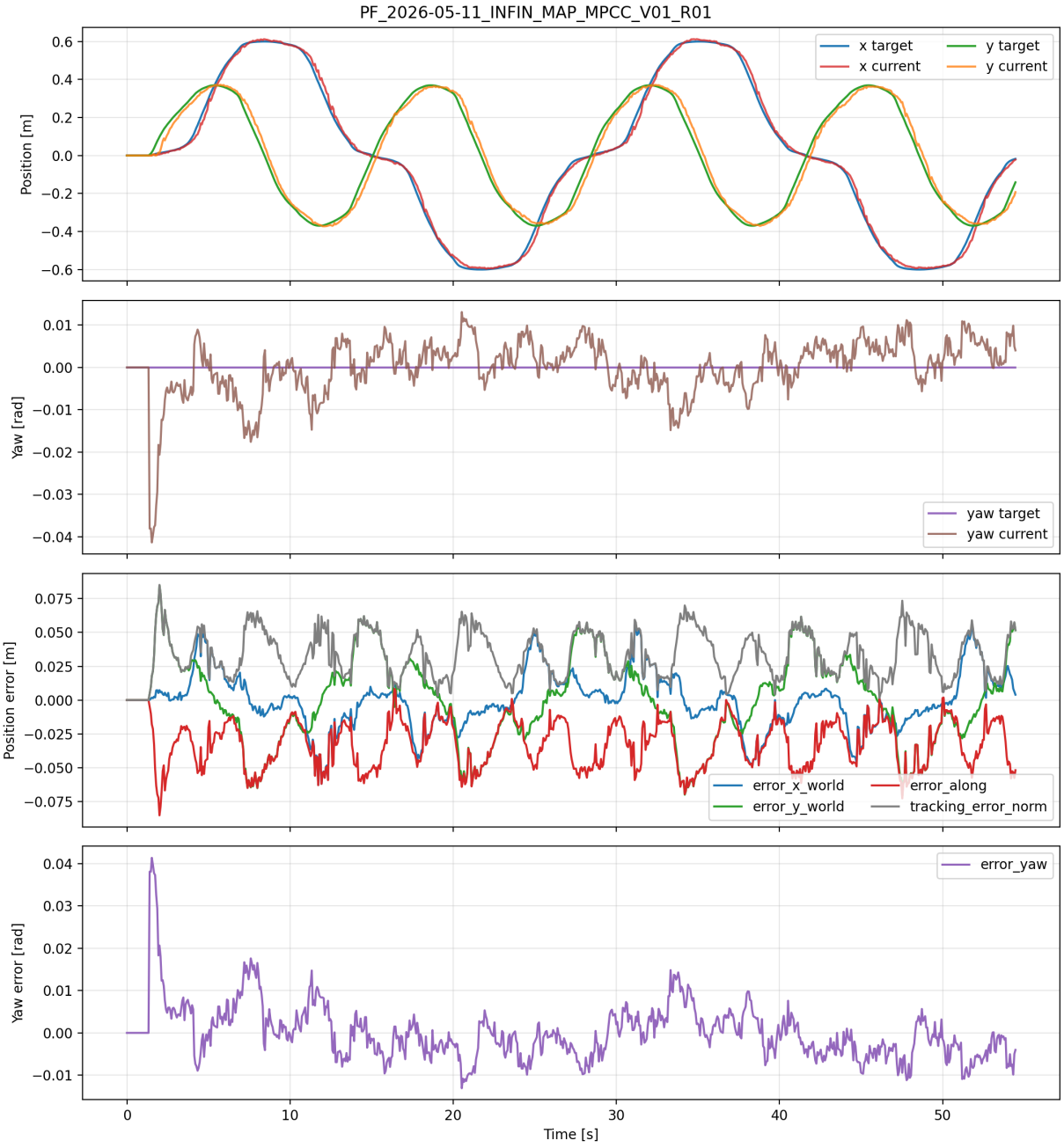


Figure 8.18: Target tracking and error results for the MPC_01 scenario.

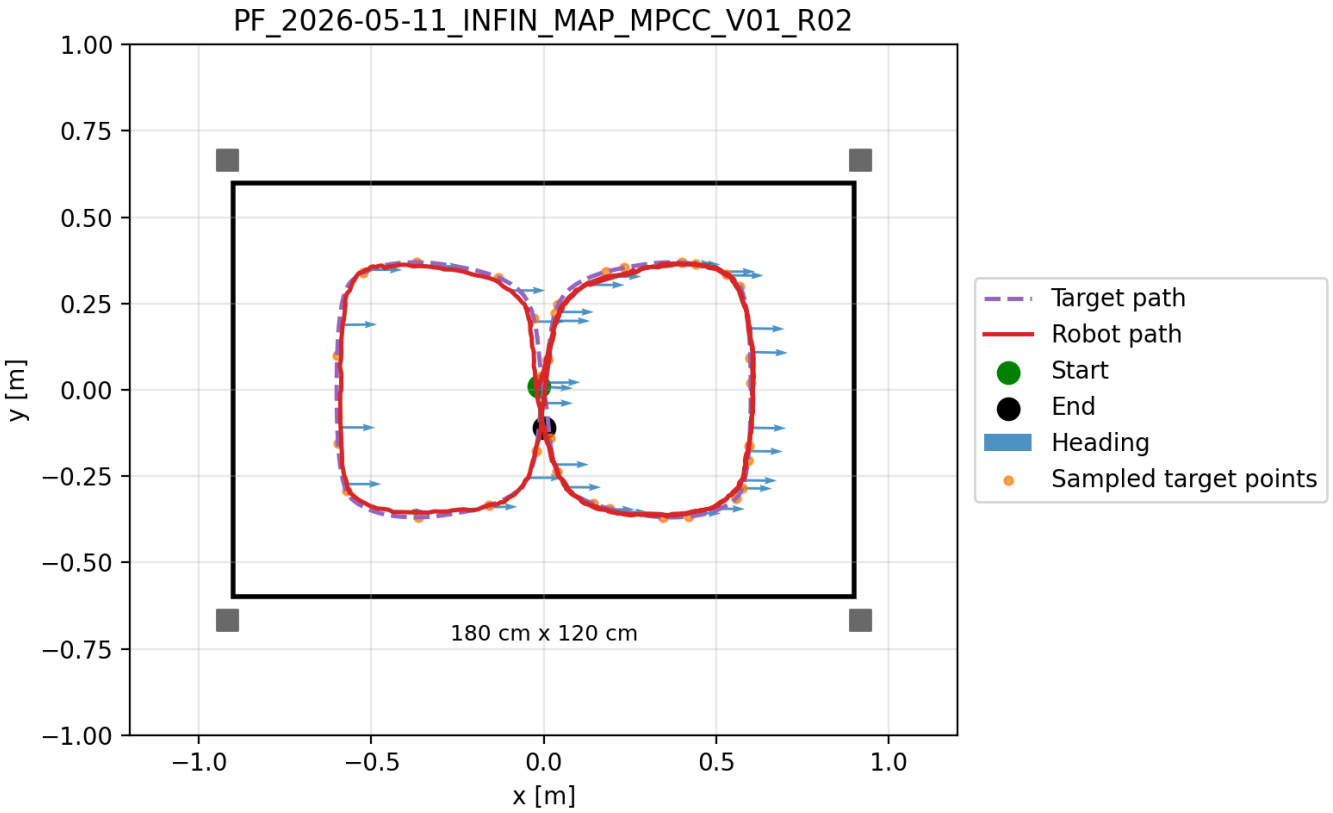


Figure 8.19: XY map for the MPC_02 scenario.

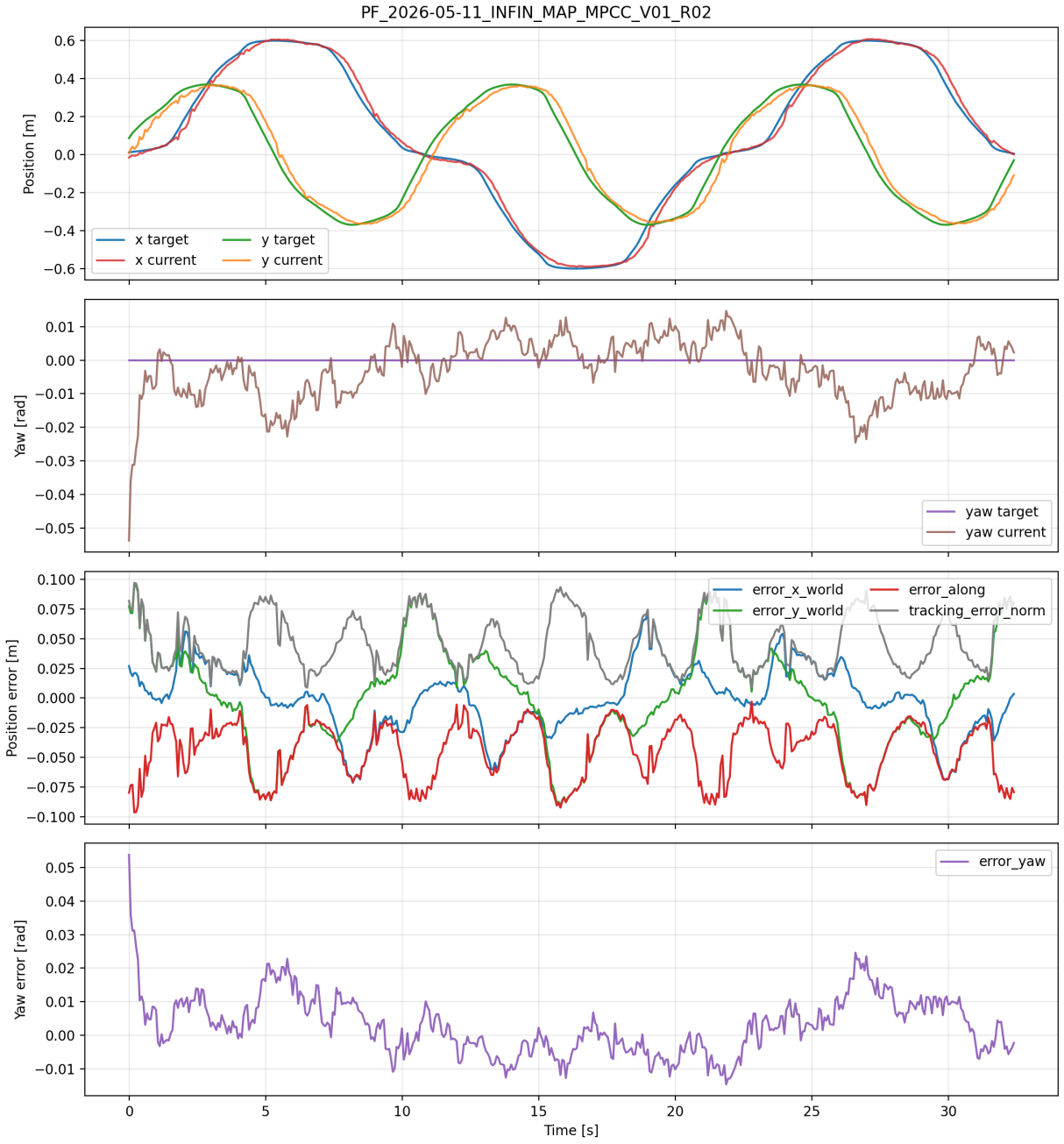


Figure 8.20: Target tracking and error results for the MPC_02 scenario.

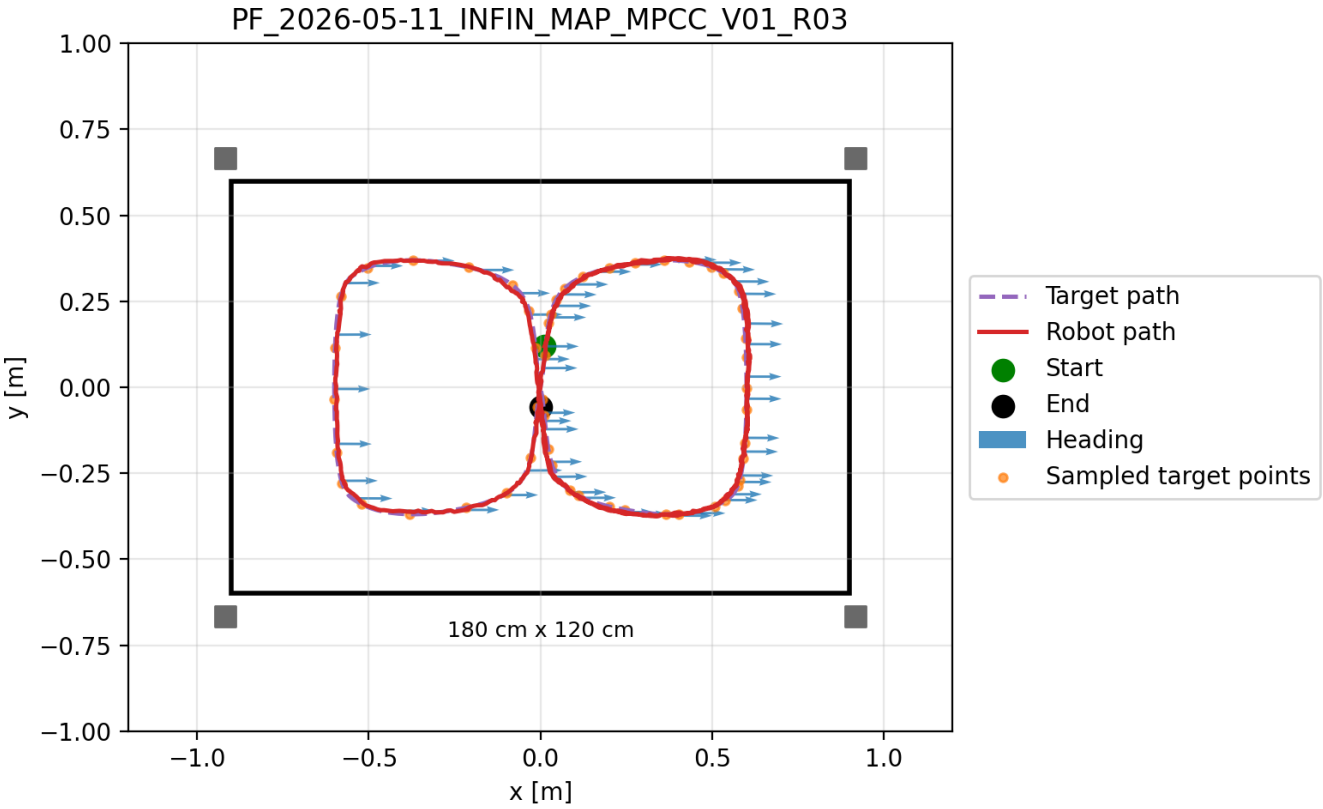


Figure 8.21: XY map for the MPC_03 scenario.

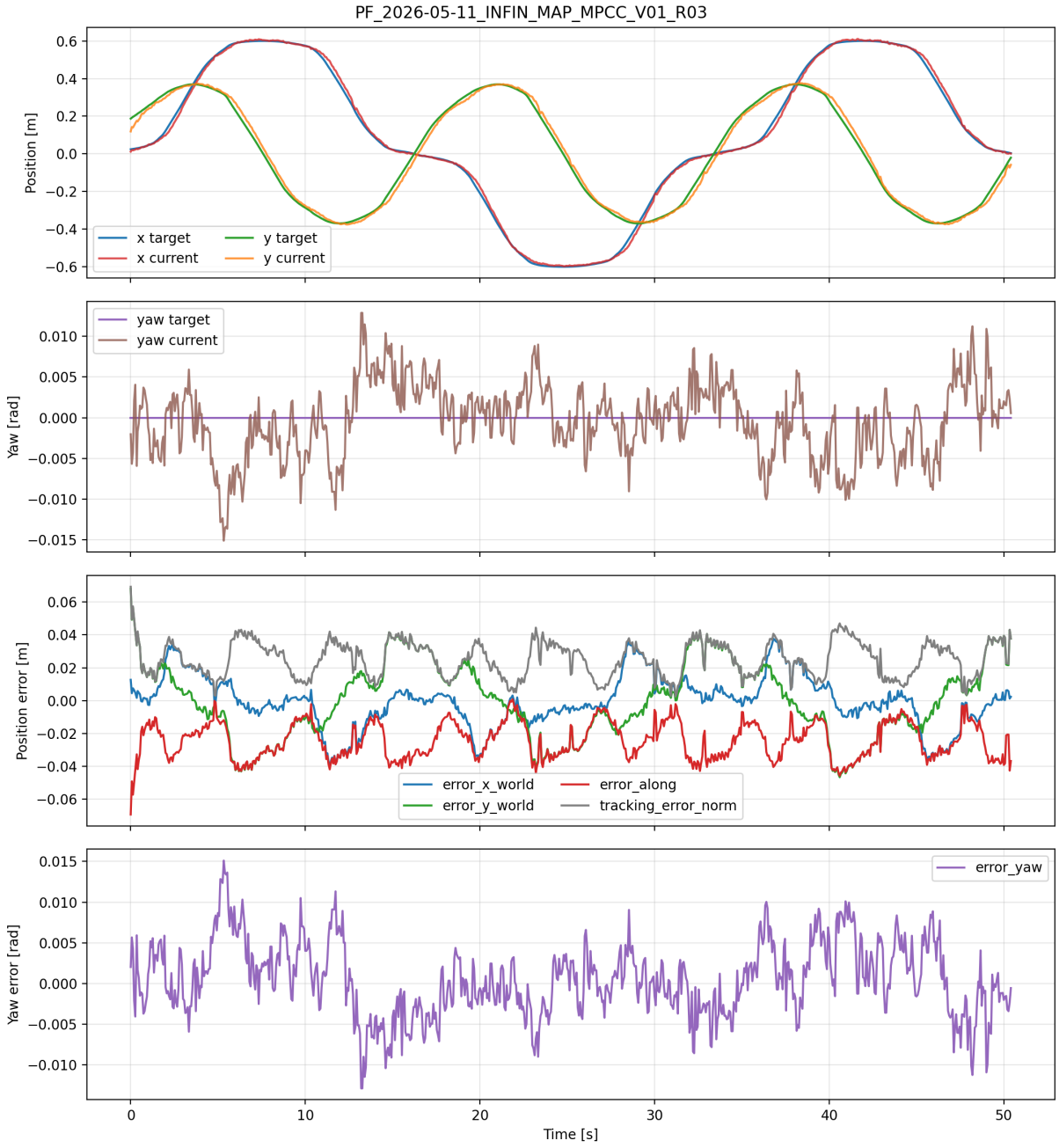


Figure 8.22: Target tracking and error results for the MPC_03 scenario.

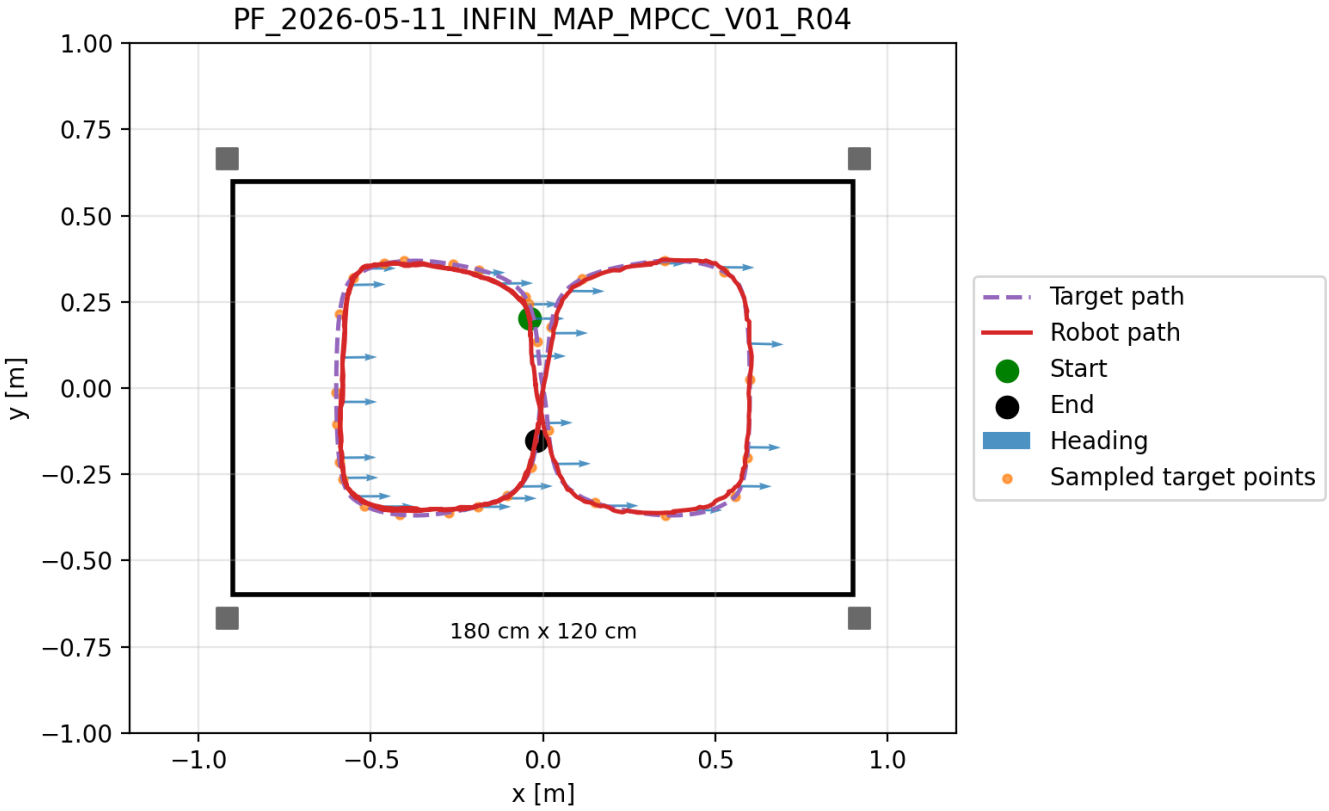


Figure 8.23: XY map for the MPC_04 scenario.

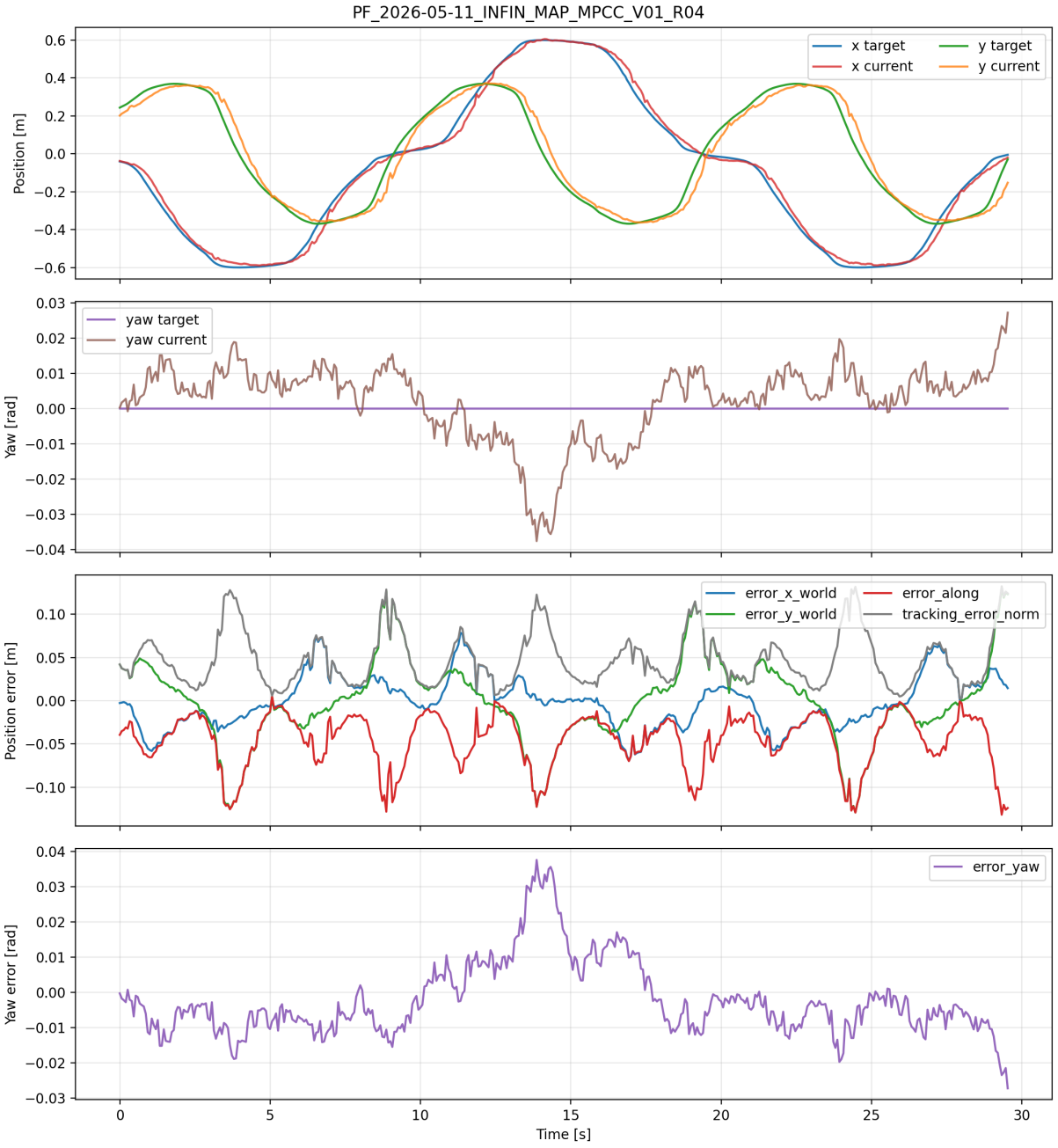


Figure 8.24: Target tracking and error results for the MPC_04 scenario.

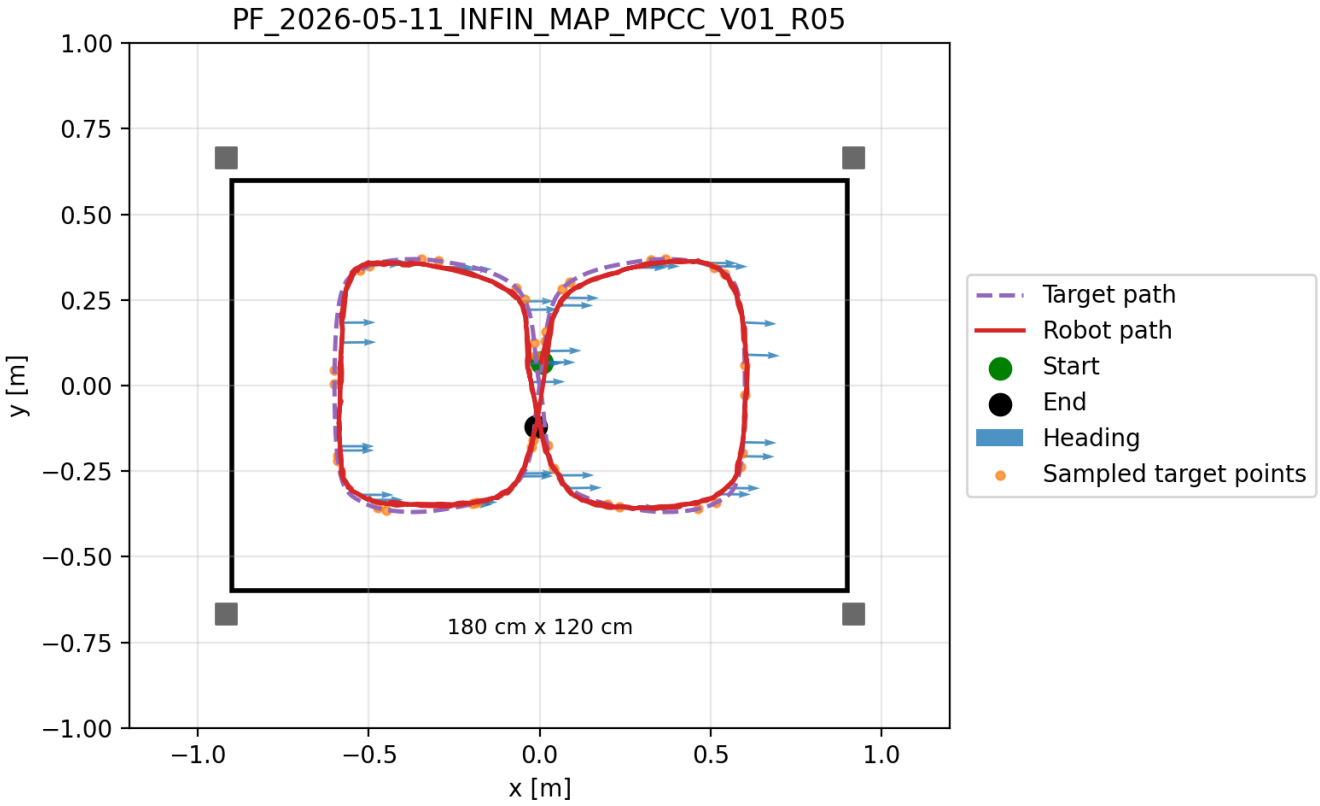


Figure 8.25: XY map for the MPC_05 scenario.

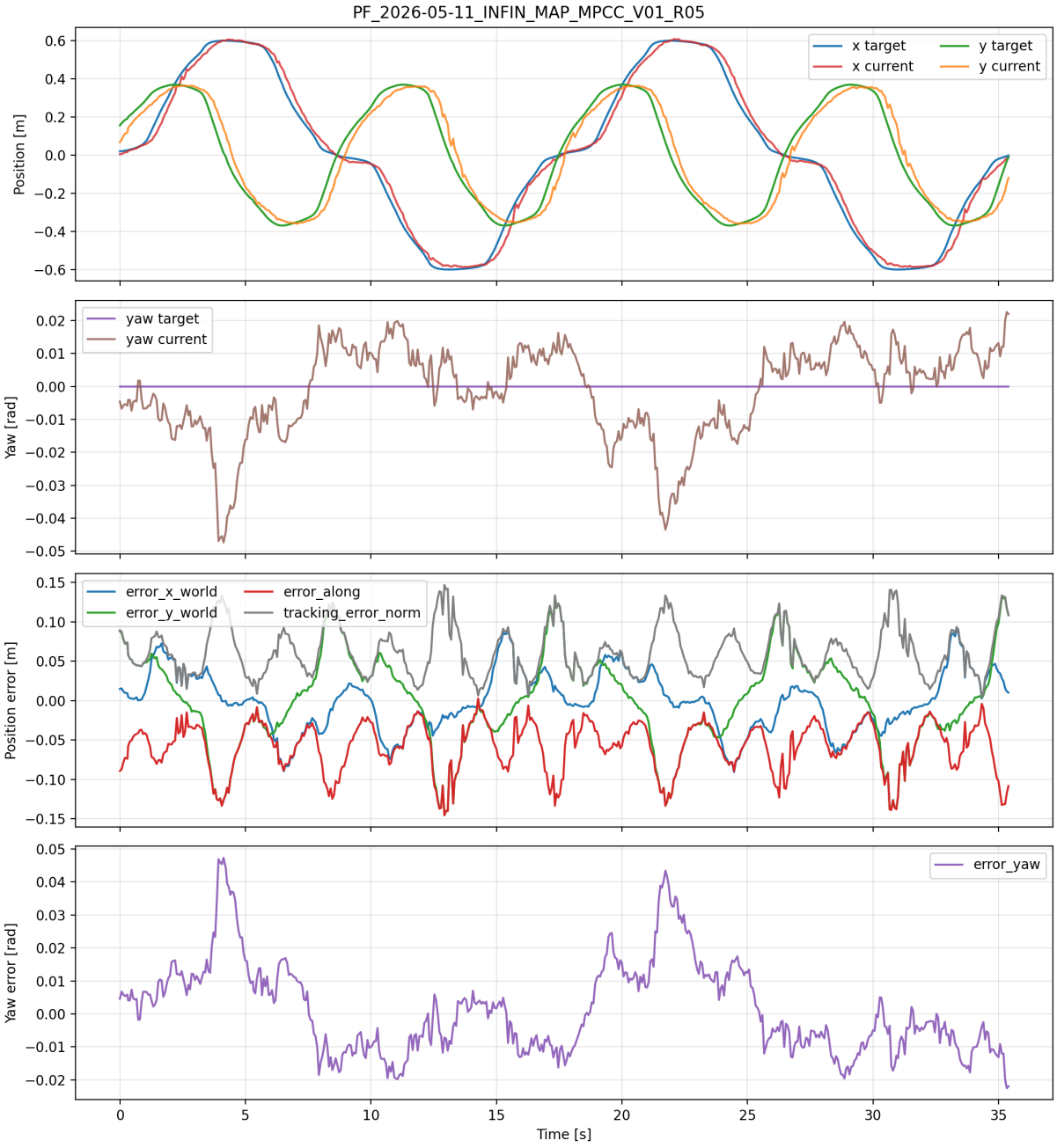


Figure 8.26: Target tracking and error results for the MPC_05 scenario.

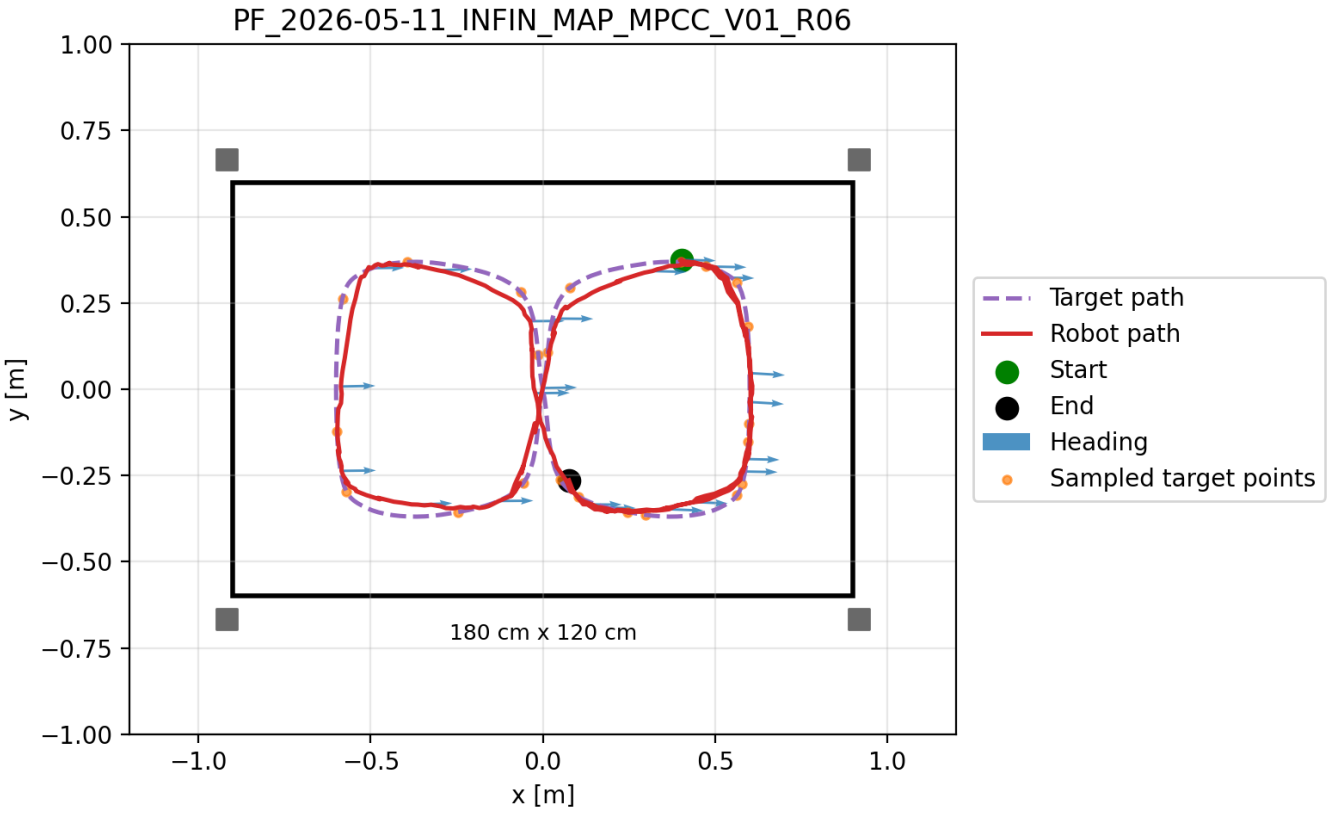


Figure 8.27: XY map for the MPC_06 scenario.

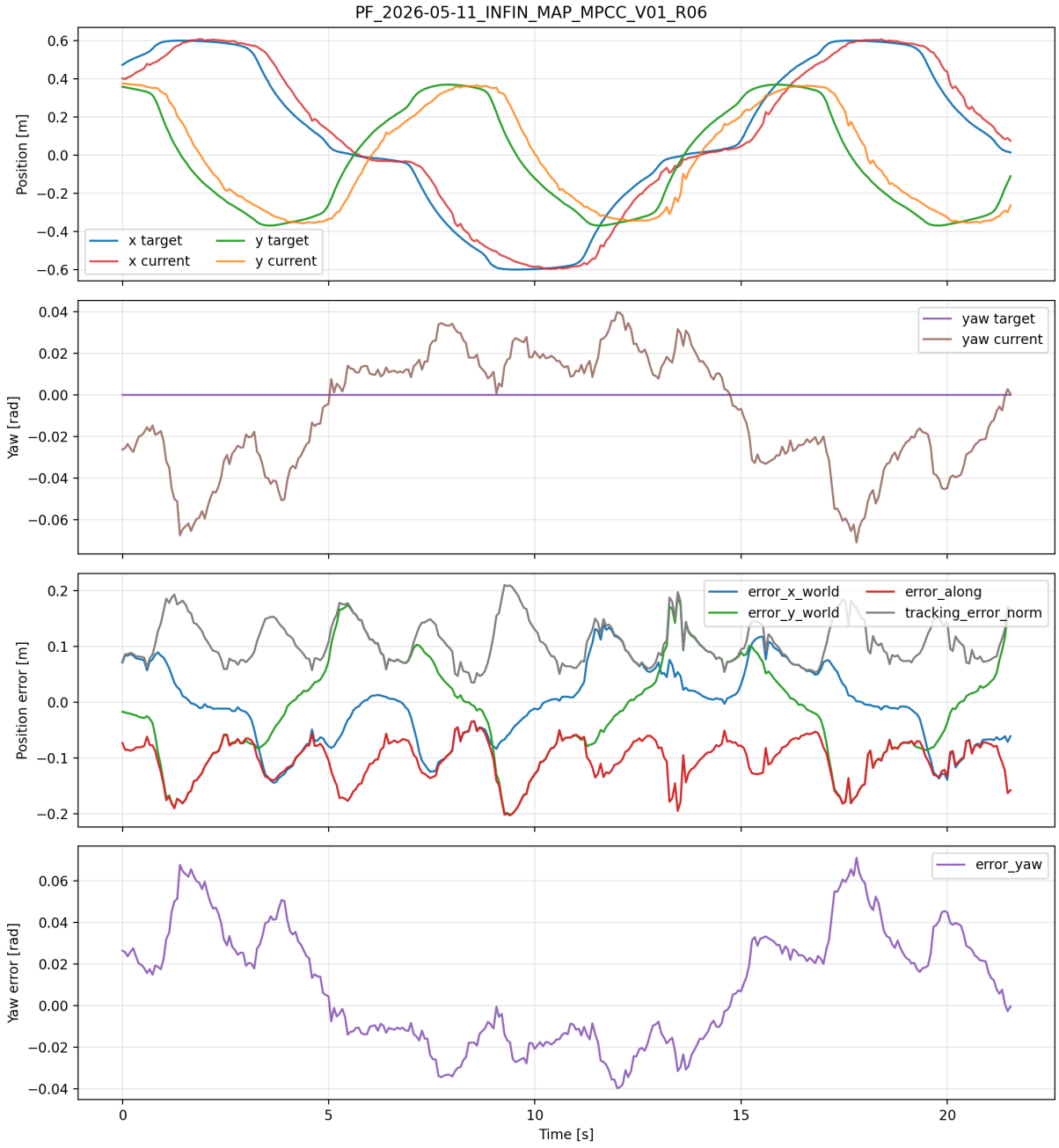


Figure 8.28: Target tracking and error results for the MPC_06 scenario.

Commands to use nonlinear MPCC controller

```
CMD 1 -- Launch the robot bringup
    ros2 launch sdpo_ratf_driver omnicar_bringup.launch.py

CMD 2 -- Launch the nonlinear MPCC path follower
    ros2 launch sdpo_motion_control nonlinear_mpcc_map.launch
        .py \
    test_name:=<TEST_NAME> \
    path_file:="<PATH_FILE>" \
    heading_mode:=<HEADING_MODE> \
    record_bag:=true
```

Commands to analyze nonlinear MPCC test records

```
python3 "<ANALYSIS_SCRIPT>" \
    --bag "<BAG_PATH>" \
    --test-id <TEST_ID> \
    --path-name <PATH_NAME> \
    --frame <REFERENCE_FRAME> \
    --controller-name <CONTROLLER_NAME> \
    --controller-version <CONTROLLER_VERSION> \
    --path-file "<PATH_FILE>" \
    --config "<ANALYSIS_CONFIG>" \
    --output-root "<OUTPUT_ROOT>" \
    --debug-topic /<ROBOT_ID>/path_follower_debug
```

Example values for nonlinear MPCC commands

```
<TEST_NAME>           = PF_2026-05-11_INFIN_MAP_MPCC_V01_R06
<TEST_ID>             = PF_2026-05-11_INFIN_MAP_MPCC_V01_R06

<PATH_NAME>           = INFIN
<REFERENCE_FRAME>     = MAP
<CONTROLLER_NAME>     = MPCC
<CONTROLLER_VERSION>  = V01
<HEADING_MODE>        = fixed
<ROBOT_ID>            = unnamed_robot

<PATH_FILE>           = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/src/splines/FEUP/FEUP_rounded_rect_eight.
   csv

<BAG_PATH>            = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/bags/PF_2026-05-11_INFIN_MAP_MPCC_V01_R06

<ANALYSIS_SCRIPT>     = Test/test_code/scripts/
   analyze_path_following_bag.py

<ANALYSIS_CONFIG>     = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_code/config/analysis_config_mpcc
   .yaml

<OUTPUT_ROOT>         = /home/c2sr/Omnicar_Control_Ros2/[02]
   Code/omnicar_ws/Test/test_results
```

8.5.9 Comparison Between Linear Segment and MPCC

The previous tables evaluate each experiment over its full recorded duration. This is useful to understand whether a controller is able to complete the path, but it is not always the fairest way to compare tracking quality. In fact, two tests may cover different portions of the infinity-shaped trajectory, and the tracking errors computed over the full run may therefore refer to different path segments. For this reason, a final comparison was performed by evaluating each pair of tests only over the common portion of the path covered by both runs.

The comparison was computed using the curvilinear coordinate γ. For a set of selected runs, the common interval is defined as:

$$\gamma^c_{\min} = \max_j \gamma_{\min,j}, \qquad \gamma^c_{\max} = \min_j \gamma_{\max,j} \tag{8.38}$$

Only the samples satisfying

$$\gamma \in [\gamma^c_{\min}, \gamma^c_{\max}] \tag{8.39}$$

are used in the final comparison. This makes the evaluation independent of the different total path progress reached by each controller.

The error is not computed with respect to the internal target point published by each controller. Instead, each measured robot position is compared with the same geometric reference path, interpolated at the recorded value of γ. Let $\mathbf{p} = [x\ y]^T$ be the measured robot position and $\mathbf{p}_r(\gamma) = [x_r(\gamma)\ y_r(\gamma)]^T$ the reference point on the path. The position error is:

$$\mathbf{e} = \mathbf{p} - \mathbf{p}_r(\gamma) \tag{8.40}$$

Using the local tangent and normal vectors:

$$\mathbf{t}(\gamma) = \begin{bmatrix} \cos\psi_r(\gamma) \\ \sin\psi_r(\gamma) \end{bmatrix}, \qquad \mathbf{n}(\gamma) = \begin{bmatrix} -\sin\psi_r(\gamma) \\ \cos\psi_r(\gamma) \end{bmatrix}, \tag{8.41}$$

the error is decomposed into lag and contouring components:

$$e_l = \mathbf{e}^T \mathbf{t}(\gamma), \qquad e_c = \mathbf{e}^T \mathbf{n}(\gamma) \tag{8.42}$$

The contouring error e_c measures the lateral distance from the path and is the most relevant quantity for geometric path-following accuracy. The lag error e_l measures whether the robot is ahead of or behind the reference point along the path direction. For this final comparison, the reported metrics are:

$$\mathrm{RMSE}_{e_c} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_{c,i}^2}, \qquad \mathrm{MAE}_{e_c} = \frac{1}{N}\sum_{i=1}^{N} |e_{c,i}|, \tag{8.43}$$

$$\max|e_c| = \max_{i=1,\dots,N} |e_{c,i}|, \qquad \mathrm{RMSE}_{e_l} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} e_{l,i}^2}, \qquad \mathrm{MAE}_{e_l} = \frac{1}{N}\sum_{i=1}^{N} |e_{l,i}|. \tag{8.44}$$

The path progress ratio and the velocity saturation ratios are not included in this final comparison, because the objective here is to isolate the tracking quality on the same path segment.

Table 8.8: Common-segment comparison between selected Linear Segment and MPCC tests.

Run	**Common segment [m]**	**Duration [s]**	RMSE_{e_c} [m]	MAE_{e_c} [m]	$\max\lvert e_c\rvert$ [m]	RMSE_{e_l} [m]	MAE_{e_l} [m]
LIN-04	4.537	27.39	0.0354	0.0300	0.0772	0.0008	0.0005
LIN-05	4.537	22.58	0.0469	0.0392	0.1166	0.0011	0.0006
MPC-05	4.534	35.39	0.0170	0.0144	0.0357	0.0666	0.0587
MPC-06	4.534	21.53	0.0330	0.0257	0.0705	0.1105	0.1039
LIN-05	4.534	22.58	0.0469	0.0391	0.1166	0.0011	0.0006
MPC-06	4.534	21.53	0.0330	0.0257	0.0705	0.1105	0.1039

The first comparison considers two Linear Segment runs. The corresponding overlapped XY trajectories are shown in Figure 8.29, while the contouring and lag error metrics are reported in Figure 8.30. The slower configuration LIN-05 achieves the best contouring performance. Increasing the velocity in LIN-06 reduces the duration from 50.14 s to 38.86 s, but the maximum contouring error increases from 0.0367 m to 0.0646 m, which is almost doubled. This confirms that the Linear Segment controller is very accurate at moderate speed, but its lateral tracking quality degrades as the motion becomes more aggressive.

The second comparison considers two MPCC runs. Their overlapped XY trajectories are shown in Figure 8.31, while the corresponding contouring and lag error metrics are presented in Figure 8.32. MPC-05 provides better geometric tracking, with $\mathrm{RMSE}_{e_c} = 0.0170$ m and $\max|e_c| = 0.0357$ m. MPC-06 is faster, completing the common segment in 21.53 s, but the contouring error increases to $\mathrm{RMSE}_{e_c} = 0.0330$ m, with a maximum contouring error of 0.0705 m. The lag error also increases significantly, from $\mathrm{RMSE}_{e_l} = 0.0666$ m to $\mathrm{RMSE}_{e_l} = 0.1105$ m. This indicates that the aggressive MPCC configuration favours path advancement, but introduces a larger mismatch between the robot motion and the virtual path coordinate.

The final cross-comparison evaluates the aggressive Linear Segment run LIN-05 against MPC-06 over the same 4.534 m common segment. The overlapped XY trajectories are shown in Figure 8.33, while the contouring and lag error metrics are reported in Figure 8.34. In this case, MPC-06 achieves better lateral path-following accuracy:

$$\mathrm{RMSE}_{e_c} = 0.0330 \text{ m} \quad \text{against} \quad 0.0469 \text{ m}$$

for LIN-05. The maximum contouring error is also lower for MPC-06:

$$\max|e_c| = 0.0705 \text{ m} \quad \text{against} \quad 0.1166 \text{ m}.$$

However, the lag error remains much larger for MPC-06 ($\mathrm{RMSE}_{e_l} = 0.1105$ m) than for the Linear Segment controller ($\mathrm{RMSE}_{e_l} = 0.0011$ m). Therefore, the MPCC shows better

lateral geometric tracking in the aggressive comparison, but still requires further tuning of the lag penalty and path advancement dynamics. The Linear Segment controller remains simpler and very effective when its velocity is moderate, while the MPCC becomes more promising when higher-speed geometric path following is required.

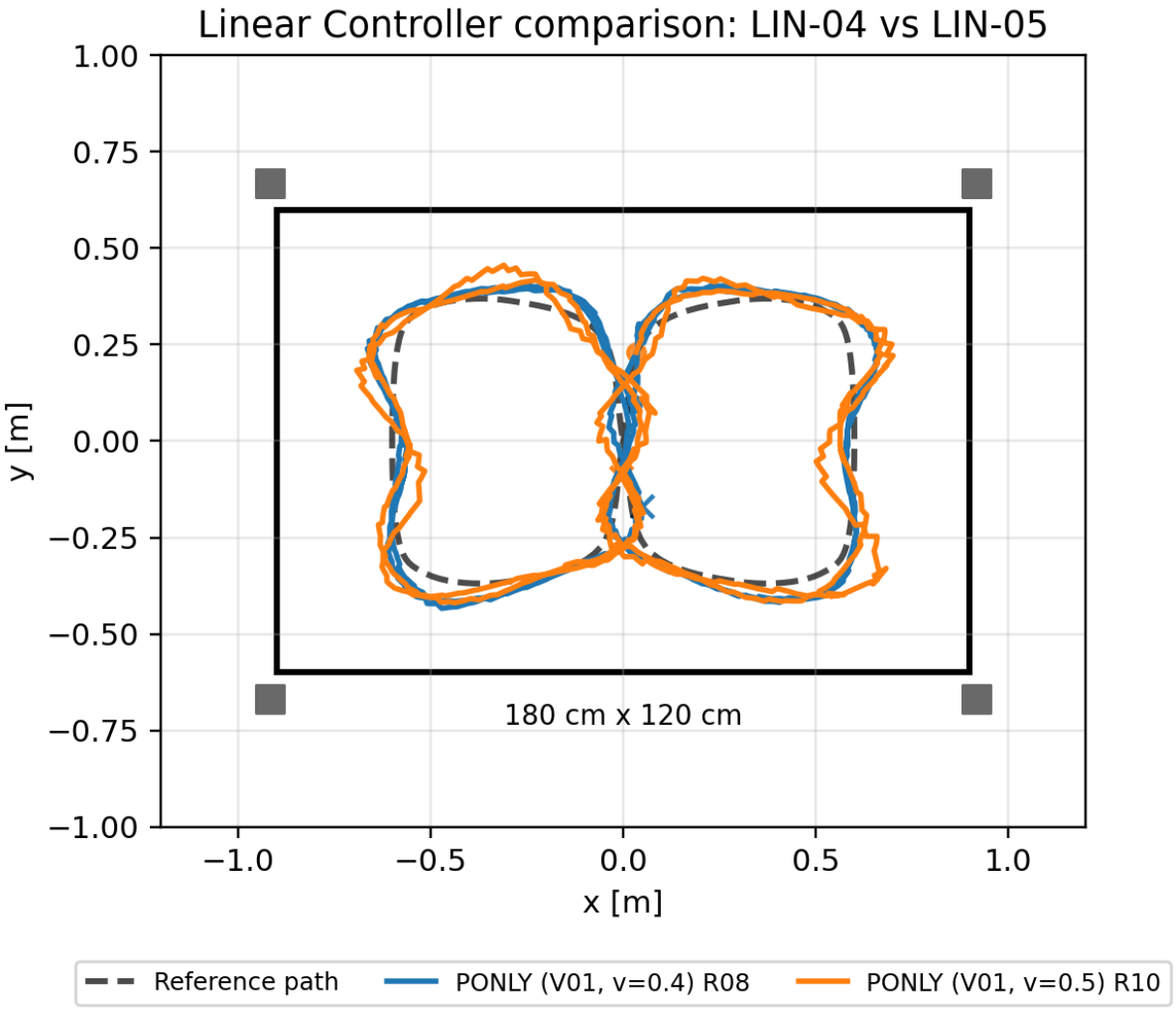


Figure 8.29: Overlapped xy trajectories for the LIN-05 and LIN-06 comparison.

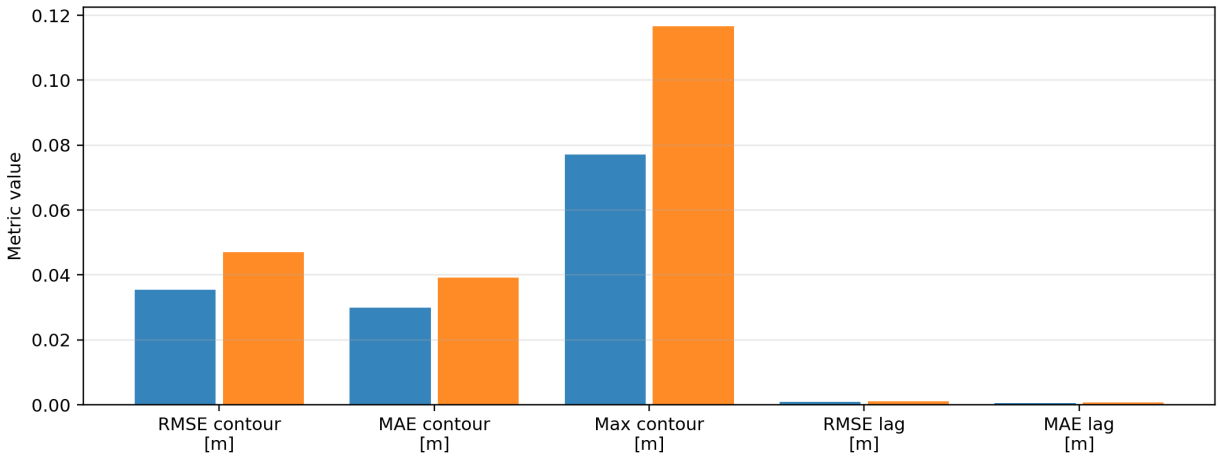


Figure 8.30: Contouring and lag error metrics for the LIN-05 and LIN-06 comparison.

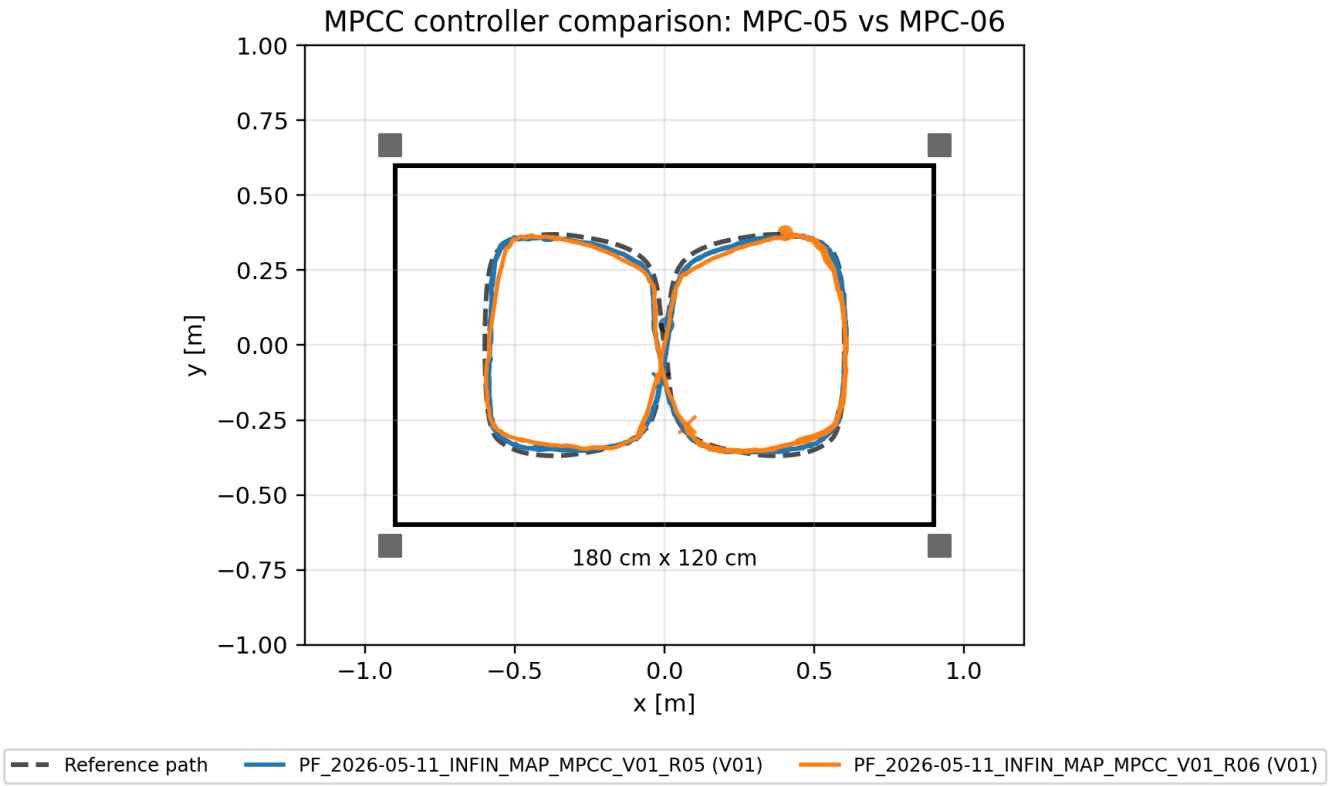


Figure 8.31: Overlapped xy trajectories for the MPC-05 and MPC-06 comparison.

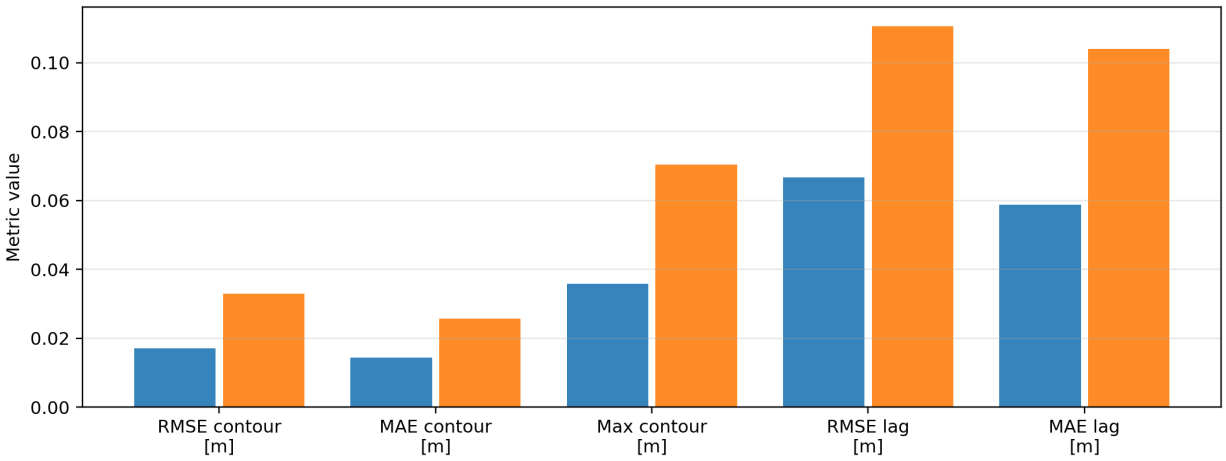


Figure 8.32: Contouring and lag error metrics for the MPC-05 and MPC-06 comparison.

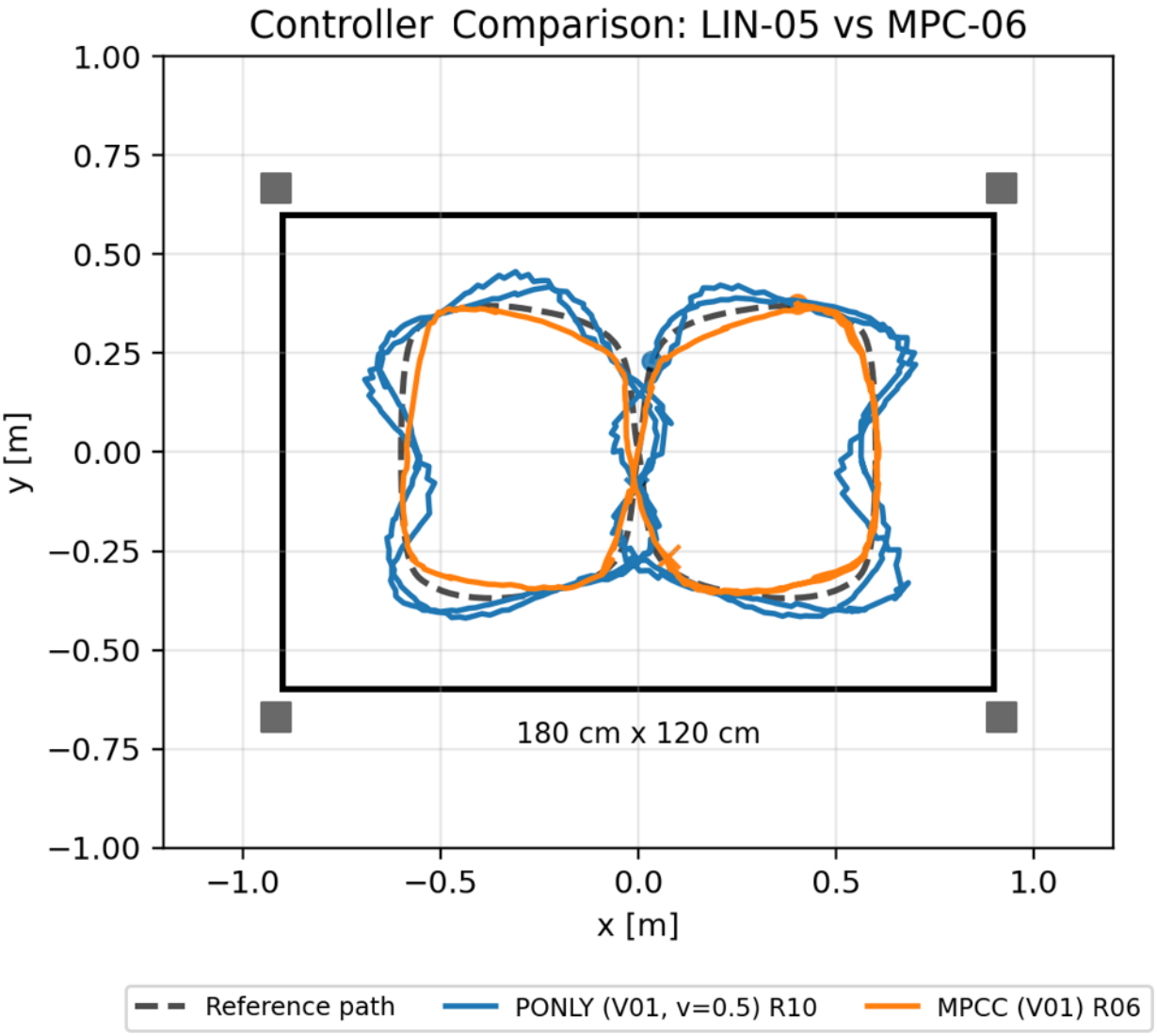


Figure 8.33: Overlapped xy trajectories for the LIN-05 and MPC-06 comparison.

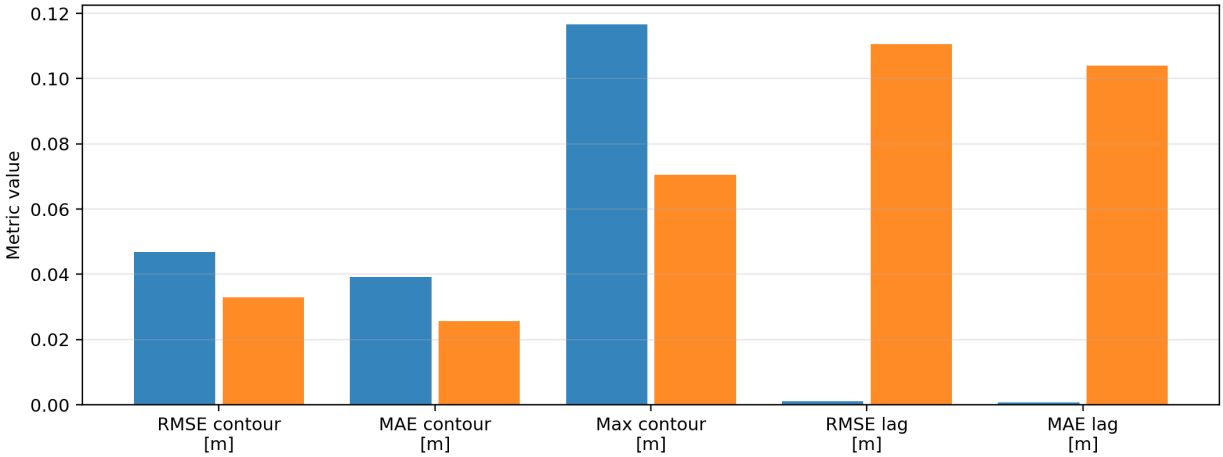


Figure 8.34: Contouring and lag error metrics for the LIN-05 and MPC-06 comparison.

Commands for compare records

```
    --tests \
      PF_2026-04-29_INFIN_MAP_LINEAR_V01_R08 \
      PF_2026-04-29_INFIN_MAP_LINEAR_V01_R10 \
    --results-root Test/test_results \
    --title "Linear Controller comparison: LIN-04 vs LIN-05"
```

Chapter 9

Conclusions

This thesis presented the redesign, implementation, and experimental validation of the OMNICAR omnidirectional robotic platform. The work covered the complete development chain, from the mechanical and electrical design of the vehicle to the embedded firmware, ROS2 integration, and high-level motion-control algorithms.

The first part of the work focused on the physical realization of the platform. A modular two-layer chassis was designed and manufactured through 3D printing, with dedicated supports for the motors, battery, Raspberry Pi 5, ESP32, lidar. The electrical architecture was designed to separate power and logic signals, while maintaining accessibility for debugging and maintenance. The assembly and validation phases confirmed the correct operation of the power distribution, motor drivers, PWM generation, and encoder signal conditioning.

At the embedded level, the ESP32 firmware was developed using PlatformIO and a modular C++ structure. The motor and encoder interfaces, centralized hardware configuration, OTA update support, and logging tools provided a reliable basis for experimental testing. Open-loop experiments allowed the motors to be approximated as first-order systems, while closed-loop tests showed that a conservative PI controller operating at 50 Hz provided stable wheel-speed tracking. The experiments also showed that higher control frequencies increased velocity-estimation noise due to encoder quantization and timing jitter.

The high-level software was migrated from ROS1 to ROS2 Jazzy and deployed on the Raspberry Pi 5. The resulting architecture separated the serial driver, joystick interface, odometry, localization, lidar integration, and motion-control nodes into modular ROS2 packages. Experimental tests confirmed the correct operation of the communication chain between ROS2 and the ESP32, the consistency of encoder feedback, and the feasibility of joystick teleoperation, localization visualization, and waypoint regulation. The go-to-point validation demonstrated stable closed-loop behaviour, with residual positioning errors mainly associated with odometric drift, wheel slip, and pose-estimation inaccuracies.

The final part of the thesis evaluated different high-level path-following controllers. The Moving Reference proportional controller provided a simple and stable baseline,

but its performance depended strongly on the balance between path progression and feedback gains. The Linear Segment controller achieved the best accuracy at moderate speed, with the LIN-02 configuration reaching low position errors and almost complete path progression without velocity saturation. At higher speeds, however, its performance degraded due to increasing command saturation and reduced correction margin.

A first Nonlinear Model Predictive Contouring Controller was also introduced and tested. The MPCC formulation made it possible to explicitly balance contouring error, lag error, path progression, command effort, and command smoothness over a prediction horizon. The experimental results showed that moderate MPCC tunings could complete almost the entire path, while aggressive tunings improved lateral geometric tracking compared with an aggressive Linear Segment configuration. However, the MPCC also introduced a larger lag error, showing that further tuning of the path-advancement dynamics and cost weights is required.

Overall, the work demonstrates that OMNICAR is now a functional experimental platform for omnidirectional motion-control research. The robot can be assembled, commanded, monitored, and tested through an integrated embedded and ROS2 control architecture. The experiments also identify the main remaining limitations: localization drift, wheel slip, actuator saturation at high speed, and the simplified prediction model used by the first MPCC implementation.

Future work should focus on improving the accuracy and robustness of the state estimation chain, including better odometric calibration and tighter integration between lidar-based localization and wheel feedback. On the control side, the Linear Segment controller could be extended with adaptive velocity scheduling, while the MPCC should be improved by using a continuous nonlinear optimizer, a full control sequence over the horizon, curvature-dependent speed limits, and explicit compensation for actuator delay and wheel slip. Finally, integrating obstacle constraints and local planning would allow the platform to move from path-following experiments toward complete autonomous navigation in structured indoor environments.

Bibliography

[1] Roland Siegwart, Illah Reza Nourbakhsh, and Davide Scaramuzza. *Introduction to Autonomous Mobile Robots*. 2nd ed. MIT Press, 2011.

[2] Bruno Siciliano et al. *Robotics: Modelling, Planning and Control*. Springer, 2009.

[3] Ricardo B. Sousa et al. "A Robotic Framework for the Robot@Factory 4.0 Competition". In: *2024 IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)*. 2024, pp. 66–73. DOI: 10.1109/ICARSC61747.2024.10535935.

[4] Sharan Srinivas, Surya Ramachandiran, and Suchithra Rajendran. "Autonomous Robot-Driven Deliveries: A Review of Recent Developments and Future Directions". In: *Transportation Research Part E: Logistics and Transportation Review* 165 (2022), p. 102834. DOI: 10.1016/j.tre.2022.102834.

[5] Bc507. *OCME AURIGA LGV.jpg*. Wikimedia Commons, licensed under CC BY-SA 3.0 / GFDL. Accessed: 2026. 2014. URL: https://commons.wikimedia.org/wiki/File:OCME_AURIGA_LGV.jpg.

[6] Oregon State University. *MGL0050.jpg*. Wikimedia Commons, licensed under CC BY-SA 2.0. Accessed: 2026. 2020. URL: https://commons.wikimedia.org/wiki/File:MGL0050.jpg.

[7] Oh-V1G0-Rally. *Omnicar Control ROS2 Repository*. Accessed: 2026. 2026. URL: https://github.com/Oh-V1G0-Rally/Omnicar_Control_Ros2.

[8] Open Robotics. *ROS 2 Documentation*. Accessed: 2026. 2026. URL: https://docs.ros.org/.

[9] José Pedro Carvalho, António Paulo Moreira, and A. Pedro Aguiar. "Model Predictive Control for B-Spline Trajectory Tracking in Omnidirectional Robots". In: *2024 7th Iberian Robotics Conference (ROBOT)*. 2024, pp. 1–7. DOI: 10.1109/ROBOT61475.2024.10797459.

[10] James B. Rawlings, David Q. Mayne, and Moritz Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017.

[11] Mrmw. *Mecanum Wheel Control Principle*. Wikimedia Commons, CC0 1.0 Universal Public Domain Dedication. Accessed: 2026. 2020. URL: https://commons.wikimedia.org/wiki/File:Mecanum_wheel_control_principle.svg.

[12] MathWorks. *What Is Model Predictive Control?* MATLAB Help Center, The MathWorks, Inc. Accessed: 2026. 2026. URL: https://www.mathworks.com/help/mpc/gs/what-is-mpc.html.

[13] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005.

[14] Espressif Systems. *ESP32 Series Datasheet*. Accessed: 2026. 2026. URL: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.

[15] Raspberry Pi Ltd. *Raspberry Pi 5 Product Brief*. Accessed: 2026. 2026. URL: https://datasheets.raspberrypi.com/rpi5/raspberry-pi-5-product-brief.pdf.

[16] Ian Gibson, David Rosen, and Brent Stucker. *Additive Manufacturing Technologies: 3D Printing, Rapid Prototyping, and Direct Digital Manufacturing*. 2nd ed. Springer, 2015.

[17] PlatformIO. *What Is PlatformIO?* Accessed: 2026. 2026. URL: https://docs.platformio.org/en/stable/what-is-platformio.html.

[18] Lennart Ljung. *System Identification: Theory for the User*. Prentice Hall, 1999.

[19] Paolo Bolzern, Riccardo Scattolini, and Nicola Schiavoni. *Fondamenti di Controlli Automatici*. IV. McGraw-Hill Education, 2017.

[20] Karl Johan Åström and Tore Hägglund. *PID Controllers: Theory, Design, and Tuning*. 2nd ed. Research Triangle Park, North Carolina: ISA - The Instrumentation, Systems and Automation Society, 1995.

[21] Steven Macenski et al. "Robot Operating System 2: Design, Architecture, and Uses in the Wild". In: *Science Robotics* 7.66 (2022), eabm6074. DOI: 10.1126/scirobotics.abm6074.

[22] Denise Lam, Chris Manzie, and Malcolm C. Good. "Multi-axis Model Predictive Contouring Control". In: *International Journal of Control* 86.8 (2013), pp. 1410–1424. DOI: 10.1080/00207179.2013.770170.

Acknowledgements

Desidero innanzitutto ringraziare il professor Visioli per la presenza costante e il supporto che mi ha offerto durante il mio percorso universitario. Mi ha accompagnato in alcune delle esperienze più importanti della mia formazione, dal primo Erasmus alla tesi triennale, fino allo svolgimento all'estero del lavoro di tesi magistrale.

Lo ringrazio per la grande disponibilità, per la fiducia che ha sempre riposto in me e per avermi proposto attività ed esperienze durnte il corso degli anni. Il suo incoraggiamento mi ha permesso di costruire un profilo professionale più completo, ma soprattutto di crescere dal punto di vista personale, spingendomi a mettermi continuamente alla prova senza fermarmi davanti alle difficoltà.

Ringrazio il professor António Pedro Aguiar e il dottorando José Pedro Carvalho per avermi seguito e supportato durante lo svolgimento della mia tesi magistrale.

Un ringraziamento particolare va a te, José, per la tua costante presenza e per la sincera amicizia con cui mi hai accompagnato in questi mesi. Ti auguro di raggiungere tutti gli obiettivi che ti sei posto e di ottenere le soddisfazioni che meriti.

Ringrazio i miei genitori, che mi hanno sopportato senza mai lamentarsi delle mie pazzie, permettendomi di vivere tutte le esperienze che desideravo, a volte a modo mio e altre volte sostenendomi anche quando una scelta sembrava giusta soltanto dal mio punto di vista. Grazie per avermi permesso di investire su me stesso e sulla mia formazione, lasciandomi libero nelle decisioni e continuando a fidarvi di me. Spero che ciò che riuscirò a costruire in futuro possa ripagare tutti gli sforzi che abbiamo affrontato insieme.

Ringrazio le mie Cucciole, ovvero le mie sorellone Vanessa e Greta, le persone che mi sopportano da più tempo e che sanno bene quanto il loro fratellino possa essere un po' folle. So però che mi avete sempre accettato così, con tutta la mia pazzia. Vi ringrazio per esserci sempre state e per avermi aiutato ogni volta che ne avevo bisogno. Siete le mie ancore in un mare agitato e il porto che cerco quando perdo la rotta. Soprattutto nei momenti più delicati della mia vita personale e nelle questioni di cuore, siete sempre state il mio punto di riferimento.

Ringrazio i miei nipotini, Leonardo, Matilde e Lorenzo. In voi ritrovo quella spensieratezza, e quella naturale capacità di vivere senza preoccupazioni che appartengono ai bambini e che, crescendo, troppo spesso si dimenticano. Passare del tempo con voi mi permette di riscoprirle e di tornare, almeno per un momento, a guardare il mondo con la vostra semplicità. Siete una delle mie più grandi fonti di serenità e divertimento.

Grazie Leo per aver assecondato lo zio Ale in tutte le sue avventure, anche nei film

che ti ho fatto guardare quando non ti piacevano, e per avermi fatto compagnia durante tutti questi anni. Mentre tu crescevi, in qualche modo sono cresciuto anch'io insieme a te, imparando a essere uno zio e una persona migliore. Spero di averti trasmesso almeno una parte dei miei valori e dei miei ideali e che, crescendo, tu possa farli tuoi a modo tuo.

Ringrazio tutta la mia famiglia: i miei adorati zii, le mie zie e i miei cugini. Siamo una famiglia tanto grande quanto speciale, capace di far sentire sempre la propria vicinanza, anche quando siamo lontani. Grazie per esservi sempre interessati al mio percorso, ai miei viaggi e a tutto ciò che facevo, per tutte le chiamate fatte semplicemente per sapere come stessi e per le volte in cui avete saputo confortarmi e darmi forza. Ogni occasione in cui riusciamo a riunirci tutti è sempre qualcosa di bellissimo e mi ricorda quanto sia fortunato ad avervi.

Ringrazio i miei zii Alberto V. e Luigi V., il diavolo e l'angelo sulle mie spalle. Siete entrambi sempre presenti, interessati a tutto ciò che faccio e pronti ad ascoltarmi e supportarmi senza mai giudicarmi.

Ringrazio Pacio, Ishak, Fusi, Marita, Zano, Dema, Mazze e tutto il gruppo. Nonostante i mille impegni, i viaggi e il tempo trascorso lontani, ogni volta che sono tornato mi avete sempre fatto sentire a casa. Con voi sembra che il tempo non sia mai passato e che si possa ripartire immediatamente da dove ci eravamo lasciati, conservando soltanto i momenti migliori.

Ringrazio tutti gli amici dell'Emi e gli Automatici, che mi hanno accompagnato durante questo percorso. Siete un gruppo unico e ormai abbiamo persino i posti assegnati su quelle bellissime sedie dell'aula studio. Avete affrontato insieme a me tutte le sessioni d'esame e vi ringrazio per avermi aspettato per mangiare, anche quando ero io a non aspettare voi, e per avermi sempre permesso di andare in ufficio per primo.

Grazie Gilbo, Kova, Cometti, Gio, Dani, Max, Dada, Elisa F., Eliza Z., Pezzo, Ale R., Riccardo e Wen.

Ringrazio gli amici di Casa Candido, conosciuti a Porto durante il mio ultimo semestre di laurea magistrale. Nicola, Silvia, Kati, Marta e Sofia, siete stati la mia famiglia. Questa avventura era iniziata, per me, con la pioggia e con la tristezza di sentirmi solo, ma si è conclusa con la felicità di aver incontrato persone speciali, capaci di accettarsi nelle proprie follie e di divertirsi insieme. Con voi ho condiviso tantissimi momenti speciali e vi ringrazio perché, durante questi mesi, siete stati la mia casa e il mio punto di riferimento. Grazie, in particolare, per aver ascoltato tutti i miei sfoghi e per i consigli che mi avete dato. È davvero bello quando riusciamo a sentirci tutti al telefono: anche se siamo lontani, vi sento sempre vicini. Spero, anzi credo, che continueremo a far parte gli uni della vita degli altri, senza perdere il legame che abbiamo costruito. Grazie per tutte le avventure, i viaggi e i momenti vissuti insieme. Spero che, nei mesi trascorsi insieme, anche la mia presenza vi abbia lasciato qualcosa, oltre al ricordo della mia singolare risata e della mia ormai ben nota ossessione per i soldi.

Per ultimi, ma certamente non per importanza, ringrazio i miei più grandi compagni di questa avventura: Toni, Robi, Albi, Samu, Paolo e Davidino. Con voi ho vissuto questi anni, dentro e fuori dall'università, con la serenità di sapere di avere sempre qualcuno

pronto ad aiutarmi, ad ascoltare i miei sfoghi e, soprattutto, a non mandarmi a quel paese. Grazie per aver sopportato anche le mie idee di viaggio, le giornate in cui vi facevo camminare per venticinque chilometri e le notti in cui, pur di non spendere due euro per un taxi, vi costringevo a tornare a piedi. Grazie per aver organizzato, condiviso e vissuto con me tutte queste esperienze.

Ringrazio Toni per essere stato il mio socio in tutto quello che ho fatto durante questa esperienza. Sei stato la metà che mi ha sempre assecondato e non mi ha mai abbandonato in nessuno dei progetti nei quali ci siamo buttati. Grazie per non farmi mai stare sugli allori e per spronarmi a migliorare sempre. I tuoi consigli sono oro pesato che, per fortuna, non devo pagare.

Grazie Robi per la tua sincerità e per la tua forza da Leoncino. A volte abbai, ma sei saggio e sai consigliare la cosa giusta al momento giusto. Durante questi anni mi hai reso fiero di te, perché ho visto una persona capace di spingersi fuori dalla propria comfort zone per crescere e maturare.

Grazie al mio fratellino di studio Albi, un guerriero pronto a diventare generale. Ti stimo e continuerò a stimarti per la tua determinazione e per la forza con cui insegui i tuoi obiettivi, senza mai lamentarti e senza lasciare che qualcuno possa fermarti. In questi anni ho visto nascere una macchina pronta a percorrere il giro migliore e a superare ogni ostacolo. Grazie per tutte le volte in cui mi chiami: è sempre bello sapere che ci sei e che non ti dimentichi di nessuno di noi. Spero che tu riesca a trovare il tuo posto e la tua strada a casa, in Albania, e ti auguro il meglio. Sai già che ogni tanto mi autoinviterò a trovarti.

Grazie Samu. Mi dispiace per Kova, ma tu sei il mio informatico preferito e il nostro pilota migliore. Ti ringrazio per avermi ascoltato ogni volta che ti raccontavo qualche stupidata che volevo fare e per esserti messo a ridere, forse per non piangere. Mi raccomando, non mollare. So che a volte ti sembra di vivere su delle montagne russe perennemente in discesa perché, diciamocelo, non sei fortunatissimo. So però che sei estremamente competente e che dovresti credere molto di più in te stesso e nelle tue capacità. Hai la stessa determinazione che accomuna tutti noi e spero che tu non ti spenga in qualche ufficio triste, ma che continui a puntare sempre più in alto, fino a trovare anche il tuo giro della morte.

Grazie Paolo. Da dietro le quinte hai sempre gestito il gruppo: sei il più silenzioso, ma anche il più letale di tutti. Se ognuno di noi ha qualcosa di folle che lo caratterizza, tu hai raccolto un po' della follia di tutti e te la porti dietro in silenzio. Spero che prima o poi riusciremo a scoprire qualcosa della tua vita privata, anche se già sapere dove abiti significa tanto.

Grazie Davidino. Anche se non sei stato propriamente un nostro "collega" di studio, sei stato un grande amico e compagno di viaggio. In questi due anni abbiamo viaggiato entrambi moltissimo: tu hai vissuto esperienze assurde, senza però perdere mai le tue immancabili occhiaie. Anche rivedendoti dopo tanto tempo, ho sempre ritrovato lo stesso Davidino estremamente sereno e in pace con il mondo. Con una gentilezza e una sincerità che sembrano non avere limiti, sei capace di conquistare chiunque incontri.

In ognuno di voi ho visto una grande crescita e so che tutti noi siamo in grado di

conquistare il mondo grazie alle nostre capacità e alle caratteristiche che ci rendono unici. Forse, viste dall'esterno, alcune di esse potrebbero sembrare pura follia, ma tranquilli: bisogna essere folli per fare cose grandi.

Se vedete che ho dimenticato qualcuno, sapete che per me è normale, quindi non offendetevi.

Ringrazio ancora tutti, perché ognuno di voi mi ha lasciato qualcosa e ha contribuito a farmi diventare ciò che sono oggi: un folle sognatore.